IN410 appunti

Filippo Rossi

A.A. 2026/2027

Questo documento raccoglie gli appunti del corso di IN410 tenuto dal prof. Marco Pedicini. È costruito a partire dalle note fornite dal professore, integrate con gli appunti presi a lezione, ed è ancora in fase di scrittura. Ogni errore è da attribuire unicamente a me: se ne trovate, scrivetemi!

Indice

1 Introduzione **3**
1.1 Alfabeti e parole . . . 3
1.2 Algoritmo e decidibilita' . . . 3

2 Automi a stati finiti **6**
2.1 Definizione di automa a stati finiti. . . . 6
2.2 Definizione di algoritmo associato ad un automa . . . 6

3 Rappresentazione grafica degli automi **9**
3.1 Automi come grafi . . . 9
3.2 Matrici di adiacenza per grafi di automi finiti . . . 10

4 Linguaggi regolari **18**

5 Automi non deterministici **20**
5.1 Automi NDFA . . . 20
5.2 Automi ϵ-NDFA . . . 24
5.3 Chiusura per concatenazione e star di Kleene . . . 28

6 Espressioni regolari **33**
6.1 Sintassi ed interpretazione . . . 33
6.2 Automi non deterministici generalizzati . . . 35
6.3 Da automa ad espressione regolare (e viceversa) . . . 37

7 Macchina di Turing **45**
7.1 Configurazione e algoritmo associato . . . 45
7.2 Decidibilità per macchina di Turing . . . 48

8 Teoria della complessità **51**
8.1 Classi di complessità . . . 51
8.2 Look-up Table . . . 55
8.3 Relazioni tra le classi DTIME e DSPACE . . . 58
8.4 Pumping lemma . . . 59

9 Macchine a seminastro e multinastro **63**
9.1 Simulazione di algoritmi . . . 63
9.2 Macchine a seminastro . . . 64
9.3 Macchine multinastro . . . 68
9.4 Equivalenza tra macchine a multinastro e macchine a nastro singolo . . . 71
9.5 Cambiamenti di alfabeto . . . 75
9.6 Teorema di Speed-up . . . 81
9.7 Macchine Speciali . . . 86
9.8 Rappresentazione degli interi . . . 87

10 Funzioni ricorsive **91**
10.1 Funzioni Turing-computabili . . . 91
10.2 Funzioni ricorsive . . . 95

11 Prerequisiti **101**
11.1 Definizioni Preliminari . . . 101
11.2 Grafi e Matrici di Adiacenza . . . 101
11.3 Grafi Etichettati . . . 102
11.4 Semianelli e Serie Formali . . . 102
11.5 Matrici su Semianelli . . . 104
11.6 Monoidi, Serie Formali e Algebre di Kleene . . . 105
11.6.1 Monoidi . . . 105
11.6.2 Serie formali . . . 105
11.6.3 Algebra di Kleene . . . 107

12 Wolfram **108**
12.1 Introduzione . . . 108
12.2 Entropia . . . 111
12.3 Libreria TM-lib . . . 113
12.4 Automi . . . 114

13 Esercizi **116**
13.1 Automi . . . 116
13.2 Macchine di Turing . . . 129

14 Esoneri ed esami **137**
14.1 Esonero 1 - 13/11/2024 . . . 137
14.2 Esonero 1 - 10/11/2023 . . . 143
14.3 Esonero 2 - 08/11/2022 . . . 144

1 Introduzione

1.1 Alfabeti e parole

Definizione 1.1 (Insieme delle parole su un alfabeto)**.** Fissato un alfabeto A, si definisce *l'insieme delle parole definite su* A come l'insieme

$$A^* := \bigcup_{k \geq 0} A^{\{1,\dots,k\}}.$$

dove $A^{\{1,\dots,k\}} := \{f \text{ è una funzione} \mid f : \{1,\dots,k\} \to A\}$.

Si noti che per convenzione l'insieme $\{1,\dots,k\}$ per $k = 0$ corrisponde all'insieme vuoto. E pertanto l'insieme A^* per definizione contiene anche la funzione $A^\emptyset$ che è detta la *parola vuota*.

Definizione 1.2 (Lunghezza di una parola)**.** La *lunghezza* di una parola (in simboli $|u|$) è definita come il più piccolo n tale che $u \in A^{\{1,\dots,n\}}$.

Definizione 1.3 (Concatenazione di parole)**.** La *concatenazione di due parole* u *e* v è una funzione $uv \in A^{\{1,\dots,|u|+|v|\}}$ (indicata a volte anche come $u \cdot v$) definita come segue:

$$uv(k) := \begin{cases} u(k) & \text{se } k \leq |u| \\ v(k - |u|) & \text{se } |u| + 1 \leq k \leq |u| + |v| \end{cases}$$

Osservazione 1.4. Equivalentemente, possiamo introdurre l'insieme delle parole su A, con A^*, come l'insieme delle successioni finite di elementi di A (compresa la parola vuota). Se $u, v \in A^*$ allora $uv \in A^*$ indicherà la parola ottenuta concatenando u con v. Data l'equivalenza delle definizioni, useremo una piuttosto che l'altra in modo completamente arbitrario. Inoltre, visto che vale $A \hookrightarrow A^*$, considereremo A come un suo sottoinsieme a tutti gli effetti.

Osservazione 1.5. La tripla $(A^*, A^\emptyset, \cdot)$ è un monoide.

1.2 Algoritmo e decidibilita'

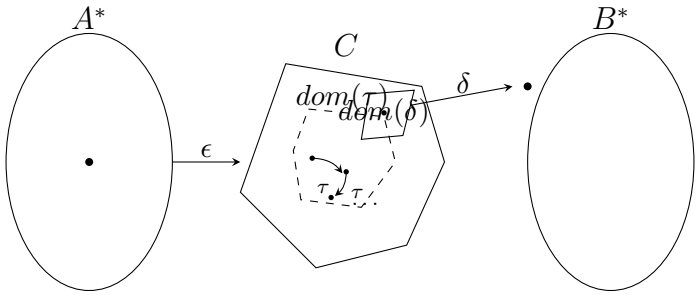
Definizione 1.6 (Algoritmo)**.** Un *algoritmo* è una sestupla $\mathcal{A} := (A, B, C, \epsilon, \tau, \delta)$ dove A e B sono insiemi chiamati rispettivamente *alfabeto di entrata* e *alfabeto di uscita*, C è un insieme chiamato *spazio di configurazioni possibili* e tre funzioni

$$\epsilon : A^* \to C$$
$$\tau : C \to C$$
$$\delta : C \to B^*$$

tali che $\{dom(\tau), dom(\delta)\}$ è una partizione di C. La *computazione associata ad una parola* $w \in A^*$ è la successione

$$(\tau^n(\epsilon(w)))_{n \in \mathbb{N}} = (\epsilon(w), \tau(\epsilon(w)), \tau^2(\epsilon(w)), \dots)$$

diremo che questultima è *finita* (oppure diremo che *termina*) se esiste un $n \in \mathbb{N}$ tale che $\tau^n(\epsilon(w)) \in dom(\delta)$, altrimenti diremo che è *infinita*. Se la sequenza termina in una configurazione $\tau^n(\epsilon(w))$ allora avremo che il *risultato della computazione* è $\delta(\tau^n(\epsilon(w)))$. Se invece la computazione associata non termina si dirà che l'algoritmo *non termina partendo da* w.



Definizione 1.7 (Funzione associata ad un algoritmo)**.** Sia $\mathcal{A} := (A, B, C, \epsilon, \tau, \delta)$ un algoritmo. Possiamo definire la funzione

$$f_{\mathcal{A}}: \begin{array}{ccc} A^* & \to & B^* \\ w & \mapsto & \begin{cases} \delta(\tau^n(\epsilon(w))) & \text{se } n \text{ è il più piccolo intero} \\ & \text{per cui } \tau^n(\epsilon(w)) \in dom(\delta)\ , \\ \perp & \text{se } \mathcal{A} \text{ non termina partendo da } w \end{cases} \end{array}$$

che chiameremo *funzione associata all'algoritmo* $\mathcal{A}$.

In pratica, ogni volta che diremo che la funzione f associata ad un certo algoritmo formale è definita per ogni possibile parola $w \in A^*$, intenderemo dire che per ogni possibile input la computazione termina e restituisce attraverso la funzione di output δ il valore della funzione $f(w)$.

Le due precedenti definizioni sono state date nel modo più generale possibile. D'ora in poi invece, a meno di casi particolari, considereremo solo algoritmi tali che $B = \{0, 1\}$.

Definizione 1.8 (Algoritmo che decide un insieme)**.** Diremo che un certo *algoritmo decide* un insieme $X \subset A^*$ quando la funzione associata all'algoritmo coincide su ogni elemento con la funzione caratteristica di X, quindi se vale:

$$f_{\mathcal{A}}(w) = \begin{cases} 1 & w \in X \\ 0 & w \notin X. \end{cases}$$

Osservazione 1.9. Se un algoritmo $\mathcal{A}$ decide un insieme X, allora abbiamo chiaramente che $f_{\mathcal{A}}$ e' una funzione totale. In altre parole: quando un algoritmo decide un insieme, la computazione dell'algoritmo a partire da una qualsiasi parola di alfabeto A termina.

Definizione 1.10 (Algoritmo che semidecide un insieme)**.** Diremo che un certo algoritmo *semidecide* un insieme $X \subseteq A^*$ quando la funzione parziale associata all'algoritmo $f_{\mathcal{A}}$ è definita esattamente sugli elementi di X e, quando è definita, restituisce il valore 1. In altre parole:

$$f_{\mathcal{A}}(x) = \begin{cases} 1, & \text{se } x \in X, \\ \perp, & \text{se } x \notin X, \end{cases}$$

Chiaramente, uno dei due concetti e/' più forte dell'altro.

Proposizione 1.11. Se esiste un algoritmo $\mathcal{A}$ che decide un un iniseme $X \subset A^*$ su un alfabeto A; allora esiste un algortmo $\mathcal{A}'$ che semidecide X.

Dimostrazione. Per ipotesi abbiamo un algoritmo $\mathcal{A} = (A, \{0,1\}, C, \epsilon, \tau, \delta)$ che decide X. Possiamo considerare

$$T_0 := \delta^{-1}(\{0\}), \quad T_1 := \delta^{-1}(\{1\})$$

in modo tale da poter definire le seguenti funzioni:

$$\begin{array}{rccc} \tau'(c): & C & \to & C \\ & c & \mapsto & \begin{cases} c, & \text{se } c \in T_0 \\ \tau(c), & \text{se } c \notin T_0 \end{cases} \end{array}$$

$$\begin{array}{rccc} \delta'(c): & C & \to & \{0,1\}^* \\ & c & \mapsto & \begin{cases} \delta(c), & \text{se } c \in T_1 \\ \bot, & \text{se } c \notin T_1 \end{cases} \end{array}$$

Possiamo definire quindi l'algoritmo $\mathcal{A} := (A, \{0,1\}, C, \epsilon, \tau', \delta')$. Ora, per ogni $w \in A^*$ esiste un $k \in \mathbb{N}$ tale che $\tau^k(\epsilon(w)) \in dom(\delta) = T_0 \sqcup T_1$. Questo vuol dire che, per ogni $i < k$ vale che $\tau^i(\epsilon(w)) \notin T_0$. Quindi $\tau'^k(\epsilon(w)) = \tau^k(\epsilon(w))$. A qeusto punto dividiamo in casi:

$w \in X$: allora $\tau'^k(\epsilon(w)) \in T_1$, quindi $\delta'(\tau'^k(\epsilon(w))) = \delta(\tau'^k(\epsilon(w))) = 1$;

$w \notin X$: allora $\tau'^k(\epsilon(w)) \in T_0$, quindi $\delta'(\tau'^k(\epsilon(w))) = \bot$, entrando cosi/' in un ciclo.

Quindi, $\mathcal{A}'$ semidecide X.

□

Il viceversa non vale, vedremo un esempio più avanti CONTROLLA

controlla qui: Pertanto, consideriamo i processi computazionali come metodi di decisione simbolici, in cui si rende automatico (per questo formalmente ineccepibile) il metodo di decisione di una proprietà. Le garanzie sulla correttezza della decisione vengono dalla verificabilità: il processo lavora su sequenze di simboli, e applica in maniera asettica alcune regole di trasformazione (i principi deduttivi) in maniera puntuale: da cui viene la richiesta specifica che le regole di trasformazione siano locali, applicabili in tempo finito e soprattutto in modo che la verifica della corretta applicazione sia finita.

In questo quadro, se anche il problema di decisione riguardasse un insieme di cardinalità più che numerabile, un processo computazionale finito potrebbe procedere modificando ed essendo influenzato solo da un numero finito di elementi. Quindi è la descrizione stessa del problema che ha senso solo quando è finita.

Con queste premesse la classe dei problemi più semplici che si possono porre è in prima istanza quella dei *problemi di decisione* per un fissato alfabeto $A = \{a_1, \ldots, a_n\}$. Essi consistono nella possibilità di definire un metodo per stabilire la separazione tra le parole di un sottoinsieme $X \subset A^*$ e quelle del complemento $A^* \setminus X$, procedura con le caratteristiche che abbiamo ricordato.

2 Automi a stati finiti

2.1 Definizione di automa a stati finiti.

Definizione 2.1. Siano A e Q insiemi finiti, un *automa finito deterministico* di alfabeto A e insieme degli stati Q è una tripla (T, q_0, F) dove

$$T : Q \times A \to Q$$

$F \subset Q$ e $q_0 \in Q$ è uno stato particolare, detto stato iniziale.

2.2 Definizione di algoritmo associato ad un automa

A partire dalla tripla (T, q_0, F) che specifica un dato automa di alfabeto A e insieme degli stati Q si costruisce prima l'insieme delle configurazioni possibili:

Definizione 2.2 (Configurazione per automa finito deterministico)**.** Una configurazione per automa a stati finiti di alfabeto A e insieme degli stati Q è una tripla (w, p, q), dove

1. $w \in A^*$ è la parola da analizzare,
2. $p \in \mathbb{N}$, è un intero che indica la posizione della parola da analizzare nella prossima transizione,
3. $q \in Q$ indica lo stato corrente e costituisce la memoria delle transizioni effettuate nei precedenti passi di computazione.

L'insieme delle configurazioni di alfabeto A e insieme degli stati Q per automa finito sarà denotato con

$$Conf_{DFA}^{A,Q} := A^* \times \mathbb{N} \times Q$$

ed allora possiamo definire:

Definizione 2.3 (Algoritmo associato ad un automa finito deterministico)**.** L'algoritmo associato ad un automa di alfabeto A e insieme degli stati Q è l'algoritmo

$$(A, Conf_{DFA}^{A,Q}, \{0,1\}, \epsilon, \tau, \delta)$$

dove le tre funzioni sono definite come segue:

- la **funzione di ingresso** che associa ad una parola $w \in A^*$, la configurazione $\varepsilon(w) = (w, 1, q_0)$,
- la **funzione di transizione** $\tau : Conf_{DFA}^{A,Q} \to Conf_{DFA}^{A,Q}$ è data da

$$\tau((w, p, q)) = (w, p + 1, T(q, w(p))),$$

- la **funzione di uscita** $\delta : Conf_{DFA}^{A,Q} \to \{0,1\}^*$, data da

$$\delta((w, p, q)) = \begin{cases} 0 & \text{se } p > |w| \text{ e se } q \notin F, \\ 1 & \text{se } p > |w| \text{ e se } q \in F. \end{cases}$$

Consideriamo quindi una parola $w = a_1 a_2 \cdots a_n \in A^*$ e un automa finito determenistico come sopra. Chiameremo *computazione* di w tramite l'automa, la seguente successione

$$(c_0, \ldots, c_{n+1})$$

dove

$$\begin{cases} c_0 := e(w) \\ c_{i+1} := \tau(c_i) \quad \text{se } i \geq 0 \end{cases}$$

Osservazione 2.4. L'idea dietro la definizione precedente e/' quella di leggere una lettera alla volta della parola w e tramite la funzione T cambiare lo stato della macchina in base alla lettera letta e allo stato corrente della macchina. Ovviamente la computazione finisce quando sono state esaminate tutte le lettere della parola (che vegnono esaminate esattamente una volta sola). Inoltre, un automa non ha una vera e propria memoria, bensi/' ha gli stati (che giocano il ruolo della memoria dell'automa).

Definizione 2.5 (Insiemi decidibili per automa)**.** Un insieme $X \subset A^*$, è *decidibile* per automa finito (oppure *DFA-decidibile* oppure *regolare*), se esiste un insieme finito Q e un automa finito con alfabeto A e insieme degli stati Q tale che l'algoritmo associato decide X.

Esempio 2.6. Vogliamo ora descrivere l'automa finito che decide l'insieme delle parole di lunghezza pari (risp. dispari) su un alfabeto dato A. A partire dall'alfabeto finito A, e da un insieme di due stati $Q = \{0, 1\}$ che corrisponderanno rispettivamente allo stato "abbiamo letto un numero pari (oppure dispari) di caratteri della parola"; costruiamo la funzione di transizione che ad ogni carattere letto cambia stato:

$$T : Q \times A \to Q$$

tale che

$$T(q, a) = \begin{cases} 1 & \text{se } q = 0 \\ 0 & \text{se } q = 1, \end{cases}$$

per ogni $a \in A$. Avremo allora che lo stato iniziale $q_0 = 0$, poiché inizialmente non avremo ancora letto nessun carattere e che l'insieme degli stati accettanti sarà: $F = \{0\}$. Sia quindi

$$X = \{\, w \in A^* : |w| \text{ è pari} \,\}.$$

e dimostriamo che l'algoritmo associato all'automa

$$(T, q_0, F)$$

decide l'insieme X. Sia dunque $w = a_1 a_2 \cdots a_n \in A^*$. La configurazione iniziale è

$$\epsilon(w) = (w, 1, 0).$$

Ad ogni applicazione della funzione di transizione viene letta una lettera della parola e lo stato viene invertito:

$$0 \longleftrightarrow 1.$$

Per induzione sul numero di transizioni effettuate si ottiene che dopo aver letto i primi k caratteri della parola la configurazione è

$$(w, k+1, q_k),$$

dove

$$q_k = \begin{cases} 0 & \text{se } k \text{ è pari,} \\ 1 & \text{se } k \text{ è dispari.} \end{cases}$$

Dopo la lettura dell'intera parola si raggiunge quindi la configurazione

$$(w, n+1, q_n).$$

Poiché

$$n + 1 > |w|,$$

questa appartiene al dominio della funzione di uscita. Inoltre

$$q_n = \begin{cases} 0 & \text{se } |w| \text{ è pari,} \\ 1 & \text{se } |w| \text{ è dispari.} \end{cases}$$

Essendo $F = \{0\}$, segue che

$$\delta(w, n+1, q_n) = \begin{cases} 1 & \text{se } |w| \text{ è pari,} \\ 0 & \text{se } |w| \text{ è dispari.} \end{cases}$$

Pertanto

$$f_{\mathcal{A}}(w) = \begin{cases} 1 & \text{se } w \in X, \\ 0 & \text{se } w \notin X, \end{cases}$$

cioè

$$f_{\mathcal{A}} = \mathbf{1}_X.$$

Ne consegue che l'algoritmo associato all'automa decide l'insieme delle parole di lunghezza pari.

3 Rappresentazione grafica degli automi

3.1 Automi come grafi

A partire da un automa finito (T, q_0, F) se ne può dare una descrizione grafica nel modo seguente: si consideri il grafo i cui nodi sono gli stati Q, e gli archi etichettati con elementi di A che collegano due stati (q_i, q_j) se esiste la transizione $T(q_i, a) = q_j$. inoltre avremo una funzione che ci indica lo stato iniziale

$$\texttt{init}(G_{(T,q_0,F)}) = q_0$$

(di solito indicata da un arco entrante nel nodo iniziale q_0) ed una che distingue gli stati accettanti (risp. non accettanti)

$$\texttt{acc}(G_{(T,q_0,F)}) = F \qquad (\text{risp. } \texttt{ref}(G_{(T,q_0,F)}) = Q \setminus F)$$

(di solito si indicano gli stati accettanti con un doppio cerchio, mentre tutti gli altri nodi sono considerati non accettanti). Per essere più formali:

Definizione 3.1 (Grafo associato ad un automa)**.** Dato un automa (T, q_0, F) definito su un alafabeto A e insieme di stati Q definiamo il suo grafo associato $G = (V, E, L)$ come segue:

- l'insieme dei vertici è
$$V := Q;$$
- l'insieme dei lati è
$$E := \{(q_i, a, q_j) \in Q \times A \times Q \mid T(q_i, a) = q_j\};$$
- la funzione di etichettatura è
$$L : E \to A, \qquad L(q_i, a, q_j) = a.$$

Ad esempio il grafo associato all'automa dell'Esempio 2.6, è rappresentato in Figura 1.

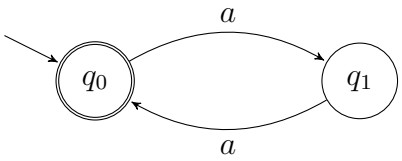


Figura 1: Automa dell'Esempio 2.6, notare che a è un qualsiasi elemento nell'alfabeto $A = \{0, 1\}$.

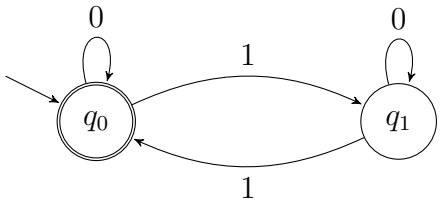


Figura 2: In modo simile all'automa dell'Esempio 2.6 si può dare l'automa che decide l'insieme di tutte le parole con un numero pari di 1.

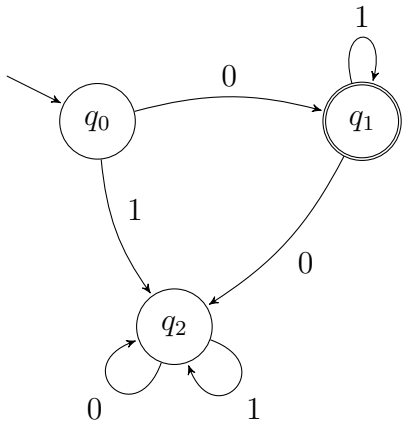


Figura 3: Automa che decide l'insieme $X = \{01^n \mid n \geq 0\}$.

3.2 Matrici di adiacenza per grafi di automi finiti

Consideriamo un automa (T, q_0, F) definito su un alafabeto A e insieme di stati Q. Possiamo sempre considerare il monoide $M = (A^*, A^\emptyset, \cdot)$ e l'anello (che e/' anche un semianello) $\mathbb{N}$. A questo punto possiamo definire l'insieme delle serie formali

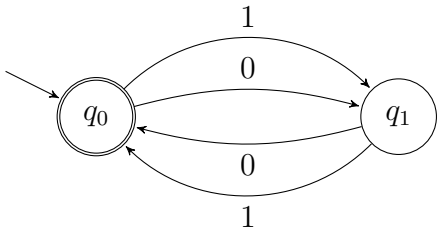
$$\mathbb{B}[M] := \left\{ \sum_{w \in A^*} n_w w \,|\, n_w \in \mathbb{B} \right\}$$

(in generale scrivieremo $\mathbb{B}[A^*]$ al posto di $\mathbb{B}[M]$ per indicare che il monoide e/' quello delle parole su un alfabeto A). Ora possiamo considerare la matrice di adiacenza Z dell'automa, che non sarà nient'altro che una matrice cha nelle entrate elementi di $\mathbb{B}[M]$:

$$Z_{ij} = \begin{cases} \sum_k a_k & \text{se } T(q_i, a_k) = q_j \\ \bot & \text{altrimenti} \end{cases}$$

Come sappiamo, fare la potenza k-esima di una matrice di adiacenza con etichette, fornisce nell'entrata (i, j) tutti e i soli cammini di lunghezza k che collegano il vertice i al vertice j (nel caso degli automi, dallo stato q_i allo stato q_j). Vediamo un esempio:

Esempio 3.2. Consideriamo l'automa che decide l'inisieme delle parole di lunghezza pari. Questo e/' rappresentato dal seguente grafo etichettato.



La sua matrice di adiacenza e/'

$$Z = \begin{pmatrix} \bot & 0+1 \\ 0+1 & \bot \end{pmatrix}$$

Infatti, gli archi 0 e 1 collegano i vertici q_0 e q_1 in entrambe le direzioni. Inoltre, visto che non ci sono loop, sulla diagonale si inserisce convenzionalmente la serie formale vuota, indicata con $\bot$. A questo punto vogliamo trovare i cammini di lunghezza 2 del grafo. Per farlo basta calcolare la potenza seconda di Z:

$$\begin{aligned} Z^2 &= \begin{pmatrix} \bot\bot + (0+1)(0+1) & \bot(0+1) + (0+1)\bot \\ (0+1)\bot + \bot(0+1) & (0+1)(0+1) + \bot\bot \end{pmatrix} \\ &= \begin{pmatrix} 00+01+10+11 & \bot+\bot \\ \bot+\bot & 00+01+10+11 \end{pmatrix} \\ &= \begin{pmatrix} 00+01+10+11 & \bot \\ \bot & 00+01+10+11 \end{pmatrix} \end{aligned}$$

Per verificare le operazioni effettuate con $\bot$, basta usare la definizione presente in (11.23). Dalla matrice possiamo quindi dedurre che esistono 4 cammini che collegano q_0 e q_1 con loro stessi, e nessuno che collega q_0 con q_1 o viceversa.

A questo punto ci vogliamo porre la seguente domanda: si può dedurre dala matrice di adiacenza tutti i cammini che uniscono lo stato iniziale ad uno stato accettante dell'automa? In altre parole: dalla matrice di adiacenza si può dedurre il calcolo che l'automa compie per decidere un insieme? La risposta e' si. In partiolare, e/' anche possibile dedurre l'intero insieme X da decidere. Prima di affrontare il caso generale analizziamo questo problema nel caso dell'esempio precedente.

Esempio 3.3. In questo caso esiste un solo stato accettante, che coincide esattamente con lo stato iniziale. Quindi, in una qualsiasi potenza di Z, dovremmo considerare l'entrata $(1,1)$ per trovare le parole accettate dall'automa. Più precisamente, per ogni k naturale, nell'entrata $(1,1)$ della matrice Z^k troviamo le uniche e sole parole di lunghezza k accettate dall'automa.

Lavoriamo ora al caso generale. Se con Z indichiamo la matrice di adiacenza di un automa, possiamo definire definiamo

$$E(Z) := \sum_{k=0}^{\infty} Z^k$$

dove Z^0 e/' la matrice cha ha nell'entrata entrata $(1,1)$ l'elemento $\perp$ e la serie vuota $\perp$ nelle altre entrate, quindi

$$Z^0_{i,j} = \begin{cases} A^\emptyset & \text{se } i = j = 1 \\ \perp & \text{altrimenti} \end{cases}$$

Assumendo che lo stato iniziale sia q_0 e che corrisponda alla prima riga della matrice Z, allora sulla prima riga di $E(Z)$ e nelle colonne corrispondenti agli stati accettanti troveremo le parole che l'automa accetta.

Esempio 3.4. Facendo ancora un paio di potenze di Z, si può dedurre la seguente formula:

$$Z^k = \begin{cases} \begin{pmatrix} (0+1)^k & \perp \\ \perp & (0+1)^k \end{pmatrix}, & \text{se } k \text{ e/' pari} \\ \\ \begin{pmatrix} \perp & (0+1)^k \\ (0+1)^k & \perp \end{pmatrix}, & \text{se } k \text{ e/' dispari} \end{cases}$$

quindi abbiamo

$$\begin{aligned} E(Z) &= \begin{pmatrix} A^\emptyset & \perp \\ \perp & \perp \end{pmatrix} + \begin{pmatrix} \perp & (0+1) \\ (0+1) & \perp \end{pmatrix} + \begin{pmatrix} (0+1)^2 & \perp \\ \perp & (0+1)^2 \end{pmatrix} + \cdots \\ &= \begin{pmatrix} A^\emptyset + \sum_{k=1}^{\infty}(0+1)^{2k} & \perp + \sum_{k=0}^{\infty}(0+1)^{2k+1} \\ \perp + \sum_{k=0}^{\infty}(0+1)^{2k+1} & \perp + \sum_{k=1}^{\infty}(0+1)^{2k} \end{pmatrix} \\ &= \begin{pmatrix} A^\emptyset + \sum_{k=1}^{\infty}(0+1)^{2k} & \sum_{k=0}^{\infty}(0+1)^{2k+1} \\ \sum_{k=0}^{\infty}(0+1)^{2k+1} & \sum_{k=1}^{\infty}(0+1)^{2k} \end{pmatrix} \end{aligned}$$

Per quanto osservato prima, nel posto $(1,1)$ della matrice $E(Z)$ troviamo le uniche e solo parole accettate dall'automa. Come si puo'/ osservare, la parola vuota e/' accettata. In effetti, ogni qual volta che uno stato iniziale e uno stato accettante coincidono, allora la parola vuota e/' accettata (questo non ci deve suonare strano, infatti l'automa non deve fare niente per accettarla).

Usando un po' di algebra lineare, e/' possibile esplicitare esattamente l'inisieme delle parole accettate. Infatti, fi possiamo definire i seguenti vettori:

$$V_0(i) = \begin{cases} A^\emptyset, & q_i = q_0 \\ \perp, & q_i \neq q_0 \end{cases}$$

$$V_F(i) = \begin{cases} A^\emptyset, & q_i \in F \\ \perp, & q_i \notin F \end{cases}$$

E calcolando il prodotto righe per colonne

$$V_0 \cdot E(Z) \cdot V_F^t$$

otteniamo la serie formale che ha come addendi esattamente le parole accettate da X. La formula appena enunciata prende il nome di *formula dell'esecuzione.* [1]

[1] Ricordiamo che $\perp$ e/' l'elemento neutro additivo, che $A^\emptyset$ e/' l'elemento neutro moltiplicativo, e che $\perp \cdot w = \perp$ per ogni serie formale $w \in \mathbb{N}[M]$.

Esempio 3.5. Nel caso dell'esempio precedente abbiamo

$$V_0 = (A^{\emptyset}, \perp)$$

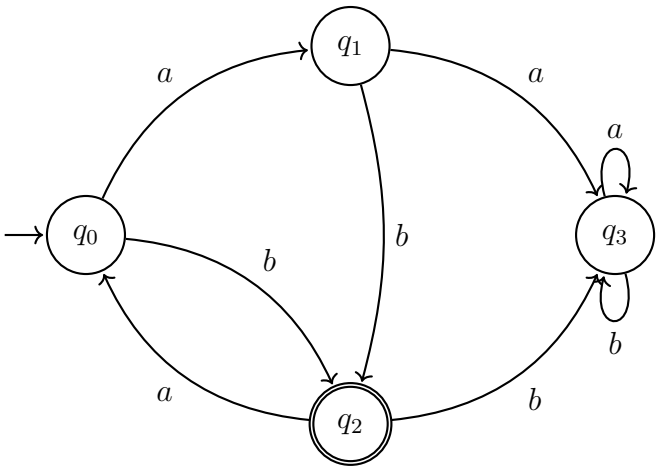
$$V_F = (A^{\emptyset}, \perp)$$

Facendo il prodotto abbiamo

$$\begin{aligned} V_0 \cdot E(Z) \cdot V_F^t &= \begin{pmatrix} A^{\emptyset} & \perp \end{pmatrix} \cdot \begin{pmatrix} A^{\emptyset} + \sum_{k=1}^{\infty}(0+1)^{2k} & \sum_{k=0}^{\infty}(0+1)^{2k+1} \\ \sum_{k=0}^{\infty}(0+1)^{2k+1} & A^{\emptyset} + \sum_{k=1}^{\infty}(0+1)^{2k} \end{pmatrix} \cdot \begin{pmatrix} A^{\emptyset} \\ \perp \end{pmatrix} \\ &= \begin{pmatrix} A^{\emptyset} + \sum_{k=1}^{\infty}(0+1)^{2k} & \sum_{k=0}^{\infty}(0+1)^{2k+1} \end{pmatrix} \cdot \begin{pmatrix} A^{\emptyset} \\ \perp \end{pmatrix} \\ &= A^{\emptyset} + \sum_{k=1}^{\infty}(0+1)^{2k} \end{aligned}$$

Che e/' esattamente quanto abbiamo osservato precedentemente.

Esercizio 3.2.1. Si consideri il seguente automa di alfabeto $A = \{a, b\}$ e insieme degli stati $Q = \{q_0, q_1, q_2\}$:



1. Scrivere la descrizione dell'automa in termini della tabella $T : Q \times A \to Q$ dello stato iniziale e dell'insieme F degli stati finali accettanti;

2. Calcolare la matrice M corrispondente all'automa, separare la matrice M in due matrici di adiacenza (con elementi 0/1) relative al grafo delle transizioni etichettate con a e quelle etichettate con b, in modo tale che:

$$M = aM_a + bM_b.$$

3. Calcolare M_a^2, M_a^3, M_b^2, M_b^3;

4. Calcolare, con l'aiuto del punto precedente, M^4.

Soluzione esercizio 3.2.1. 1. La tabella delle transizioni T è la seguente:

$$\begin{array}{rccc} T: & Q \times A & \to & Q \\ & (q_0, a) & \mapsto & q_1 \\ & (q_0, b) & \mapsto & q_2 \\ & (q_1, a) & \mapsto & q_3 \\ & (q_1, b) & \mapsto & q_2 \\ & (q_2, a) & \mapsto & q_0 \\ & (q_2, b) & \mapsto & q_3 \\ & (q_3, a) & \mapsto & q_3 \\ & (q_3, b) & \mapsto & q_3 \end{array}$$

mentre lo stato iniziale è q_0 e l'insieme degli stati finali accettanti è $F = \{q_2\}$.

2. La matrice M di adiacenza del grafo etichettato corrispondente all'automa è:

$$M = \begin{pmatrix} \bot & a & b & \bot \\ \bot & \bot & b & a \\ a & \bot & \bot & b \\ \bot & \bot & \bot & a+b \end{pmatrix}$$

Ricordando che $\bot$ rappresenta la serie formale vuota, e quindi vale $\bot \cdot w = \bot$ per ogni $w \in \mathbb{N}[M]$; possiamo separere la matrice M in due matrici di adiacenza M_a e M_b come segue:

$$M_a = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

3. Per fare i conti si hanno due strade: si possono calcolare le potenze di M_a e M_b separatamente, oppure si puo/' consideraare il grafo etichettato di sopra e contare i cammini di lunghezza 2 e 3 che collegano i vertici q_i e q_j che utilizzano solamente archi etichettati con a (per calcolare M_a^2, M_a^3) o solamente archi etichettati con b (per calcolare M_b^2, M_b^3).

$$M_a^2 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad M_a^3 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b^2 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad M_b^3 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

e abbiamo che

$$M_a^k = M_b^k = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

per ogni $k \geq 2$.

4. A questo punto possiamo calcolare M^4 come segue:

$$\begin{aligned} M^4 &= (aM_a + bM_b)^4 \\ &= (aM_a + bM_b)^2(aM_a + bM_b)^2 \\ &= (a^2M_a^2 + 2abM_aM_b + b^2M_b^2)(a^2M_a^2 + 2abM_aM_b + b^2M_b^2) \\ &= a^4M_a^4 + a^3bM_a^3M_b + a^2baM_a^2M_bM_a + a^2b^2M_a^2M_b^2 \\ &+ aba^2M_aM_bM_a^2 + ababM_aM_bM_aM_b + ab^2aM_aM_b^2M_a + ab^3M_aM_b^3 \\ &+ ba^3M_bM_a^3 + ba^2bM_bM_a^2M_b + babaM_bM_aM_bM_a + bab^2M_bM_aM_b^2 \\ &+ b^2a^2M_b^2M_a^2 + b^2abM_b^2M_aM_b + b^3aM_b^3M_a + b^4M_b^4 \\ &= \begin{pmatrix} baba & abaa & baab + abab & ba(aa+bb) + ab(ba+bb) + (aa+bb)(a+b)^2 \\ baba & \perp & baab & ba(aa+bb) + (aa+ab+bb)(a+b)^2 \\ aaba & abaa & abab & aa(aa+ab+bb) + ab(ba+bb) + (ba+bb)(a+b)^2 \\ \perp & \perp & \perp & (a+b)^4 \end{pmatrix} \end{aligned}$$

Lasciamo anche esplicitamente tutte le potenze di M_a e M_b che compaiono nel calcolo

di M^4:

$$M_a^4 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_a^3 M_b = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_a^2 M_b M_a = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_a^2 M_b^2 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_a M_b M_a^2 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_a M_b M_a M_b = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_a M_b^2 M_a = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_a M_b^3 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b M_a^3 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b M_a^2 M_b = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b M_a M_b M_a = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b M_a M_b^2 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b^2 M_a^2 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b^2 M_a M_b = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$M_b^3 M_a = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b^4 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Osserviamo che non abbiamo potuto usare la formula del binomio di Newton per calcolare M^4 dalmomento che ne/' il prodotto nel monoide delle parole ne/' il prodotto tra matrici di

adiacenza e/' commutativo. Se avessimo utilizzato il binomio di Newton, avremmo avuto

$$\begin{aligned} M^4 &= (aM_a + bM_b)^4 \\ &= a^4M_a^4 + a^3bM_a^3M_b + a^2b^2M_a^2M_b^2 + ab^3M_aM_b^3 + b^4M_b^4 \\ &= \begin{pmatrix} \bot & \bot & \bot & a^4 + a^3b + a^2b^2 \\ \bot & \bot & \bot & a^4 + a^3b + a^2b^2 + ab^3 \\ \bot & \bot & \bot & a^4 + a^3b \\ \bot & \bot & \bot & a^4 + a^3b + a^2b^2 + ab^3 + b^4 \end{pmatrix} \end{aligned}$$

Questo risultato e/' chiaramente sbagliato poiche/' nell'entrata $(0, 0)$ appare la serie vuota, mentre dal grafo dell'automa sappiamo che esiste il cammino *baba* di lunghezza 4 che parte e ritorna nello stato q_0.

4 Linguaggi regolari

Per determinare se un determinato linguaggio è regolare: mentre la dimostrazione sufficiente di AF- decidibilità (esibendo esplicitamente un automa) può risultare noiosa e complicata, è possibile fornire dimo- strazioni alternative (possibilmente più semplici) mostrando che l'insieme è ottenibile a partire da insiemi AF-decidibili utilizzando operazioni "compatibili" con l'AF-decidibilità (proprietà di chiusura). In questa sezione vedremo dunque che esistono operazioni fatte su insiemi AF-decidibili che permettono di ottenere un altro insieme DFA-decidibile.

Definizione 4.1. Sia A un alfabeto e X_1 e X_2 due inisemi di parole su A, ovvero $X_1, X_2 \subset A^*$. Definiamo allora:

- l'operazione di *concatenazione*:

$$X_1 \cdot X_2 := \{w_1 w_2 \in A^* \mid w_i \in X_i\};$$

- l'operazione di *somma*:

$$X_1 + X_2 := \{w \in A^* \mid w \in X_1 \text{ oppure } w \in X_2\}$$

  (di fatto, è l'unione insiemistica di X_1 con X_2);

- l'operazione di *star di Kleene*:

$$X^* := \{w_1 w_2 \dots w_n \mid w_i \in X, \text{for } i = 1, \dots, n \text{ e } n \geq 0\}$$

Osservazione 4.2. Si puo/' osservare che, un sottoinsieme X di A^* puo' essere visto come una serie, infatti la funzione

$$\begin{array}{ccc} \mathcal{P}(A^*) & \to & \mathbb{N}[A^*] \\ X & \mapsto & \sum_{w \in X} w \end{array}$$

ammmette come inversa la funzione

$$\begin{array}{ccc} \mathbb{N}[A^*] & \to & \mathcal{P}(A^*) \\ \sum_{w \in A^*} a_w w & \mapsto & \{w \,|\, a_w \neq 0\} \end{array}$$

Per convincersi basta osservare che la somma nell'anello delle serie formali e/' commutativo. Questo "giustifica" la notazione $X_1 + X_2$ per l'unione di due insiemi di parole.

Proposizione 4.3. Siano X_1 e X_2 regolari allora anche $X_1 + X_2$ è regolare.

Dimostrazione. Sia $\mathcal{A}_1 = (T_1, q_0', F_1)$ l'automa che decide l'insieme X_1 e sia $\mathcal{A}_2 = (T_2, q_0'', F_2)$ l'automa che decide X_2. L'automa che riconosce $X_1 + X_2$ è ottenuto utilizzando il prodotto cartesiano degli stati dei due automi e calcolando l'evoluzione "parallela" di entrambi gli automi a partire da uno stato coppia, in questo modo se una delle due componenti dello stato è accettante alla fine della computazione la parola sarà accettata poiché apparterrà ad

uno dei due insiemi e quindi sarà nella loro unione. Costruiamo l'automa $\mathcal{A} = (T, q_0, F)$ di alfabeto A e insieme degli stati $Q = Q_1 \times Q_2$ definendo

$$q_0'' = (q_0, q_0'), \qquad F = \{(q_1, q_2) \in Q_1 \times Q_2 \mid q_1 \in F_1 \wedge q_2 \in F_2\} = F_1 \times Q_2 \cup Q_1 \times F_2$$

e

$$T((q_1, q_2), a) = (T_1(q_1, a), T_2(q_2, a)).$$

Mostriamo che $\mathcal{A}$ decide precisamente $X_1 + X_2$, ovvero che la funzione associata all'automa coincide con la funzione caratteristica di tale insieme.
Per farlo introduciamo la seguente notazione: per ogni parola $w \in A^*$, indichiamo con $T^*((q_0', q_0''), w)$ lo stato in cui si trova l'automa $\mathcal{A}$ dopo aver letto tutta la parola w a partire dallo stato (q_0', q_0''), e con $T_i^*(q_i, w)$ lo stato in cui si trova l'automa $\mathcal{A}_i$ dopo aver letto tutta la parola w a partire dallo stato q_i, per $i = 1, 2$. Sia $w \in A^*$, dalla definizione dell'insieme degli stati finali si ha:

$$\begin{aligned}
f_{\mathcal{A}}(w) = 1 &\iff T^*((q_0, q_0'), w) \in F \\
&\iff (T_1^*(q_0, w), T_2^*(q_0', w)) \in (F_1 \times Q_2) \cup (Q_1 \times F_2) \\
&\iff T_1^*(q_0, w) \in F_1 \text{ oppure } T_2^*(q_0', w) \in F_2 \\
&\iff f_{\mathcal{A}_1}(w) = 1 \text{ oppure } f_{\mathcal{A}_2}(w) = 1 \\
&\iff w \in X_1 \text{ oppure } w \in X_2 \\
&\iff w \in X_1 + X_2.
\end{aligned}$$

Quindi, per ogni $w \in A^*$,

$$f_{\mathcal{A}}(w) = \mathbf{1}_{X_1 + X_2}(w).$$

Pertanto l'automa $\mathcal{A}$ decide l'insieme $X_1 + X_2$, e quindi $X_1 + X_2$ è regolare.

□

Osservazione 4.4. La proposizione precedente rimane vera anche se si considera l'operazione di interesezione al posto della somma, infatti se X_1 e X_2 sono regolari allora anche $X_1 \cap X_2$ è regolare. L'automa che decide $X_1 \cap X_2$ si ottiene costruendo l'automa di prima e modificando l'insieme degli stati accettanti, che diventa $F = F_1 \times F_2$.

La domanda che ci vogliamo porre e/' la seguente: la proposizione precedente rimane vera anche se si considera l'operazione di concatenazione al posto della somma? La risposta e/' affermativa, ma la dimostrazione non e/' banale. Potremmo tentare la costruzione di un automa come nella proposizione precedente in modo che ogni parola $w \in X_1X_2$ abbia un inizio riconosciuto dall'automa che riconosce X_1 e continui con una seconda parte che sia accettata dall'automa che riconosce X_2. In pratica si potrebbe concatenare il primo automa con il secondo, ma il problema sarebbe che potrebbero esserci più modi di fattorizzare w in modo che il primo automa accetti la prima parte di w. Per poter definire l'automa concatenazione bisognerebbe sapere a quale punto passare dal primo al secondo automa (e non è detto che ci sia un unico punto che vada bene per tutte le parole di $X_1 \cdot X_2$). Per risolvere questo problema si introduce una nuova nozione: la nozione di computazione *non-deterministica*.

5 Automi non deterministici

5.1 Automi NDFA

Il concetto di computazione non deterministica viene introdotto per modellare la possibilità di effettuare transizioni multiple (in questo caso il non determinismo modella una sorta di computazione parallela) oppure la possibiltà di scegliere tra più possibili evoluzioni della computazione. Per ottenere una versione non deterministica del concetto di automa a stati finiti introdotto nella sezione precedente, bisogna variare la definizione nel punto in cui si stabilisce che Q è il codominio della tabella delle transizioni, sostituendo Q con l'insieme delle parti di Q, ovvero 2^Q: quindi dando alla funzione T la possibilità di fornire non un unico stato ma un insieme di stati; in questo modo avremo un modello di calcolo simile al precedente con una dinamica non deterministica poiché può seguire più computazioni allo stesso tempo, quindi definiamo:

Definizione 5.1 (Automa Finito Non-deterministico)**.** Siano A e Q insiemi finiti. Un *automa finito non-deterministico* (chiamato anche *NDFA*) di alfabeto A e insieme degli stati Q è una tripla (T, q_0, F) dove

$$T : Q \times A \longrightarrow 2^Q,$$

$$F \subseteq Q,$$

e $q_0 \in Q$ è uno stato particolare, detto *stato iniziale*.

Come nel caso deterministico, associamo alla nozione di automa finito non-deterministico quella di algoritmo formale che fornisce una semantica (operazionale) al modello di calcolo, mostrando come la computazione viene effettuata dall'automa. Per prima cosa è necessario stabilire lo spazio delle configurazioni.

Definizione 5.2 (Configurazione per Automa Finito Non-deterministico)**.** Una configurazione per un automa finito non-deterministico di alfabeto A e insieme degli stati Q è una tripla (w, p, S), dove

1. $w \in A^*$ è la parola da analizzare;
2. $p \in \mathbb{N}$ è il puntatore alla posizione della parola che contiene il simbolo dell'alfabeto da considerare per effettuare la successiva transizione;
3. $S \in 2^Q$ è un insieme di stati.

L'insieme delle configurazioni di alfabeto A e insieme degli stati Q per un automa finito sarà denotato con

$$\mathrm{Conf}_{ND}^{A,Q} = A^* \times \mathbb{N} \times 2^Q.$$

Inoltre introduciamo la seguente notazione, quando $S \subseteq Q$:

$$T(S, a) := \bigcup_{q \in S} \{T(q, a)\}$$

Grazie a tale notazione possiamo definire l'algoritmo associato ad un automa finito non-deterministico.

Definizione 5.3 (Algoritmo associato ad un Automa Finito Non-deterministico)**.** L'algoritmo associato ad un automa finito non-deterministico di alfabeto A e insieme degli stati Q è la tupla $(A, \{0,1\}, Conf_{ND}^{A,Q}, \epsilon, \tau, \delta)$ dove le tre funzioni si definscoo come segue :

- la *funzione di ingresso*
$$e : A^* \longrightarrow Conf_{ND}^{A,Q},$$
definita da
$$e(w) = (w, 1, \{q_0\});$$
- la *funzione di transizione*
$$\tau : Conf_{ND}^{A,Q} \longrightarrow Conf_{ND}^{A,Q},$$
definita da
$$\tau((w, p, S)) = (w, p+1, T(S, w(p)));$$
- la *funzione di uscita*
$$\delta : Conf_{ND}^{A,Q} \longrightarrow \{0,1\}^*,$$
definita da
$$\delta((w,p,S)) = \begin{cases} 0, & \text{se } p > |w| \text{ e } F \cap S = \emptyset, \\ 1, & \text{se } p > |w| \text{ e } F \cap S \neq \emptyset. \end{cases}$$

Di conseguenza resta definito il concetto di decidibilità di un insieme mediante automi finiti non-deterministici.

Definizione 5.4 (Insieme decidibile per Automa Finito Non-deterministico)**.** Un insieme $X \subseteq A^*$ è detto *decidibile per automa finito non-deterministico* (o *NDFA-decidibile*) se esistono un insieme finito Q ed un automa finito non-deterministico di alfabeto A e insieme degli stati Q tali che l'algoritmo associato decide X.

Per alleggerire la notazione, con $\widehat{T}(q_0, w)$ (a volte useremo anche T^*) indichiamo l'insieme degli stati raggiungibili partendo dallo stato q_0 dopo la lettura della parola $w = a_1 a_2 \cdots a_n \in A^*$, ovvero:

$$\widehat{T}(q_0, w) := T^*(q_0, w) := \{q \in Q \mid \exists\, p_0, \ldots, p_n \in Q : \ p_0 = q_0,\ p_n = q,\ p_i \in T(p_{i-1}, a_i) \ \ \forall\, i = 1, \ldots, n\}$$

Osservazione 5.5. Un automa di questo tipo ci permette di effettuare transizioni multiple, quindi può seguire più computazioni allo stesso tempo. In particolare, se una di queste computazioni termina in uno stato finale, la parola viene accettata. In altre parole, l'automa non-deterministico può essere visto come un automa deterministico che esplora tutte le possibili transizioni in parallelo. Come per il caso deterministico, anche per gli automi non-deterministici è possibile considerare la matrice di adiacenza associata al grafo etichettato.

Vediamo un primo esempio di automa finito non-deterministico:

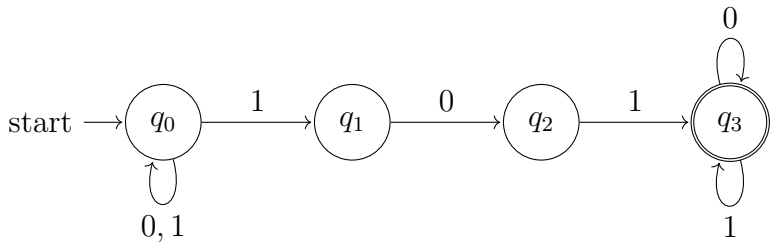
Esempio 5.6 (Un esempio di NDFA)**.** Consideriamo $A = \{0, 1\}$, $Q = \{q_0, q_1, q_2, q_3\}$ e definiamo il seguente NDFA

$$(T, q_0, F)$$

dove $F := \{q_3\}$ e

$$\begin{array}{rccc} T: & Q \times \{0,1\} & \to & 2^Q \\ & (q_0, 0) & \mapsto & \{q_0\} \\ & (q_0, 1) & \mapsto & \{q_0, q_1\} \\ & (q_1, 0) & \mapsto & \{q_2\} \\ & (q_1, 1) & \mapsto & \emptyset \\ & (q_2, 0) & \mapsto & \emptyset \\ & (q_2, 1) & \mapsto & \{q_3\} \\ & (q_3, 0) & \mapsto & \{q_3\} \\ & (q_3, 1) & \mapsto & \{q_3\} \end{array}$$

che può essere rappresenato graficamente come



Analizziamo ora il comportamento dell'automa su alcune parole (ricordiamo che con $\widehat{T}(q_0, w)$ indichiamo l'insieme degli stati raggiungibili partendo dallo stato q_0 dopo la lettura della parola w):

- Consideriamo la parola
$$w = 1101.$$
Calcoliamo l'insieme degli stati raggiungibili dopo la lettura di ciascun simbolo:
$$\begin{aligned} \widehat{T}(q_0, 1) &= \{q_0, q_1\}, \\ \widehat{T}(q_0, 11) &= T(q_0, 1) \cup T(q_1, 1) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}, \\ \widehat{T}(q_0, 110) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}, \\ \widehat{T}(q_0, 1101) &= T(q_0, 1) \cup T(q_2, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}. \end{aligned}$$
Poiché l'insieme finale contiene lo stato finale q_3, la parola
$$1101$$
è accettata.

- Consideriamo ora la parola
$$w = 1000.$$

Si ha

$$\begin{aligned}
\widehat{T}(q_0, 1) &= \{q_0, q_1\},\\
\widehat{T}(q_0, 10) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\},\\
\widehat{T}(q_0, 100) &= T(q_0, 0) \cup T(q_2, 0) = \{q_0\} \cup \emptyset = \{q_0\},\\
\widehat{T}(q_0, 1000) &= T(q_0, 0) = \{q_0\}.
\end{aligned}$$

L'insieme degli stati raggiungibili non contiene alcuno stato finale, quindi la parola

$$1000$$

è rifiutata.

- Consideriamo infine la parola

$$w = 1100101.$$

Si ha

$$\begin{aligned}
\widehat{T}(q_0, 1) &= \{q_0, q_1\},\\
\widehat{T}(q_0, 11) &= T(q_0, 1) \cup T(q_1, 1) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\},\\
\widehat{T}(q_0, 110) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\},\\
\widehat{T}(q_0, 1100) &= T(q_0, 0) \cup T(q_2, 0) = \{q_0\} \cup \emptyset = \{q_0\},\\
\widehat{T}(q_0, 11001) &= T(q_0, 1) = \{q_0, q_1\},\\
\widehat{T}(q_0, 110010) &= T(q_0, 0) \cup T(q_1, 0) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\},\\
\widehat{T}(q_0, 1100101) &= T(q_0, 1) \cup T(q_2, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}.
\end{aligned}$$

Poiché l'insieme finale contiene lo stato finale q_3, la parola

$$1100101$$

è accettata.

Si può mostrare infatti che questo automa è quello che decide l'insieme delle parole che contengono come sottostringa 101. Consideriamo ora lo stesso automa sostituendo la transizione

$$T(q_0, 1) = \{q_0, q_1\}$$

con

$$T(q_0, 1) = \{q_1\},$$

lasciando invariate tutte le altre transizioni. Mostriamo che questo automa non riconosce più tutte le parole contenenti la sottostringa 101. Infatti, la parola

$$w = 1101$$

contiene la sottostringa 101, ma l'automa si comporta nel modo seguente:

$$\begin{aligned}
\widehat{T}(q_0, \epsilon) &= \{q_0\}, \\
\widehat{T}(q_0, 1) &= \{q_1\}, \\
\widehat{T}(q_0, 11) &= T(q_1, 1) = \emptyset, \\
\widehat{T}(q_0, 110) &= \bigcup_{q \in \emptyset} T(q, 0) = \emptyset, \\
\widehat{T}(q_0, 1101) &= \bigcup_{q \in \emptyset} T(q, 1) = \emptyset.
\end{aligned}$$

Poiché la computazione termina con l'insieme vuoto, la parola 1101 viene rifiutata.

L'errore consiste nel fatto che l'automa è costretto a scegliere il *primo* simbolo 1 come possibile inizio della sottostringa 101. Nell'automa non deterministico originale, invece, il ramo che rimane nello stato q_0 permette di ignorare il primo 1 e di scegliere il secondo 1 come inizio della sottostringa, accettando così correttamente la parola.

5.2 Automi ϵ-NDFA

In vista, dell'obiettivo di mostrare che l'insieme $X_1 \cdot X_2$ ottenuto a partire da due insiemi X_1 e X_2 DFA-decidibili è lui stesso DFA-decidibile dobbiamo poter costruire un automa che "concatena" la computazione di un automa con quella del secondo, a questo fine è comodo utilizzare una ulteriore variante della definizione di NDFA che può effettuare cambi di stato senza far avanzare il puntatore alla posizione corrente:

Definizione 5.7 (Automa Finito non-deterministico con ϵ-mosse)**.** Siano A e Q insiemi finiti, un automa finito non-deterministico con ϵ-mosse *di alfabeto A e insieme degli stati Q* è una tripla (T, q_0, F) dove

$$T : Q \times A \cup \{\epsilon\} \to 2^Q$$
$$F \subset Q$$

e $q_0 \in Q$ è uno stato particolare, detto *stato iniziale*.

Osservazione 5.8. Di solito si usa simbnolo ϵ poiché rappresenta il simbolo della parola vuota, ovvero la parola di lunghezza zero in A^* (quella che sopra abbiamo indicato come $A^{\emptyset}$).

Per la descrizione della computazione dovremo introdurre una variante della funzione di transizione che prenda in considerazione le ϵ-mosse, a questo scopo introduciamo la seguente notazione per $S \in 2^Q$:

$$T^k(S, a) := \begin{cases} S & \text{se } k = 0 \\ T(T^{k-1}(S, a), a) & k > 0 \end{cases}$$

Perche dobbiamo introdurre questa notazione? Dobbiamo farlo perché, a partire da un qualsiasi stato q_i vogliamo poter raggiungere tutti gli stati possibili dopo un numero finito ma arbitrariamente grande di ϵ-mosse, senza conusmare lettere della parola che stiamo analizzando.

Definizione 5.9 (Algoritmo associato ad un automa finito non-deterministico con ϵ-mosse)**.** L'algoritmo associato ad un automa non-deterministico di alfabeto A e insieme degli stati Q viene descritto definendo le tre funzioni:

- la **funzione di ingresso** che associa ad un parola $w \in A^*$, la configurazione

$$\epsilon(w) = \left(w, 1, \bigcup_{k \geq 0} T^k(\{q_0\}, \epsilon) \right),$$

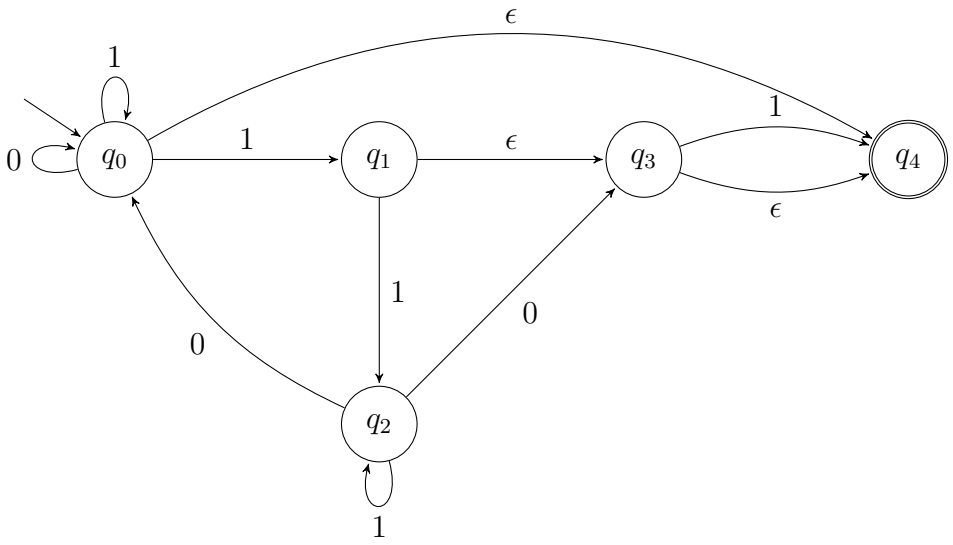
- la **funzione di transizione** $\tau : \mathrm{Conf}_{ND}^{A,Q} \to \mathrm{Conf}_{ND}^{A,Q}$ è data da

$$\tau((w, p, S)) = \left(w, p+1, \bigcup_{k \geq 0} T^k(T(S, w(p)), \epsilon) \right),$$

- la **funzione di uscita** $\delta : \mathrm{Conf}_{ND}^{A,Q} \to \{0, 1\}$, data da

$$\delta((w, p, S)) = \begin{cases} 0 & \text{se } p > |w| \text{ e } F \cap S = \emptyset \\ 1 & \text{se } p > |w| \text{ e } F \cap S \neq \emptyset \end{cases}$$

Esempio 5.10. Consideriamo il seguente grafo etichettato, che rappresenta un automa finito non-deterministico con ϵ-mosse:



In questo caso, se mi trovo nello stato q_0 e leggo 1 ottengo gli stati $\{q_0, q_1\}$. Inoltre, senza leggere alcun simbolo (utilizzando quinid le ϵ-mosse), posso passare:

- dallo stato q_0 allo stato q_4;
- dallo stato q_1 allo stato q_3 e poi
  dallo stato q_3 allo stato q_4.

Quindi, dopo aver letto il simbolo 1 partendo dallo stato iniziale q_0, si raggiunge lo stato $\{q_0, q_1, q_3, q_4\}$. Se non ci fosse stata la ϵ-mossa da q_0 a q_4, avremmo raggiunto comunque anche lo stato q_4 poiché, possiamo fare un numero finito ma arbitrariamente grande di ϵ-mosse, quindi avremmo potuto passare da q_1 a q_3 e poi da q_3 a q_4 con le 2 ϵ-mosse che ci sono tra questi stati.

Esempio 5.11 (Un esempio di ϵ-NDFA)**.** Consideriamo l'alfabeto

$$A = \{0, 1\},$$

l'insieme degli stati

$$Q = \{q_0, q_1, q_2, q_3\},$$

e definiamo il seguente ϵ-NDFA

$$(T, q_0, F),$$

dove

$$F = \{q_3\}$$

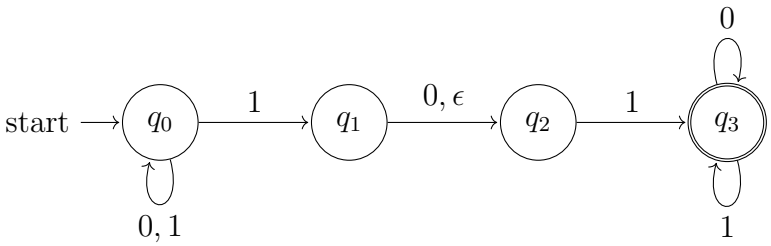
e

$$T : Q \times (A \cup \{\epsilon\}) \to 2^Q$$

è definita da

$$\begin{array}{rcl} T(q_0, 0) & = & \{q_0\}, \\ T(q_0, 1) & = & \{q_0, q_1\}, \\ T(q_1, 0) & = & \{q_2\}, \\ T(q_1, 1) & = & \emptyset, \\ T(q_1, \epsilon) & = & \{q_2\}, \\ T(q_2, 0) & = & \emptyset, \\ T(q_2, 1) & = & \{q_3\}, \\ T(q_3, 0) & = & \{q_3\}, \\ T(q_3, 1) & = & \{q_3\}. \end{array}$$

L'automa può essere rappresentato graficamente come



Per descrivere la computazione consideriamo la funzione di transizione estesa

$$\widehat{T} : 2^Q \times A^* \to 2^Q$$

definita considerando le ϵ-mosse come transizioni gratuite. In particolare, per ogni insieme di stati $S \subseteq Q$ poniamo

$$T^0(S, \epsilon) = S,$$

e

$$T^k(S, \epsilon) = T(T^{k-1}(S, \epsilon), \epsilon)$$

per $k > 0$. La computazione dell'automa consiste quindi nel considerare tutti gli stati ottenibili dopo un numero finito di ϵ-mosse.

- Consideriamo la parola

  $$w = 101.$$

  Partendo dallo stato iniziale otteniamo

  $$\widehat{T}(q_0, \epsilon) = \{q_0\}.$$

  Dopo aver letto il primo simbolo 1:

  $$T(\{q_0\}, 1) = \{q_0, q_1\}.$$

  Applicando successivamente le ϵ-mosse:

  $$T^0(\{q_0, q_1\}, \epsilon) = \{q_0, q_1\},$$

  $$T^1(\{q_0, q_1\}, \epsilon) = T(\{q_0, q_1\}, \epsilon) = \{q_2\}.$$

  Quindi dopo il primo simbolo:

  $$\widehat{T}(q_0, 1) = \{q_0, q_1, q_2\}.$$

  Ora leggiamo il simbolo 0:

  $$T(\{q_0, q_1, q_2\}, 0) = \{q_0\} \cup \{q_2\} \cup \emptyset = \{q_0, q_2\}.$$

  Non sono possibili ulteriori ϵ-mosse, quindi:

  $$\widehat{T}(q_0, 10) = \{q_0, q_2\}.$$

  Infine leggiamo 1:

  $$T(\{q_0, q_2\}, 1) = \{q_0, q_1\} \cup \{q_3\} = \{q_0, q_1, q_3\}.$$

  Poiché

  $$q_3 \in \widehat{T}(q_0, 101),$$

  la parola 101 è accettata.

- Consideriamo la parola
$$w = 10.$$
Dopo il simbolo 1 abbiamo già visto che
$$\widehat{T}(q_0, 1) = \{q_0, q_1, q_2\}.$$
Leggendo 0 otteniamo:
$$T(\{q_0, q_1, q_2\}, 0) = \{q_0\} \cup \{q_2\} \cup \emptyset = \{q_0, q_2\}.$$
Non essendo presente lo stato finale:
$$q_3 \notin \widehat{T}(q_0, 10),$$
la parola 10 è rifiutata.

5.3 Chiusura per concatenazione e star di Kleene

A questo punto possiamo mostrare che la concatenazione di due insiemi decidibili per automa finito non-deterministico è anch'essa decidibile per automa finito non-deterministico. Dimostriamo prima il seguente risultato intermedio:

Proposizione 5.12. Siano X_1 e X_2 insiemi decidibili per automa finito non-deterministico allora $X_1 \cdot X_2$ è decidibile per automa finito non-deterministico con ϵ-mosse.

Dimostrazione. Siano i due automi che decidono $X_1 \subset A^*$ e $X_2 \subset A^*$ rispettivamente dati su insiemi degli stati disgiunti Q_1 e Q_2, ovvero con $Q_1 \cap Q_2 = \emptyset$: $\mathcal{A}_1 = (T', q_0', F')$ e $\mathcal{A}_2 = (T'', q_0'', F'')$.

Costruiamo l'automa che decide $X_1 \cdot X_2$ aggiungendo alle transizioni di $\mathcal{A}_1$ e $\mathcal{A}_2$ quelle che vanno dagli stati accettanti di $\mathcal{A}_1$ allo stato iniziale q_0'' del secondo automa, ovvero definiamo l'automa $\mathcal{A}$ con insieme degli stati $Q := Q_1 \cup Q_2$ dove lo stato iniziale è $q_0 := q_0' \in Q_1$ e gli stati finali accettanti sono definiti come quelli di $\mathcal{A}_2$, dunque:
$$\mathcal{A} := (T, q_0, F)$$
dove
$$T(q, a) := \begin{cases} T'(q, a) & \text{se } q \in F' \text{ e } a \neq \epsilon \text{ oppure } q \notin F', \\ T'(q, a) \cup \{q_0''\} & q \in F' \text{ e } a = \epsilon, \\ T''(q, a) & q \in Q_2, \end{cases}$$
e l'insieme degli stati finali accettanti è dato da $F := F''$. □

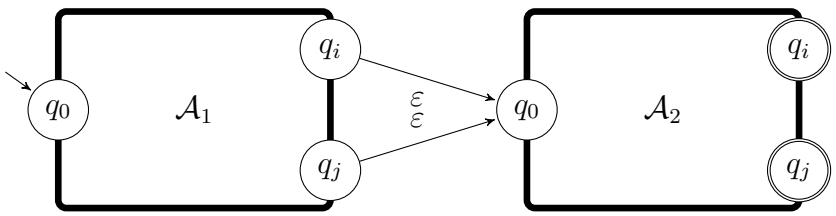


Figura 4: L'idea della dimostrazione consiste nel costruire un automa usando le ϵ-mosse per concatenare le computazioni dei due automi.

Proposizione 5.13. Sia dato un insieme $X \subseteq A^*$ per un certo alfabeto finito A. Allora

$$X \text{ è DFA-decidibile} \iff X \text{ è NDFA-decidibile} \iff X \text{ è } \epsilon\text{-NDFA-decidibile.}$$

Dimostrazione. Procediamo dimostrando le tre implicazioni.

(DFA $\Rightarrow$ NDFA) Sia (T, q_0, F) un DFA. Basta considerare la funzione di transizione come funzione a valori nell'insieme delle parti:

$$T' : Q \times A \longrightarrow 2^Q,$$

definita da

$$T'(q, a) := \{T(q, a)\}.$$

L'automa così ottenuto è un NDFA che produce esattamente la stessa computazione.

NDFA $\Rightarrow$ ϵ-NDFA) Sia (T, q_0, F) un NDFA. Definiamo

$$T' : Q \times (A \cup \{\epsilon\}) \longrightarrow 2^Q$$

ponendo

$$T'(q, a) := T(q, a), \qquad a \in A,$$

e

$$T'(q, \epsilon) := \varnothing.$$

In questo modo l'automa non utilizza mai ϵ-mosse e coincide con quello di partenza.

(ϵ-NDFA $\Rightarrow$ DFA) Sia (T, q_0, F) un ϵ-NDFA.

Consideriamo come insieme degli stati

$$Q' = 2^Q.$$

La funzione di transizione del DFA è definita da

$$T' : 2^Q \times A \longrightarrow 2^Q,$$

ponendo

$$T'(S, a) := \bigcup_{k \geq 0} T^k(T(S, a), \epsilon).$$

Lo stato iniziale è

$$q_0' := \bigcup_{k \geq 0} T^k(\{q_0\}, \epsilon),$$

mentre l'insieme degli stati finali è

$$F' := \{\, S \subseteq Q \mid S \cap F \neq \varnothing \,\}.$$

È facile controllare che per ogni parola ω le computazioni ottenute tramite i rispettivi algoritmi coincidono, pertanto le decisioni dei due automi coincideranno per ogni ω e quindi decideranno lo stesso insieme.

Ne segue che le tre nozioni di decidibilità coincidono. □

Corollario 5.14. Se X è DFA decidibile, allora anche X^* è DFA decidibile.

Da quanto appena visto otteniamo più del risultato vouluto, infatti abbiamo mostrato che ogni automa finito non-deterministico con ϵ-mosse può essere trasformato in un automa finito deterministico, quindi possiamo concludere che gli insiemi decidibili per automi finiti non-deterministici coincidono con quelli decidibili per automi finiti deterministici. In che modo si può trasformare un automa finito non-deterministico con ϵ-mosse in un automa finito deterministico?
Per farlo, dovremmo definire una funzione di transizione T' della forma

$$T' : 2^Q \times A \to 2^Q$$

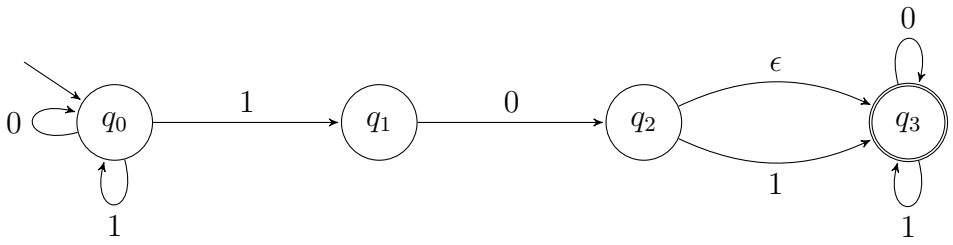
Dal momento che $|Q| < |2^Q|$, e che in generale $|Q|$ può essere molto più piccolo di $|2^Q|$, la costruzione della mappa T' del DFA può richiedere di esplicitare un numero di stati molto più grande di quello dell'automa ϵ-NDFA di partenza. Per risolvere il problema, si restringe la funzione T' (definita come nella dimostrazione della proposizione precedente) all'insieme $Q' \times A$ dove

$$Q' := \{\widehat{T}(q_0, a) \mid a \in A^*\}$$

Ovvero, si considerano solo gli stati raggiungibili a partire dallo stato iniziale q_0 definendo come stato iniziale per il DFA lo stato

$$S_0 := \bigcup_{k=0}^{\infty} T^k(\{q_0\}, \epsilon)$$

Esempio 5.15. Consideriamo l'ϵ-NDFA associato al seguente grafo etichettato:



Questo automa ha come funzione di transizione la funzione

$$\begin{array}{rccc} T: & Q \times (A \cup \{\epsilon\}) & \to & 2^Q \\ & (q_0, 0) & \mapsto & \{q_0\} \\ & (q_0, 1) & \mapsto & \{q_0, q_1\} \\ & (q_0, \epsilon) & \mapsto & \emptyset \\ & (q_1, 0) & \mapsto & \{q_2\} \\ & (q_1, 1) & \mapsto & \emptyset \\ & (q_1, \epsilon) & \mapsto & \emptyset \\ & (q_2, 0) & \mapsto & \emptyset \\ & (q_2, 1) & \mapsto & \{q_3\} \\ & (q_2, \epsilon) & \mapsto & \{q_3\} \\ & (q_3, 0) & \mapsto & \{q_3\} \\ & (q_3, 1) & \mapsto & \{q_3\} \\ & (q_3, \epsilon) & \mapsto & \emptyset \end{array}$$

Vogliamo costruire un DFA che decida lo stesso insieme di parole dell'automa precedente. Dobbiamo quindi stabilire quindi l'insime degli stati Q'. Iniziamo osservando che

$$S_0 := \bigcup_{k=0}^{\infty} T^k(\{q_0\}, \epsilon) = \{q_0\}$$

visto che dallo stato iniziale non si possono fare ϵ-mosse. Partendo quindi da S_0 possiamo arrivare nello stato $\{q_0, q_1\}$ leggendo 1 oppure rimanere nello stato $\{q_0\}$ leggendo 0. Facendo questo ragionamento per tutti gli stati raggiungibili a partire da S_0 otteniamo l'insieme degli stati del DFA:

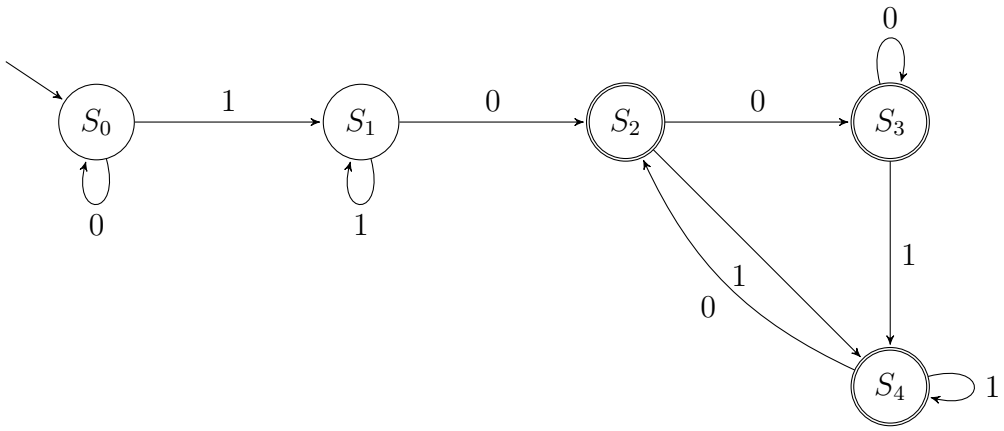
$$Q' = \{S_0 = \{q_0\}, S_1 = \{q_0, q_1\}, S_2 = \{q_0, q_2, q_3\}, S_3 = \{q_0, q_3\}, S_4 = \{q_0, q_1, q_3\}\}$$

Possiamo quindi ora scrivere (in forma tabellare) la restrizione della funzione di transizione T' all'insieme $Q' \times A$:

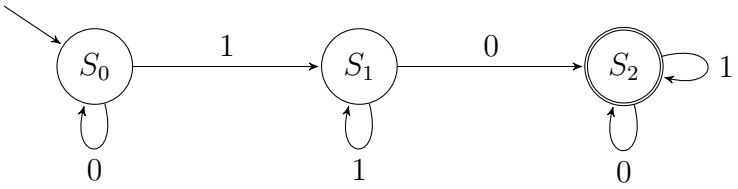
$Q' \subseteq 2^Q \setminus A$	0	1
$S_0 = \{q_0\}$	S_0	S_1
$S_1 = \{q_0, q_1\}$	S_2	S_1
$S_2 = \{q_0, q_2, q_3\}$	S_3	S_4
$S_3 = \{q_0, q_3\}$	S_3	S_4
$S_4 = \{q_0, q_1, q_3\}$	S_2	S_4

Abbiamo a tutti gli effetti costruito un DFA che decide lo stesso insieme di parole deciso dell'automa ϵ-NDFA di partenza. Se a questo punto vogliamo rappresentare graficamente il

DFA, otteniamo il seguente grafo etichettato:



Possiamo osservare che gli stati accettanti del DFA formano un pozzo per il grafo, ovvero una volta entrati in uno di questi stati non si può più uscire indipendentemente dalla lettera letta. Questo vuol dire che in realtà, l'automa puo assumere la forma di un DFA con un solo stato accettante, che è quello che si ottiene unendo tutti gli stati accettanti del DFA costruito in un unico stato accettante. Otteniamo quindi il seguente grafo etichettato:



6 Espressioni regolari

6.1 Sintassi ed interpretazione

Avendo individuato i costrutti rispetto ai quali l'insieme di tutti gli insiemi regolari resta invariato (concatenazione, unione e operatore * di Kleene), si può formalizzare una grammatica data dalla rappresentazione simbolica di alcuni insiemi regolari di base.

Definizione 6.1. Fissiamo un alfabeto A. L'insieme $\mathcal{E}$ (anche indicato con E) delle *espressioni regolari* sull'alfabeto A è il più piccolo insieme tale che:

- $\mathbb{0}, \mathbb{1} \in \mathcal{E}$;
- per ogni $a \in A$ $a \in \mathcal{E}$;
- se $E_1, E_2 \in \mathcal{E}$ allora $E_1 + E_2, E_1 \cdot E_2, E_1^* \in \mathcal{E}$

Esempio 6.2. Alcune espressioni regolari sull'alfabeto $A = 0, 1$ sono:

- $\mathbb{0}$;
- $\mathbb{0} + 0$;
- $(0 + 1)^*$;
- $0 \cdot (0 + 1 \cdot 1)^* \cdot \mathbb{1}$.

Osservazione 6.3. Chiaramente, fino ad ora, abbiamo usato i simboli $+, \cdot, \cdot$ in due contesti diversi: per la definizione formale di espressione regolare e per le definizioni di operazioni sui sottoinsiemi degli alfabeti. Questo non è un problema visto che si capisce a quale definizione si fa riferimento in base al contesto.

Definizione 6.4 (Interpretazione di un'espressione regolare)**.** Per ogni espressione regolare $e \in E$, di alfabeto A, definiamo un'interpretazione

$$[\![\ ]\!] : E \longrightarrow 2^{A^*}$$

dove

$$[\![e]\!] := \begin{cases} \varnothing & \text{se } e = \mathbb{0} & \text{(insieme vuoto)} \\ \{\epsilon\} & \text{se } e = \mathbb{1} & \text{(insieme singleton parola vuota)} \\ \{a\} & \text{se } e = a \in A & \text{(insieme singleton simbolo)} \\ [\![e_1]\!] \cdot [\![e_2]\!] & \text{se } e = e_1 \cdot e_2 & \text{(concatenazione)} \\ [\![e_1]\!] + [\![e_2]\!] & \text{se } e = e_1 + e_2 & \text{(unione)} \\ [\![e_1]\!]^* & \text{se } e = e_1^* & \text{(insieme star)} \end{cases}$$

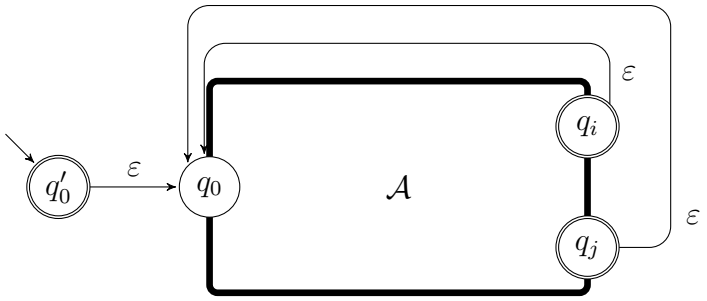


Figura 5: Costruzione dell'automa che decide X^* a partire da quello che decide X.

Osservazione 6.5. Chiaramente, riprendendo la precedente osservazione, con $[\![e_1]\!] \cdot [\![e_2]\!]$ indichiamo la conatenazione degli insiemi $[\![e_1]\!]$ e $[\![e_2]\!]$.

Proposizione 6.6. Data un'espressione regolare $e \in E$ l'insieme $[\![e]\!]$ è un insieme regolare.

Dimostrazione. Questo risultato è conseguenza delle proprietà di chiusura (per prodotto, somma e star) soddisfatte dagli insiemi DFA-decidibili, e dal fatto che le tre espressioni regolari di base si interpretano con insiemi DFA-decidibili: si veda la figura 6 per gli automi che decidono le tre espressioni regolari di base. □



(a) NDFA che decide l'insieme vuoto

(b) NDFA che decide l'insieme singleton della parola vuota

(c) NDFA che decide l'insieme singleton di una lettera dell'alfabeto

Figura 6: Gli automi che decidono le tre espressioni regolari di base

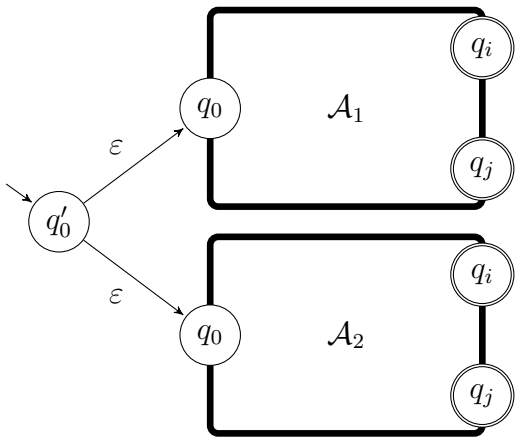


Figura 7: Costruzione dell'automa che decide $X_1 + X_2$ a partire da quelli che decidono X_1 e X_2.

6.2 Automi non deterministici generalizzati

Quello che invece vorremmo mostrare ora è che vale anche l'implicazione inversa della proposizione precedente. Per farlo introdurremo una nuova versione di automa.

Definizione 6.7. Un *automa a stati finiti non-deterministico generalizzato* di alfabeto A e insieme degli stati Q è una tripla:

$$(T, q_0, F)$$

dove

$$T : Q \times E \to 2^Q,$$

$q_0 \in Q$ è detto stato iniziale, $F \subset Q$ è detto insieme degli stati finali accettanti.

L'idea alla base di questo nuovo tipo di automa è che si può etichettare le transizioni dell'automa non solo con le lettere dell'alfabeto ma con qualsiasi espressione costruita a partire dall'alfabeto. A questo punto, dovremmo però dare di nuovo le definizioni di algoritmo associato ad un automa generalizzato e di decidibilità. Usando però la proposizione sopra, possiamo fare ciò in modo furbo:

Osservazione 6.8. Dato un automa generalizzato $A = (T, q_0, F)$ con insieme di stati Q, gli si può associare un ϵ-NDFA nel seguente modo. Per ogni coppia (q, e) tale che $T(q, e) = p$, il risultato precedente fornisce un DFA $A_e = (T^e, q_0^e, F^e)$ che decide l'iniseme regolare $[\![e]\!]$. A questo punto, possiamo modificare l'automa A in un ϵ-NDFA della forma $A' = (T', q_0', F')$, eliminando tutte le definizioni del tipo $T(q, e) = p$ e aggiungendo per ogni coppia (q, e) come sopra:

1. una ϵ-transizione che collega q a q_0^e, ovvero definendo $T'(q, \epsilon) := q_0$;
2. le transizioni interne di ogni A_e;
3. una ϵ-transizione che collega ogni stato accettante di A_e q_i a p, ovvero definendo $T'(q_i, \epsilon) := p$ per ogni $q_i \in F^e$.

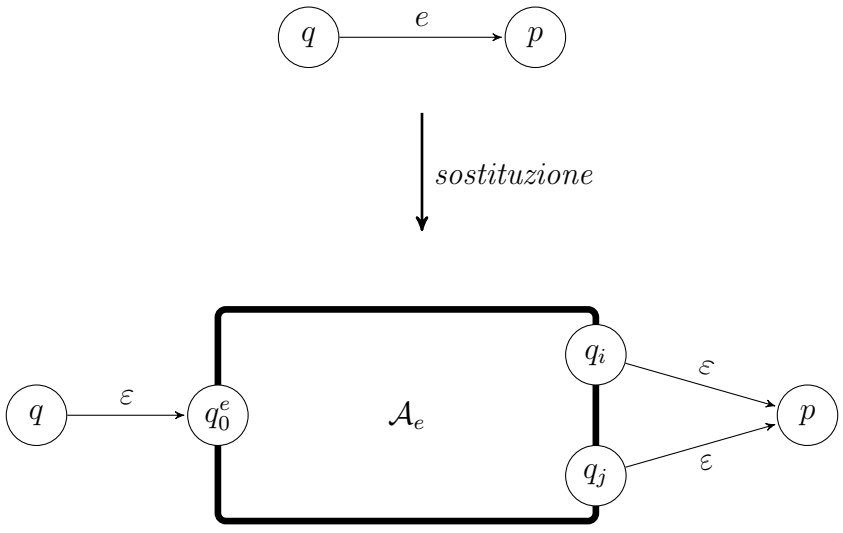


Figura 8: Un passo di sostituzione: la transizione $T(q, e) = p$ viene rimpiazzata dal DFA $\mathcal{A}_e$ che decide $[\![e]\!]$, collegato a q e a p tramite ε-transizioni.

Definizione 6.9. Dato un automa generalizzato A, *l'insieme* $X \subset A^*$ *deciso dall'automa generalizzato* A è l'insieme deciso dall'ϵ-NDFA che gli si può associare usando la procedura esposta nell'osservazione 6.8.

Ora elenchiamo tre trasformazioni che si possono effettuare su un automa generalizzato che non modicano l'insieme deciso dall'automa stesso.

- Setup restituisce l'automa con l'aggiunta di uno stato iniziale q_0^* e di un unico stato finale q_F^* che non hanno archi entranti e su cui non saranno fatte agire le trasformazioni seguenti (Merge e Erase). Il nuovo automa avrà le stesse transizioni dell'automa dato con l'aggiunta di una $\mathbb{1}$-transizione da q_0^* a q_0 e di $\mathbb{1}$-transizioni da ogni stato finale $q \in F$ verso q_F^*:

$$T'(q,e) := \begin{cases} q_0 & \text{se } q = q_0^* \text{ e } e = \epsilon \\ q_F^* & \text{se } q \in F \text{ e } e = \epsilon \\ T(q,e) & \text{altrimenti.} \end{cases}$$

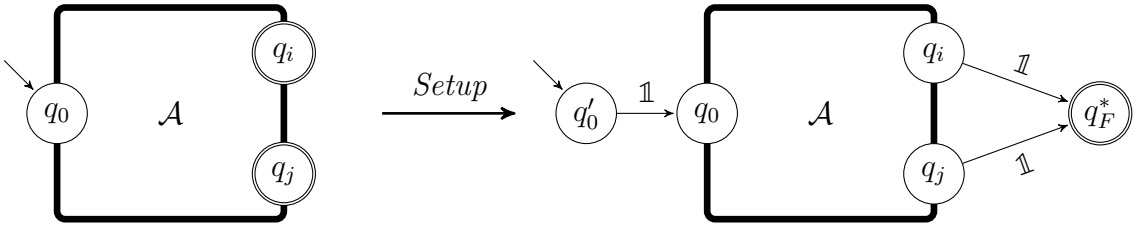


Figura 9: Trasformazione dell'automa $\mathcal{A}$ dopo Setup.

- Merge restituisce l'automa in cui due transizioni parallele sono fuse in un'unica transizione: in pratica, poiché l'automa generalizzato è non deterministico è possibile avere due espressioni e_1 ed e_2 tali che $T(q_i, e_1) = s_1$ e $T(q_i, e_2) = s_2$ e $q_j \in s_1 \cap s_2$. Sono queste due transizioni, dette *parallele* per la coppia di stati (q_i, q_j), che possono essere fuse in un'unica transizione in corrispondenza della somma delle due espressioni regolari.

  Definiamo l'automa (T', q_0, F) in cui le transizioni da q_i a q_j sono sostituite con un'unica transizione $T' := \text{Merge}(T, q_i, q_j)$:

$$T'(q,e) = \begin{cases} T(q,e) & \text{se } q \neq q_i \text{ e } T(q,e) \neq q_j \\ \displaystyle\sum_{e \in E_{i,j}} e & \text{se } E_{i,j} := \{e \mid q = q_i \text{ e } T(q_i,e) = q_j\} \end{cases}$$

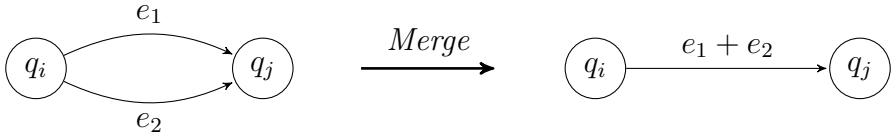


Figura 10: Fusione di due transizioni parallele da q_i a q_j in un'unica transizione etichettata dalla somma delle espressioni regolari.

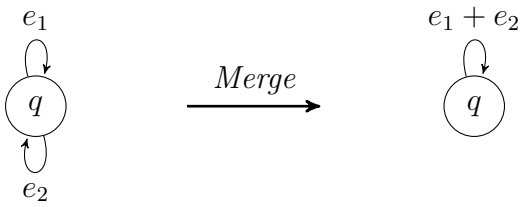


Figura 11: Caso particolare del Merge con $q_i = q_j = q$: due cappi su q sono transizioni parallele da q a q, e vengono fusi in un unico cappio etichettato da $e_1 + e_2$.

- Erase Consideriamo un automa generalizzato (T, q_0, F) senza transizioni parallele e introduciamo la seguente trasformazione che consiste nell'eliminare uno stato $\bar{q} \notin F \cup \{q_0\}$ preservando l'insieme che viene deciso dall'automa: il nuovo automa (T', q_0, F) sarà dunque definito su $Q \setminus \{\bar{q}\}$ nel modo seguente:

  se $T(q_i, e_i) = \bar{q}$, $T(\bar{q}, e_j) = q_j$ e $T(\bar{q}, e_k) = \bar{q}$ allora

  $$T'(q_i, e) = q_j \text{ dove } e = e_i e_k^* e_j$$

  per tutte le altre transizioni che non coinvolgono $\bar{q}$ non si interviene e si definisce

  $$T'(q, e) = T(q, e).$$

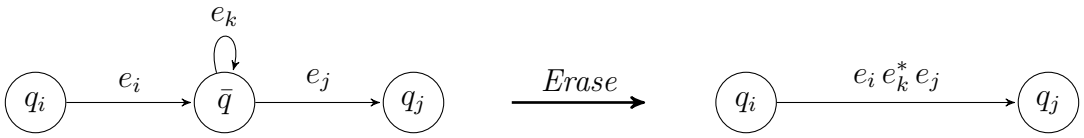


Figura 12: Eliminazione dello stato $\bar{q}$: il cammino $q_i \to \bar{q} \to q_j$ con cappio su $\bar{q}$ diventa un'unica transizione etichettata da $e_i e_k^* e_j$.

Osservazione 6.10. L'operazione "Erase" può essere applicata nel caso in cui non c'è nessun arco della forma e_j. Questo è il caso in cui esiste uno stato pozzo non accettante nel grafo (se non consideriamo q_j nella figura 12, allora lo stato pozzo è $\bar{q}$). In questo caso, seguendo la regola otteniamo la semplice rimozione dello stato pozzo e di tutti gli archi che arrivano a lui. Questa operazione è lecita, infatti quando un automa arriva in uno stato pozzo, questo non potrà mai uscirne, e nel caso in cui lo stato non è accettante, abbiamo che una parola che raggiunge quello stato non sarà mai accettata.

6.3 Da automa ad espressione regolare (e viceversa)

A questo punto possiamo mostrare il seguente risultato.

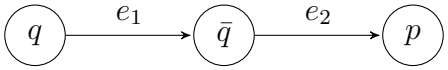
Proposizione 6.11. Dato X un insieme regolare, esiste un'espressione regolare e_X tale che $[\![e_X]\!] = X$.

Dimostrazione. Consideriamo A un DFA che decide X. Consideriamo l'automa generalizzato A' definito partendo da A e considerando come etichette gli elementi formali di del linguaggio E_A invece che le lettere di A. A questo punto abbiamo un automa generalizzato e possiamo applicare l'operazione "Setup" e poi le operazioni "Merge" e "Erase" fino a quando non otteniamo un automa con solo due stati, q_0^* e q_F^* (quelli introdotti con l'operazione "Setup") e un solo arco tra i due stati etichettato con l'espressione e. Usando che le operazioni sugli automi generalizzati definite prima non modificano l'insieme deciso, otteniamo che $e_X = e$. □

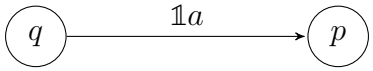
Osservazione 6.12. Consideriamo ora due espressioni, per esempio:

$$e_1 = \mathbb{1} \quad \text{e} \quad e_2 = a$$

che siano etichette di due archi di un grafo associato ad un automa come in figura:



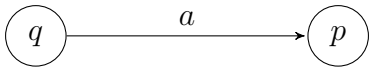
Se si vuole fare l'operazione "Erase" si ottiene il seguente grafo:



Se ora vogliamo considerare l'interpretazione di questa espressione, otteniamo:

$$[\![\mathbb{1}a]\!] = [\![\mathbb{1}]\!] \cdot [\![a]\!] = \{\epsilon\} \cdot \{a\} = \{a\} = [\![a]\!]$$

Questo vuol dire che, l'arco sopra può essere scritto come



e questo cambio di etichetta non modifica l'insieme deciso dall'automa. Vale il fatto analogo durante le operazioni di "Merge" per $\mathbb{0}$, infatti

$$[\![\mathbb{0} + a]\!] = [\![\mathbb{0}]\!] + [\![a]\!] = \emptyset \cup \{a\} = \{a\} = [\![a]\!]$$

Alla luce di questa osservazione, vale il seguente risultato:

Proposizione 6.13. Sia A il grafo associato ad un automa generalizzato e sia A' il grafo ottenuto da A semplificando ogni etichetta secondo le seguenti regole, applicate a qualsiasi sottoespressione:

- $\mathbb{1}\, e \rightsquigarrow e$ e $e\, \mathbb{1} \rightsquigarrow e$ ($\mathbb{1}$ è l'elemento neutro della concatenazione);
- $\mathbb{0} + e \rightsquigarrow e$ e $e + \mathbb{0} \rightsquigarrow e$ ($\mathbb{0}$ è l'elemento neutro della somma).

Allora l'automa generalizzato associato ad A' decide lo stesso insieme deciso dall'automa generalizzato associato ad A.

Dimostrazione. Per l'osservazione precedente, per ogni espressione e si ha

$$[\![\mathbb{1}e]\!] = [\![e\mathbb{1}]\!] = [\![e]\!], \qquad [\![\mathbb{0} + e]\!] = [\![e + \mathbb{0}]\!] = [\![e]\!].$$

Poiché $[\![\cdot]\!]$ è definita per induzione sulla struttura dell'espressione, sostituire una sottoespressione con un'altra che ha la stessa interpretazione non cambia l'interpretazione dell'espressione intera. Quindi ogni arco di A' ha la stessa interpretazione dell'arco corrispondente di A. Dato che l'insieme deciso da un automa dipende dagli archi solo tramite le interpretazioni delle loro etichette, gli automi risepttivamente di A e di A' decidono lo stesso insieme. □

Ripetendo le 3 operazioni definite sopra, possiamo ottenere l'epressione regolare associata ad ogni insieme regolare. Vediamo con un esempio come ottenere l'espressione regolare che decide un isnieme a partire da un automa che decide quell'insieme.

Esempio 6.14. Vogliamo scrivere l'espressione regolare che esprime l'insieme X deciso dal seguente automa:

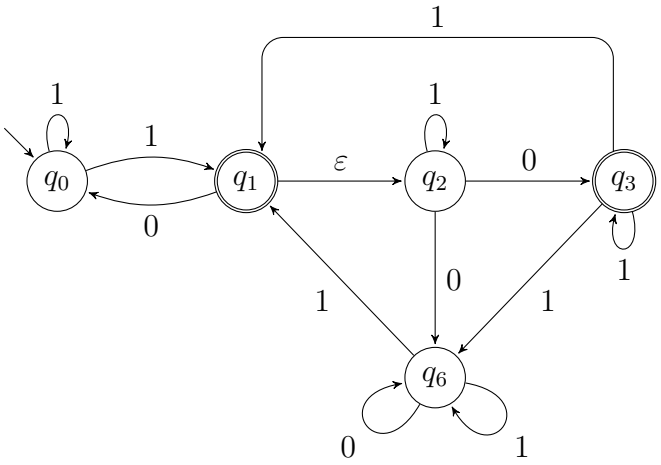


Figura 13: L'automa di partenza.

Consideriamo lo stesso automa come automa generalizzato: ogni arco è etichettato da un'espressione regolare, quindi ogni ε viene sostituito da $\mathbb{1}$.

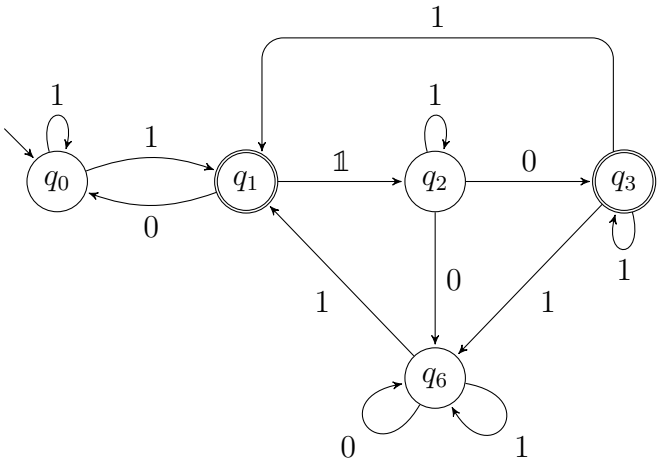


Figura 14: L'automa di partenza visto come automa generalizzato.

Per ottenere l'espressione regolare associata all'automa useremo le operazioni definite sopra: "Setup", "Merge" e "Erase". Nei grafici seguenti, i nuovi archi verranno tracciati in grassetto. Eseguiamo un'operazione alla volta:

Passo 1. Setup: aggiungiamo un nuovo stato iniziale q_0^*, collegato con un arco di etichetta $\mathbb{1}$ al vecchio stato iniziale q_0, e un nuovo stato q_F, che diventa l'unico stato accettante e che è collegato con archi di etichetta $\mathbb{1}$ ai vecchi stati accettanti (q_1 e q_3).

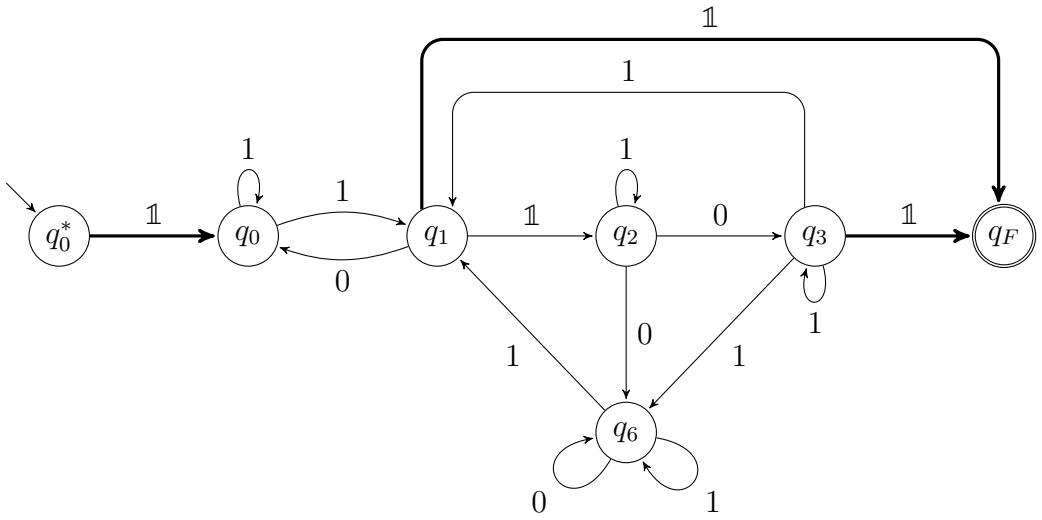


Figura 15: Passo 1, aggiunta di q_0^* e q_F.

Passo 2. Merge: gli unici archi paralleli sono i due cappi su q_6, di etichette 0 e 1. Li fondiamo in un unico cappio di etichetta $0 + 1$.

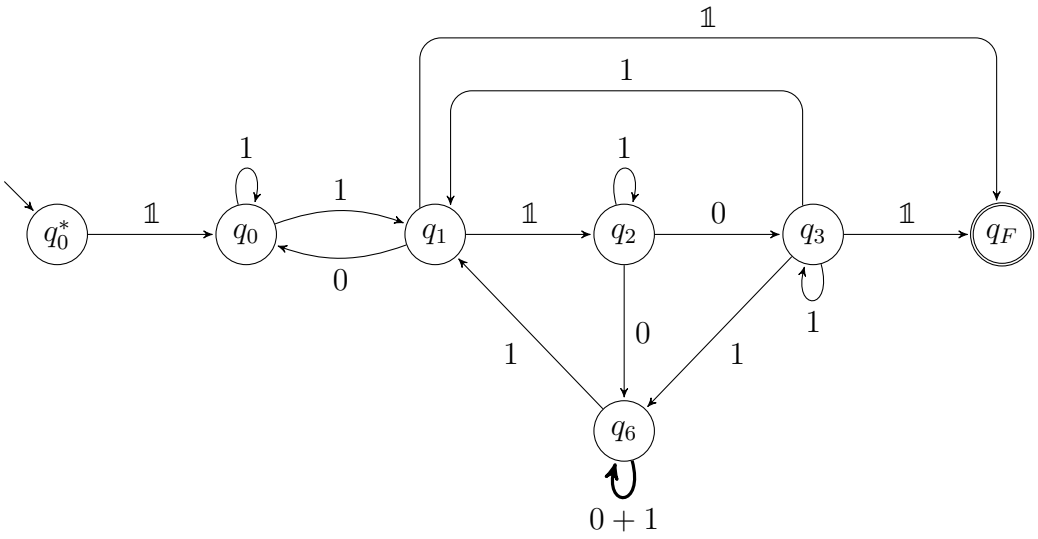


Figura 16: Passo 2, fusione dei due cappi su q_6.

Passo 3. Erase di q_2: per eliminare q_2 contiamo gli archi. Entranti: 1 (l'arco $\mathbb{1}$ da q_1). Uscenti: 2 (l'arco 0 verso q_3 e l'arco 0 verso q_6). Il cappio su q_2 ha etichetta 1. Eliminiamo q_2 e aggiungiamo $1 \cdot 2 = 2$ archi:

$$q_1 \xrightarrow{\mathbb{1}\,1^*\,0\,=\,1^*0} q_3, \qquad q_1 \xrightarrow{\mathbb{1}\,1^*\,0\,=\,1^*0} q_6.$$

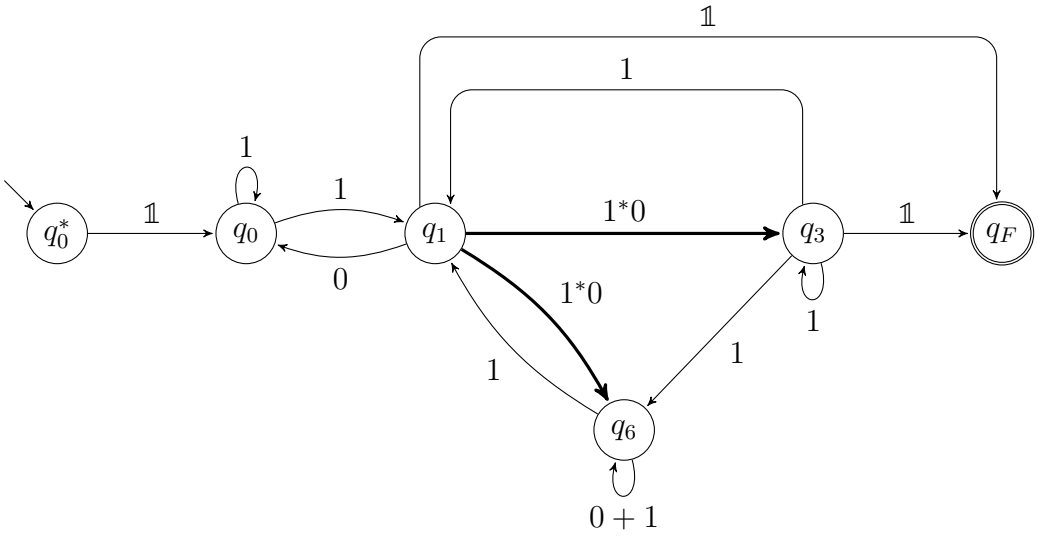


Figura 17: Passo 3, eliminazione di q_2.

Passo 4. Erase di q_6: non si sono formati nuovi archi paralleli, quindi ripetiamo Erase e cancelliamo q_6. Gli archi entranti sono 2 (da q_1 con 1^*0 e da q_3 con 1), l'arco uscente è 1 (verso q_1 con 1) e il cappio su q_6 ha etichetta $0 + 1$. Dobbiamo quindi

creare $2 \cdot 1 = 2$ archi:

$$q_3 \xrightarrow{1\,(0+1)^*\,1} q_1, \qquad q_1 \xrightarrow{1^*0\,(0+1)^*\,1} q_1.$$

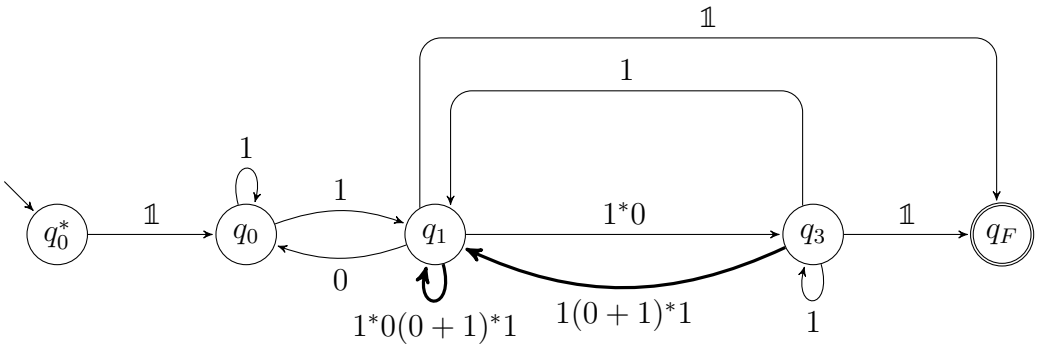


Figura 18: Passo 4, eliminazione di q_6.

Passo 5. Merge: ora ci sono due archi paralleli da q_3 a q_1, di etichette 1 e $1(0+1)^*1$. Li fondiamo ottenendo l'arco di etichetta $1 + (1(0+1)^*1)$.

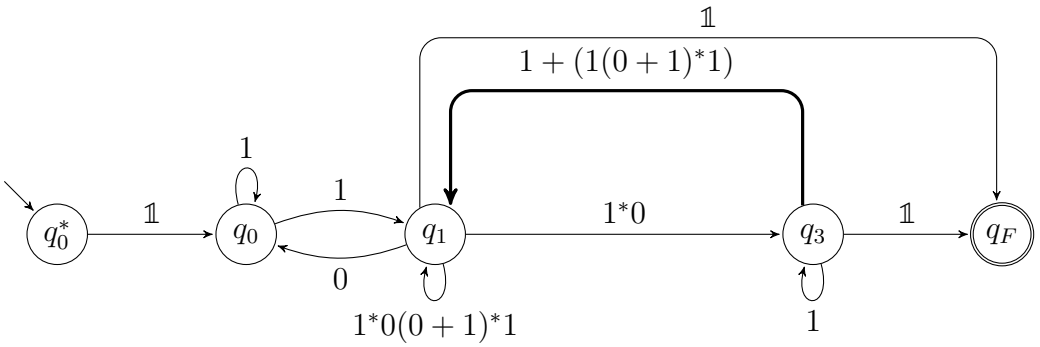


Figura 19: Passo 5, fusione degli archi paralleli da q_3 a q_1.

Passo 6. Erase di q_0: gli archi entranti in q_0 sono 2 (l'arco $\mathbb{1}$ da q_0^* e l'arco 0 da q_1), l'arco uscente è 1 (verso q_1) e il cappio ha etichetta 1. Dobbiamo quindi creare $2 \cdot 1 = 2$ archi:

$$q_0^* \xrightarrow{\mathbb{1}\,1^*\,1 = 1^*1} q_1, \qquad q_1 \xrightarrow{0\,1^*\,1} q_1.$$

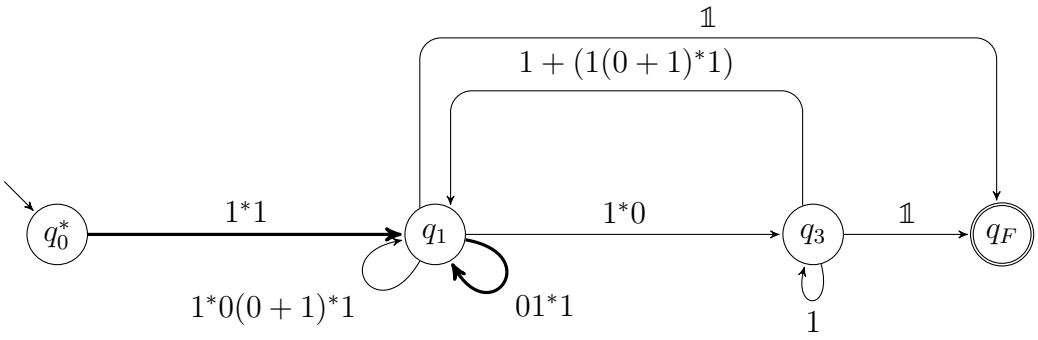


Figura 20: Passo 6, eliminazione di q_0.

Passo 7. Merge: su q_1 ci sono due cappi, di etichette $1^*0(0+1)^*1$ e 01^*1. Li fondiamo in un unico cappio di etichetta $01^*1 + 1^*0(0+1)^*1$.

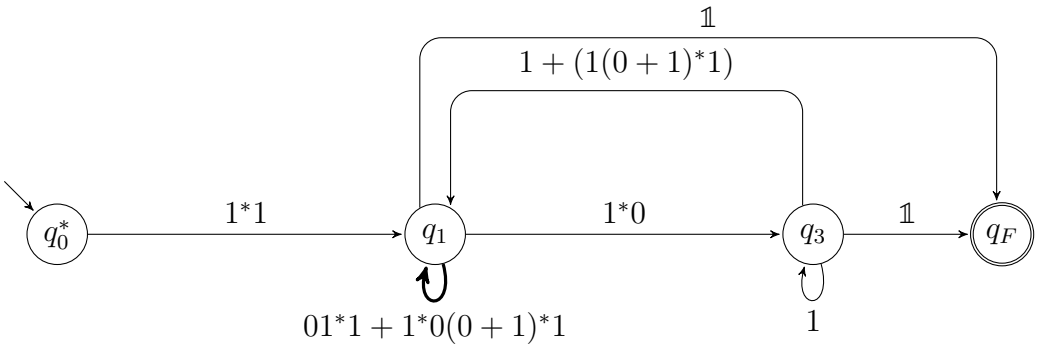


Figura 21: Passo 7, fusione dei due cappi su q_1.

Passo 8. Erase di q_3: l'arco entrante in q_3 è 1 (da q_1 con 1^*0), il cappio ha etichetta 1 e gli archi uscenti sono 2 (verso q_1 con $1+(1(0+1)^*1)$ e verso q_F con $\mathbb{1}$). Creiamo quindi $1 \cdot 2 = 2$ archi:

$$q_1 \xrightarrow{1^*0\,1^*\,(1+(1(0+1)^*1))} q_1, \qquad q_1 \xrightarrow{1^*0\,1^*\,\mathbb{1}\,=\,1^*01^*} q_F.$$

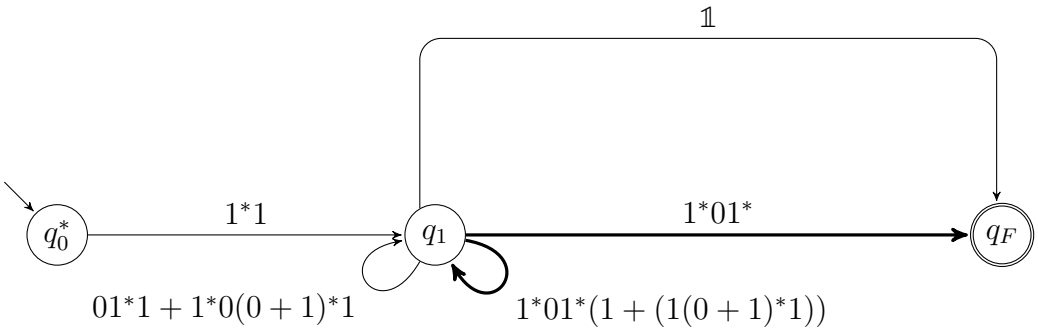


Figura 22: Passo 8, eliminazione di q_3.

Passo 9. Merge: ora abbiamo due coppie di archi paralleli, i due cappi su q_1 e i due archi da q_1 a q_F. Fondiamo entrambe le coppie. In particolare, poniamo

$$\gamma \;=\; 01^*1 \;+\; 1^*0(0+1)^*1 \;+\; 1^*01^*\big(1+(1(0+1)^*1)\big),$$

così il cappio su q_1 ha etichetta γ e l'arco da q_1 a q_F ha etichetta $\mathbb{1}+1^*01^*$.

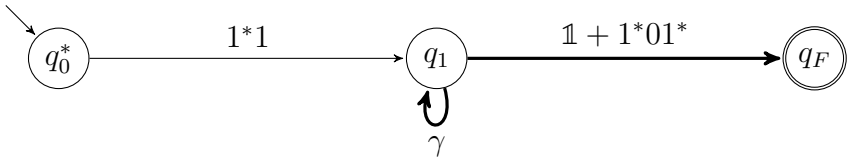


Figura 23: Passo 9, fusione dei cappi su q_1 e degli archi da q_1 a q_F.

Passo 10. Erase di q_1: ormai q_1 è l'ultimo nodo da cancellare. L'arco entrante è 1 (da q_0^* con 1^*1), il cappio ha etichetta γ e l'arco uscente è 1 (verso q_F con $\mathbb{1}+1^*01^*$). Creiamo quindi un solo arco, di etichetta $1^*1\,\gamma^*\,(\mathbb{1}+1^*01^*)$, che chiamiamo e.

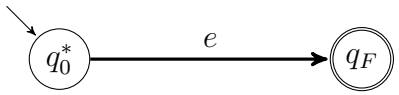


Figura 24: Passo 10, eliminazione di q_1. Rimangono solo q_0^* e q_F.

L'automa si riduce a $q_0^* \xrightarrow{e} q_F$. Poiché Merge ed Erase non cambiano l'insieme deciso, X è descritto dall'espressione regolare

$$e \;=\; 1^*1\,\Big(01^*1 \;+\; 1^*0(0+1)^*1 \;+\; 1^*01^*\big(1+(1(0+1)^*1)\big)\Big)^*\,\big(\mathbb{1}+1^*01^*\big).$$

Osservazione 6.15. Ovviamente, si può effettuare anche la procedura inversa: data un'espressione regolare e si può costruire un DFA che decide l'insieme $[\![e]\!]$. Elenchiamo la strategia: data un'espressione regolare e:

1. si costruisce il grafo generalizzato con due soli nodi, q_0^* e q_F, e un unico arco di etichetta e;
2. si scompongono le etichette in base alla loro struttura, cioè si applicano Merge ed Erase al contrario:
   - una somma $e_1 + e_2$ diventa due archi paralleli di etichette e_1 ed e_2;
   - una concatenazione e_1e_2 introduce un nodo intermedio;
   - una stella e^* introduce un nodo con un cappio di etichetta e, collegato con archi di etichetta $\mathbb{1}$;

   come già osservato, ogni passo non cambia l'insieme deciso;
3. quando tutte le etichette sono lettere o $\mathbb{1}$ si ottiene un automa generalizzato, che si trasforma in un ϵ-NDFA sostituendo le $\mathbb{1}$-mosse con le ϵ-mosse;
4. l'ϵ-NDFA si trasforma in un DFA con la costruzione per sottoinsiemi.

7 Macchina di Turing

7.1 Configurazione e algoritmo associato

Alla luce di quanto fatto prima, sembrerebbe naturale chiamare algoritmica qualsiasi procedura costruita a partire da un automa finito. In particolare è chiaro che una tale procedura può essere descritta in un linguaggio di programmazione e implementata in un qualsiasi sistema informatico. L'affermazione reciproca al contrario sembrerebbe troppo restrittiva: non sembra difendibile che ogni procedura che si voglia definire algoritmica debba essere realizzata mediante il modello di algoritmi associato agli automi finiti (più avanti vederemo degli esempi). Abolire completamente il carattere finito dell'insieme degli stati svuoterebbe di significato la nozione stessa di algoritmo. Un algoritmo è, per definizione, una procedura descritta da un insieme finito di istruzioni/regole — è proprio questo che lo rende "eseguibile", trascrivibile, comunicabile, implementabile in un programma. Se l'insieme degli stati fosse infinito, la "macchina" non sarebbe più descrivibile con una quantità finita di dati. Per conservare il carattere finitario auspicato senza rinunciare a fondamentali capacità di calcolo, si può prescindere dal principio che l'automa non modifica la parola iniziale, dunque nel leggere un certo simbolo dalla memoria potrà successivamente utilizzare questa informazione per modificare il proprio stato ma anche per modificare la memoria scrivendo sulla stessa un nuovo carattere. Questa variante risulta interessante solo se a sua volta lo scorrimento automatico dei caratteri in lettura viene modificato per permettere alla macchina di riesaminare i caratteri che sono stati modificati nei passi di calcolo precedenti, altrimenti la modifica che permette di scrivere non avrebbe alcun impatto sul calcolo effettuato.

Esempio 7.1. Consideriamo lgli insiemi

$$X_0 = \{0^n 1^m \mid n, m \in \mathbb{N}\}, \quad X_1 = \{0^n 1^n \mid n \in \mathbb{N}\}$$

Sebbene X_0 sia un insieme DFA-decidibile, X_1 non lo è. Infatti gli automi non sono in grando di "contare", quindi non si potrebbe contare il numero di occorenze di 1 e 0 (forniremo una dimostrazione più avanti).

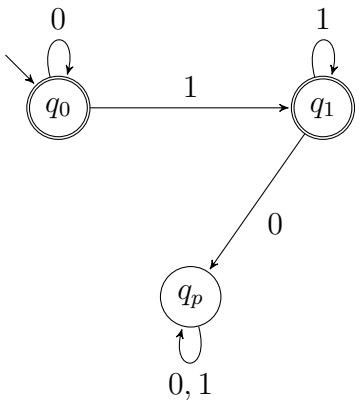


Figura 25: DFA che decide il linguaggio $\{0^n 1^m \mid n, m \in \mathbb{N}\}$.

In questa sezione vogliamo introdurre un modello di calcolo più forte degli automi.

Definizione 7.2. Una *configurazione ad un nastro di alfabeto A e insieme degli stati Q* è una tripla (f, p, q), dove

1. il *nastro* è una funzione $f : \mathbb{Z} \to A'$ dove $A' := A \cup \{\square\}$ (a volte indicato anche con $\tilde{A}$) e tale che $|f^{-1}(A)| < \infty$,
2. $p \in \mathbb{Z}$,
3. $q \in Q'$ dove $Q' = Q \cup \{q_F^0, q_F^1\}$ essendo q_F^0 e q_F^1 due stati che non appartengono a Q.

L'insieme delle configurazioni per un nastro di alfabeto A e insieme degli stati Q sarà denotato con

$$Conf_1^{A,Q} := \{(f, p, q) \text{ che verificano i punti (1), (2), e (3)}\}$$

Il criterio di arresto, che questa volta non è determinato dalla fine della lettura del nastro, deve essere specificato esplicitamente utilizzando un certo numero di stati terminali, che nella definizione sopra sono q_F^0 e q_F^1. Bisogna inoltre fissare uno stato particolare da utilizzare per la codifica della configurazione iniziale a partire dalla parola di ingresso. Questo stato iniziale indicato da $q_0 \in Q$ determina allora la configurazione iniziale tramite una funzione di input $\epsilon()$. Lo stato iniziale sarà d'ora in poi indicato da q_0.

Definizione 7.3 (Machcina di Turing)**.** Dato un alfabeto A e un insieme finito di stati Q, la *macchina di Turing di alfabeto A e stati Q* è un'applicazione

$$\mu : Q \times \tilde{A} \to Q' \times \tilde{A} \times \{+1, -1\}.$$

In realtà vorremmo che la macchina di Turing sia allo stesso tempo i seguenti tre oggetti:

$$Q \times A \to A, \qquad Q \times A \to \{-1, +1\}, \qquad Q \times A \to Q$$

Le precedenti funzioni rappresentano rispettivamente il sovrascrimento di un carattere, il cambio di stato e lo spostamento del puntatore. La funzione μ della definizione precedente formalizza in un'unica funzione i tre precedenti dati, infatti

$$\mu : Q \times \tilde{A} \to Q' \times \tilde{A} \times \{+1, -1\}$$

prende in input una coppia (q, a), dove:

- $q \in Q$ è lo *stato corrente* della macchina,
- $a \in \tilde{A}$ è il *carattere letto* sotto il puntatore (può essere un simbolo di A oppure il blank $\square$),

e restituisce una tripla (q', a', m), dove:

- $q' \in Q'$ è il *nuovo stato* in cui passa la macchina (può essere uno stato "di lavoro" in Q, oppure uno dei due stati terminali q_F^0, q_F^1 se il calcolo deve fermarsi),
- $a' \in \tilde{A}$ è il *carattere da scrivere* al posto di a nella cella corrente,

- $m \in \{+1, -1\}$ è lo *spostamento* del puntatore: $+1$ a destra, -1 a sinistra.

In altre parole, μ è la *tabella delle regole*: "se sono nello stato q e leggo il carattere a, allora scrivo a', mi sposto di m e passo allo stato q'".

Definizione 7.4. Sia μ una macchina di Turing di alfabeto A e insieme degli stati Q, l'*algoritmo formale* $\mathcal{A}_\mu$ associato a μ è la tupla $(A, \{0,1\}, Conf_1^{A,Q}, \epsilon, \tau, \delta)$: dove

1. l'insieme delle configurazioni è dato da
$$Conf_1^{A,Q}$$
2. la funzione di input $\epsilon : A^* \to Conf_1^{A,Q}$ dove $\epsilon(w) = (f_w, 1, q_0)$ con f_w corrisponde alla funzione nastro per cui
$$f_w(n) = \begin{cases} w(n) & \text{se } 1 \leq n \leq |w| \\ \square & \text{altrimenti} \end{cases}$$
3. la funzione di transizione
$$\tau : Conf_1^{A,Q} \to Conf_1^{A,Q}$$
è definita come $\tau((f,p,q)) = (f', p', q')$ dove supposta $\mu(q, f(p)) = (q', a', m)$ si porrà $p' = p + m$ e
$$f'(x) = \begin{cases} f(x) & \text{se } x \neq p \\ a' & \text{se } x = p. \end{cases}$$
4. la funzione di uscita δ associa ad una configurazione data (f,p,q) dove $q \in \{q_F^0, q_F^1\}$ il valore 0 se $q = q_F^0$ e il valore 1 se $q = q_F^1$.

Cerchiamo di capire come si esegue un passo di calcolo in questo caso.

Osservazione 7.5. Data una configurazione $(f,p,q) \in Conf_1^{A,Q}$ (nastro, posizione del puntatore, stato corrente), un passo di calcolo si esegue così:

1. si legge il carattere sotto il puntatore: $a := f(p)$;
2. si applica μ: $\mu(q,a) = (q', a', m)$;
3. si aggiorna lo stato: il nuovo stato è q';
4. si aggiorna la posizione: $p' = p + m$ (il puntatore si sposta di una cella a destra o sinistra);
5. si aggiorna il nastro: resta identico ovunque tranne che nella cella p, dove viene scritto a':
$$f'(x) = \begin{cases} f(x) & x \neq p \\ a' & x = p. \end{cases}$$

Questo definisce la funzione di transizione $\tau : Conf_1^{A,Q} \to Conf_1^{A,Q}$, $\tau(f,p,q) = (f',p',q')$.

Vediamo ora l'avvio e l'arresto del calcolo.

Osservazione 7.6.

- *Avvio:* data una parola di input $w \in A^*$, la configurazione iniziale è $\epsilon(w) = (f_w, 1, q_0)$: il nastro contiene w scritta a partire dalla cella 1 (blank altrove), il puntatore parte dalla cella 1 (inizio della parola), lo stato iniziale è q_0.
- *Iterazione:* si applica ripetutamente τ per ottenere la sequenza di configurazioni

  $$\epsilon(w), \tau(\epsilon(w)), \tau^2(\epsilon(w)), \dots$$

  questa è l'esecuzione del calcolo, passo dopo passo.
- *Arresto:* il calcolo si ferma quando si raggiunge una configurazione il cui stato è q_F^0 oppure q_F^1 (questi sono gli unici stati per cui μ non è più definita/rilevante, essendo terminali).
- *Output:* se ci si ferma in q_F^1 l'output è 1 (accetta); se ci si ferma in q_F^0 l'output è 0 (rifiuta). Se non ci si ferma mai, l'algoritmo non produce output su quell'input (il calcolo diverge).

In sintesi, μ è la "regola locale" (cosa fare dato stato+carattere letto); τ applica questa regola a una configurazione globale producendone la successiva; ϵ dice come tradurre l'input iniziale in una configurazione; l'iterazione di τ a partire da $\epsilon(w)$, fino a un eventuale stato terminale, è *l'esecuzione dell'algoritmo* $\mathcal{A}_\mu$ sull'input w.

7.2 Decidibilità per macchina di Turing

Definizione 7.7. Sia $X \subset A^*$, allora X si dice *decidibile tramite Macchina di Turing* (o *Turing-decidibile*) se esiste Q insieme finito di stati e una macchina di Turing di alfabeto A e insieme degli stati Q per cui l'algoritmo associato $\mathcal{A}_\mu$ decide l'insieme X.

Analogamente si ottiene la definizione di insieme *semi-decidibile* per macchina di Turing se esiste una macchina di Turing tale che l'algoritmo associato semi-decide X.

Proposizione 7.8. Ogni insieme decidibile per macchina di Turing è anche semidecidibile per macchina di Turing.

Dimostrazione. Per dimostrare questa proposizione è sufficiente costruire una macchina $\tilde{\mu}$ che per ogni parola $w \in A^*$ e $w \notin X$ entri in una configurazione non terminale. A questo scopo, sia μ la macchina che decide X e sia Q il suo insieme degli stati, andiamo a modificarla nelle transizioni che coinvolgono il suo stato finale non accettante. Consideriamo la macchina μ^* di alfabeto A e insieme degli stati $Q^* = Q \cup \{q^*\}$ che in corrispondenza delle transizioni di μ che coinvolgono lo stato non accettante opera una transizione verso una configurazione (f,p,q^*) per cui

$$\tau(f,p,q^*) = (f,p',q^*)$$

che porta la macchina in una computazione che non termina. Più precisamente, data la macchina

$$\mu : Q \times A' \longrightarrow Q' \times A' \times \{-1, +1\},$$

si costruisce la macchina

$$\mu^* : Q^* \times A' \longrightarrow Q^{*'} \times A' \times \{-1, +1\},$$

dove

$$Q^* = Q \cup \{q^*\}$$

e

$$\mu^*(q, a) = \begin{cases} \mu(q, a) & \text{se } \mu(q, a) = (q', a', m) \text{ e } q' \neq q_F^0, \\ (q^*, a, +1) & \text{altrimenti.} \end{cases}$$

□

Osservazione 7.9. Il modello di calcolo "macchina di Turing" è più potente del modello "automa a stati finiti", in quanto per ogni automa (T, q_0, F) resta definita una macchina di Turing μ sullo stesso alfabeto e sullo stesso insieme di stati per cui la computazione dell'algoritmo associato alla macchina μ coincide con l'algoritmo associato all'automa, più precisamente è sufficiente prendere

$$\mu(q, a) := \begin{cases} (T(q, a), a, +1) & a \neq \square \\ (q_F^1, \square, +1) & \text{se } q \in F \\ (q_F^0, \square, +1) & \text{se } q \notin F \end{cases}$$

Osservazione 7.10. Come nel caso degli automi possiamo associare ad ogni macchina un grafo etichettato che ha come vertici gli stati e le etichette sono della forma $a/b, m$ e indicano che se si legge a si sostituisce con b e si incrementa il puntatore di m.

Esempio 7.11. Vogliamo mostrare che l'insieme

$$X = \{0^n 1^n \mid n \in \mathbb{N}\}$$

è decidibile secondo macchina di Turing. Questo si può fare mediante una serie di cancellazioni successive della coppia di caratteri costituita dallo zero più a destra e dell'1 più a sinistra. Mostreremo più avanti che X non è DFA-decdibile (vedi proposizione 8.19).

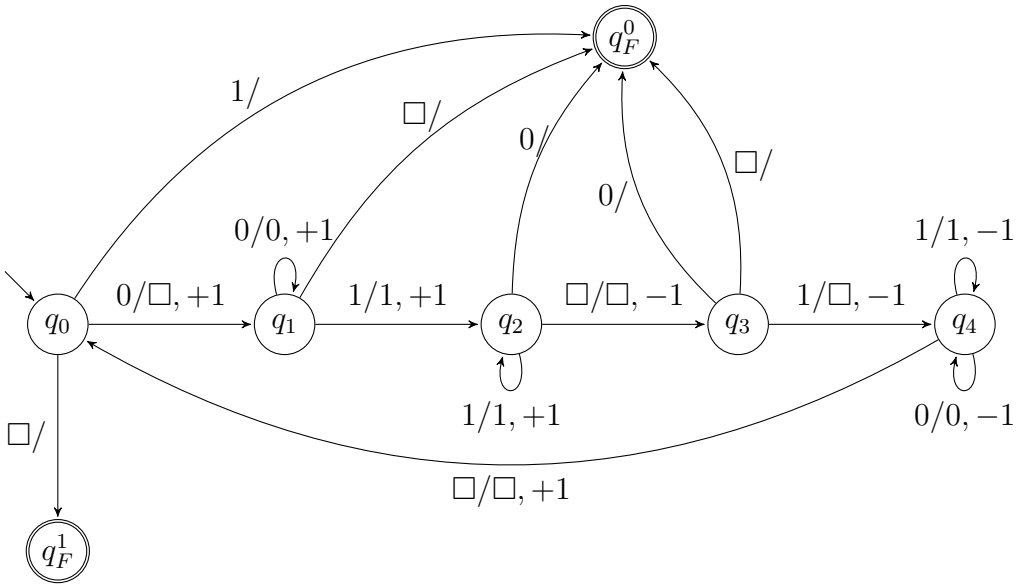


Figura 26: Macchina di Turing che decide $X = \{0^n 1^n \mid n \geq 0\}$.

8 Teoria della complessità

8.1 Classi di complessità

Definizione 8.1. Data una macchina di Turing μ di alfabeto A e insieme degli stati Q

$$\mu : Q \times A' \to Q' \times A' \times \{-1, +1\}$$

con $A' = A \cup \{\square\}$, $Q' = Q \cup \{q_F^0, q_F^1\}$:

1. sia
$$C = (c_0, \ldots, c_n)$$
una computazione di μ, il *costo in tempo di* C è dato dalla lunghezza di C diminuita di 1;

2. il *costo in spazio* è dato dalla cardinalità dell'insieme di interi p tali che almeno per una delle configurazioni $c_i = (f_i, p_i, q_i)$ si ha $p = p_i$:
$$|\{p \,|\, (f, p, q) \in C\}|;$$

3. si dirà che μ si arresta in tempo t (rispettivamente spazio s) sull'input w, se C è la computazione di μ a partire dalla configurazione iniziale associata a w, C raggiunge una configurazione terminale e il costo in tempo di C è t (rispettivamente il costo in spazio è s); in questo caso si userà la notazione $T_\mu(w) : A^* \to \mathbb{N}$ chiamata *funzione di tempo di arresto* (rispettivamente $S_\mu(w)$ $S_\mu(w) : A^* \to \mathbb{N}$ chiamata *funzione di spazio di arresto*) per indicare il tempo di arresto t (rispettivamente lo spazio di arresto s).

Di solito ci si interessa al comportamento nel caso peggiore:

$$T_{max}(n) := \max_{w \in A^*, |w| \leq n} T_\mu(w)$$

È anche possibile interessarsi al caso medio, allora si prenderebbe in considerazione non la funzione T_{max} ma una media su tutti i possibili input

$$T_{avg}(n) := \frac{\sum_{w \in A^N} T_\mu(w)}{|A^N|}$$

dove $A^N = \{w \in A^* \,|\, |w| \leq N\}$.

Si noti che la definizione di costo in tempo di una computazione associata ad una parola da una macchina di Turing si basa sulla lunghezza della computazione e pertanto si adatta a qualsiasi algoritmo formale (non necessariamente per una macchina di Turing). Viceversa, per quanto riguarda la nozione di costo in spazio la definizione è basata sulla specifica configurazione e pertanto non necessariamente è trasportabile nel caso di altri algoritmi formali.

Definizione 8.2. Sia μ una macchina di Turing sull'alfabeto A e con insieme di stati Q, e siano T, ed S due funzioni

$$T : \mathbb{N} \to \mathbb{N},$$

$$S : \mathbb{N} \to \mathbb{N}.$$

Si dice che μ si *arresta in tempo* T (rispettivamente *in spazio* S) se per ogni $w \in A^*$ di lunghezza al più uguale ad n la macchina si arresta sull'input w in tempo (risp. spazio) al più uguale a $T(n)$ (risp. $S(n)$). In formule

$$T_\mu(w) \leq T(n) \,(\text{risp. } S_\mu(w) \leq S(n)), \quad \forall w \text{ tale che } |w| \leq n$$

Osservazione 8.3. Vorremmo dare una forma più precisa alle funzioni T che danno informazioni sul tempo e lo spazio di arresto di una data macchina di Turing. Consideriamo quindi una macchina di Turing μ che si arresta in tempo T e due parole a e b tali che $|a| < |b|$. Abbiamo che l'insieme delle parole di lunghezza al più $|a|$ è un sottoinsieme delle parole di lunghezza al più $|b|$. Di conseguenza, il tempo (o lo spazio) limite richiesto per elaborare le prime non ha senso che superi il limite per le seconde; è naturale quindi imporre che $T(|a|) \leq T(|b|)$, restringendo così lo studio alle funzioni non decrescenti. Inoltre, mostreremo più avanti che fissato un intero N, allora la computazione si arresta in tempo al più N a partire da una qualsiasi parola di di lunghezza al più N. Di conseguenza, chiederemo anche che $T(n) \geq n$ definitivamente.

Hanno senso quindi le seguenti definizioni.

Definizione 8.4. Una funzione

$$T : \mathbb{N} \to \mathbb{N}$$

viene detta *funzione di complessità* se è una funzione non decrescente e se $T(n) \geq n$ definitivamente.

Definizione 8.5. Siano T ed S funzioni di complessità.

1. La macchina μ di alfabeto A e insiemi di stati Q *decide l'insieme* X *in tempo* T (risp. *spazio* S) se μ decide X e se esiste un intero N tale che per ogni intero $n \geq N$ e per ogni parola $w \in A^*$ di lunghezza al più uguale a n la macchina μ si ferma a partire da w in tempo al più uguale a $T(n)$ (risp. spazio al più uguale ad $S(n)$).

2. La *classe di complessità* $\mathrm{DTIME}_1^{A,Q}(T)$ (risp. $\mathrm{DSPACE}_1^{A,Q}(S)$) è l'insieme che contiene gli insiemi $X \subset A^*$ per i quali esiste almeno una macchina di Turing di alfabeto A che decide X in tempo T (risp. in spazio S). Quando non abbiamo interesse a specificare l'insieme degli stati scriveremo solo $\mathrm{DTIME}_1^{A}(T)$ e $\mathrm{DSPACE}_1^{A}(S)$.

Con abuso di notazione, spesso scriveremo $T(n)$ invece che T.

Osservazione 8.6. L'indice 1 in basso utilizzato in questa notazione fa riferimento al fatto che nella macchina viene utilizzato un solo nastro.

Esempio 8.7. Con questa notazione si può notare che l'insieme X degli interi divisibili per 9 appartengono alle classi $\mathrm{DTIME}_1^{\{0,1,\ldots,9\}}(n)$ e $\mathrm{DSPACE}_1^{\{0,1,\ldots,9\}}(n)$.

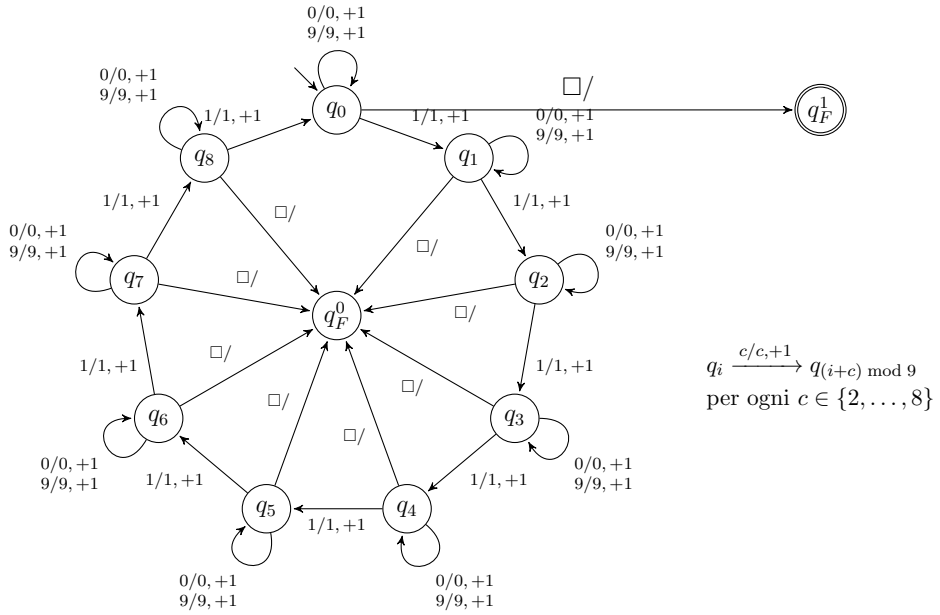


Figura 27: Macchina di Turing che decide la divisibilità per 9. Gli stati $q_0, \ldots, q_8$ memorizzano il resto modulo 9 della somma delle cifre lette finora, con q_0 stato iniziale. Leggendo una cifra c in q_i, la macchina la riscrive invariata, si sposta a destra e passa in $q_{(i+c) \bmod 9}$: i cappi corrispondono alle cifre 0 e 9, gli archi tra stati consecutivi alla cifra 1, mentre le cifre $2, \ldots, 8$ sono riassunte nella legenda. Alla lettura del simbolo vuoto □, la macchina accetta in q_F^1 se si trova in q_0 (resto 0) e rifiuta in q_F^0 se si trova in uno degli stati $q_1, \ldots, q_8$.

Esempio 8.8. Anche ogni macchina di Turing associata ad un insieme DFA-decidibile appartiene alla classe $\mathrm{DTIME}_1^A(n)$, infatti un automa per decidere una parola di lunghezza n la deve leggere tutta e visto che la macchina associata dovrà fermarsi appena letta tutta la aprola in input, farà esattamente n passi.

Osservazione 8.9. Esistono comunque dei limiti per questa definizione. Fissiamo l'alfabeto $A = \{0, 1\}$ e consideriamo l'insieme $X = \{w \in A^* \mid x(1) = 1\}$. Chiaramente, costruire una macchina di Turing che decida l'insieme X è facile. Infatti, la macchina in questione, che indicheremo con μ leggerà la prima lettera di ogni parola e poi passerà subito in uno degli stati finali. Questa macchina, indipendente dalla lunghezza della parola in input, compierà solamente un'operazione. Quindi, vale che

$$T_\mu(w) = 1, \quad \forall w \in A^*$$

Di conseguenza, definendo

$$\begin{array}{rccc} T_1: & \mathbb{N} & \to & \mathbb{N} \\ & n & \mapsto & 1 \end{array}$$

segunedo la definzione 8.2, possiamo dire che la macchina di Turing μ si arresta in tempo T_1. Vorremmo a questo punto dire che l'insieme X appartenga alla classe di complessità

$\mathrm{DTIME}_1^A(T)$. Essendo però T_1 la funzione costante 1, non potrà mai soddisfare la condizione $T_1(n) \geq n$, e quindi non sarà vera neanche definitivamente. Di conseguenza, la funzione T_1 non è una funzione di complessità; inoltre, seguendo questa definizione, non ha proprio senso considerare l'insieme $\mathrm{DTIME}_1^A(T_1)$. Seguendo questa struttura, il meglio che si può dire è che $X \in \mathrm{DTIME}_1^A(T_n)$ dove

$$\begin{array}{rccc} T_n: & \mathbb{N} & \to & \mathbb{N} \\ & n & \mapsto & n \end{array}$$

Questo ha senso poichè T_n è effettivamente una funzione di complessità.

Esempio 8.10. Calcoliamo a quale classe di complessita DTIME_1^A appartiene l'insieme $X = \{0^n 1^n \,|\, n \in \mathbb{N}\}$ dell'esempio 7.11. Per farlo usiamo la macchina di Turing esibita nell'esempio (vedi figura 26). Se la parola appartiene ad X sarà del tipo $w = 0^p 1^p$ per p tale che $n = 2p$, e pertanto con $4p + 1$ passi di computazione si finisce per aver eliminato una coppia di caratteri. Questo perchè:

- si fa 1 passo per leggere il primo "0", sostituirlo con un blank e spostarsi a destra;
- si fanno $2p$ passi: $2p - 1$ per scorrere tutti i restanti caratteri utili verso destra, più 1 passo per leggere il blank di fine stringa e riposizionarsi a sinistra sull'ultimo "1";
- si fa 1 passo per leggere l'ultimo "1", sostituirlo con un blank e spostarsi a sinistra;
- si fanno $2p - 1$ passi: $2p - 2$ per scorrere a ritroso i caratteri centrali rimasti, più 1 passo per leggere il blank iniziale e riposizionarsi a destra sul nuovo primo "0".

Quindi, si effettuano

$$1 + 2p + 1 + (2p - 2) + 1 = 4p + 1$$

passi per eliminare un "1" e uno "0" partendo da una stringa lunga $2p$. Di conseguenza, alla seconda iterazione serviranno $4(p-1)+1$ e così via, fino a quando la stringa è lunga 2 e si compiono quindi $4(p-(p-1))+1$ passi. Pertanto il tempo di arresto t_n per una parola di lunghezza $n = 2p$ sarà

$$\begin{aligned} t_n &= \sum_{i=0}^{p-1}(4(p-i)+1) = 4\left(\sum_{i=0}^{p-1}(p-i)\right) + \sum_{i=0}^{p-1} 1 \\ &= 4\left(\sum_{i=0}^{p-1} p - \sum_{i=0}^{p-1} i\right) + p = 4\left(p^2 - \frac{p(p-1)}{2}\right) + p \\ &= 4\left(\frac{2p^2 - p^2 + p}{2}\right) + p \\ &= 2\left(p^2 + p\right) + p \\ &= 2p^2 + 3p \end{aligned}$$

ovvero

$$t_n = \frac{n^2}{2} + \frac{3}{2}n.$$

Ovviamente se la parola non è accettata, allora il tempo di arresto sarà minore o uguale a t_n. Trattiamo anche il caso in cui la parola ha lunghezza dispari, ovvero quando la parola ha lunghezza $2p+1$. Il caso peggiore da considerare è quello in cui la parola ha una forma del tipo 0^p1^{p+1}. In questo caso, usando lo stesso conto di prima, la parola viene analizzata e scartata in un tempo strettamente minore di t_{2p+2}. Di conseguenza, definendo la funzione di complessità seguente

$$\begin{array}{rccl} t: & \mathbb{N} & \to & \mathbb{N} \\ & n & \mapsto & \begin{cases} t_n & \text{se } n = 2p \\ t_{n+1} & \text{se } n = 2p+1 \end{cases} \end{array}$$

che corrisponde a

$$\begin{array}{rccl} t: & \mathbb{N} & \to & \mathbb{N} \\ & n & \mapsto & \begin{cases} \frac{n^2}{2} + \frac{3}{2}n & \text{se } n \text{ è pari} \\ \frac{(n+1)^2}{2} + \frac{3}{2}(n+1) & \text{se } n \text{ è dispari} \end{cases} \end{array}$$

abbiamo che $X \in \mathrm{DTIME}_1^A(t)$. Mostreremo anche che X non appartiene a nessuna classe di complessità lineare (vedi proposizione 8.19).

8.2 Look-up Table

Il lemma seguente mostra che il valore puntuale delle funzioni di complessità non corrisponde alla reale difficoltà del problema, in quanto, data una macchina di Turing che decide un problema X con un certo tempo di arresto, è sempre possibile trovarne un'altra che decide lo stesso problema con complessità lineare per tutte le parole di input fino ad una lunghezza fissata, dunque la nozione di complessità ha senso come misura della difficoltà di un problema solo quando si considera il comportamento asintotico dell'algoritmo.

Lemma 8.11 (Look-up Table)**.** Dato un insieme $X \subset A^*$, supponiamo che esista una macchina di Turing μ che lo decide in tempo $T(n)$ e spazio $S(n)$. Per un intero qualsiasi N siano

$$T'(n) = \begin{cases} n & n < N \\ T(n) + 2N & \text{altrimenti} \end{cases}$$

e sia

$$S'(n) = \begin{cases} n & n < N \\ S(n) & \text{altrimenti} \end{cases}$$

allora esiste una macchina di Turing μ' che decide X in tempo $T'(n)$ e spazio $S'(n)$.

Dimostrazione. Si può modificare la macchina μ per includere nell'insieme degli stati direttamente la risposta che corrisponde ad una data parola di input (in un numero finito di casi).

Ed è quello che faremo nel caso delle parole di lunghezza inferiore uguale ad N, in questo modo la nuova macchina si fermerà non appena sarà stata letta la parola di input. In questo caso la macchina dovrà leggere il nastro e quindi si arresterà in tempo al più N. In caso

fossimo in presenza di una parola più lunga bisognerebbe riprendere la computazione della macchina M, reimpostando la macchina nello stato iniziale di M e nella posizione iniziale.

Sia

$$Q' = (Q \setminus \{q_0\}) \cup \{q_w \mid w \in A^N\} \cup \{q_R, q_0^\mu\}$$

dove lo stato q_R sarà utilizzato come lo stato utilizzato per il ritorno alla casella iniziale nel caso in cui l'input si trovi più lungo di N simboli.

Dunque la macchina di Turing μ' è definita a partire dalla macchina μ:

$$\mu'(q,a) = \begin{cases} \mu(a,q) & \text{se } q \in (Q \setminus \{q_0\}) \cup \{q_0^\mu\}, \\ (q_{wa}, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N \text{ e } |wa| \leq N, \\ (q_R, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } a \neq \square \text{ e } |w| = N, \\ (q_R, a, -1) & \text{se } q = q_R \text{ e } a \neq \square, \\ (q_0^\mu, a, +1) & \text{se } q = q_R \text{ e } a = \square, \\ (q_F^1, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N,\ a = \square \text{ e } w \in X, \\ (q_F^0, a, +1) & \text{se } q = q_w \text{ per qualche } w \in A(m) \text{ con } m \leq N,\ a = \square \text{ e } w \notin X; \end{cases}$$

Lo stato iniziale q_0' di μ' coincide con lo stato associato alla parola vuota q_ϵ mentre lo stato q_0 di μ viene pensato come lo stato aggiuntivo q_0^μ. Per ogni parola di lunghezza inferiore o uguale ad N la computazione viene effettuata in modo simile alla computazione per automa finito, per le altre parole la macchina μ' compie una lettura ed un ritorno alla posizione iniziale che non altera l'input e dunque quando la macchina torna alla posizione iniziale e passa nello stato iniziale (di μ) la computazione si svolge esattamente come per μ. □

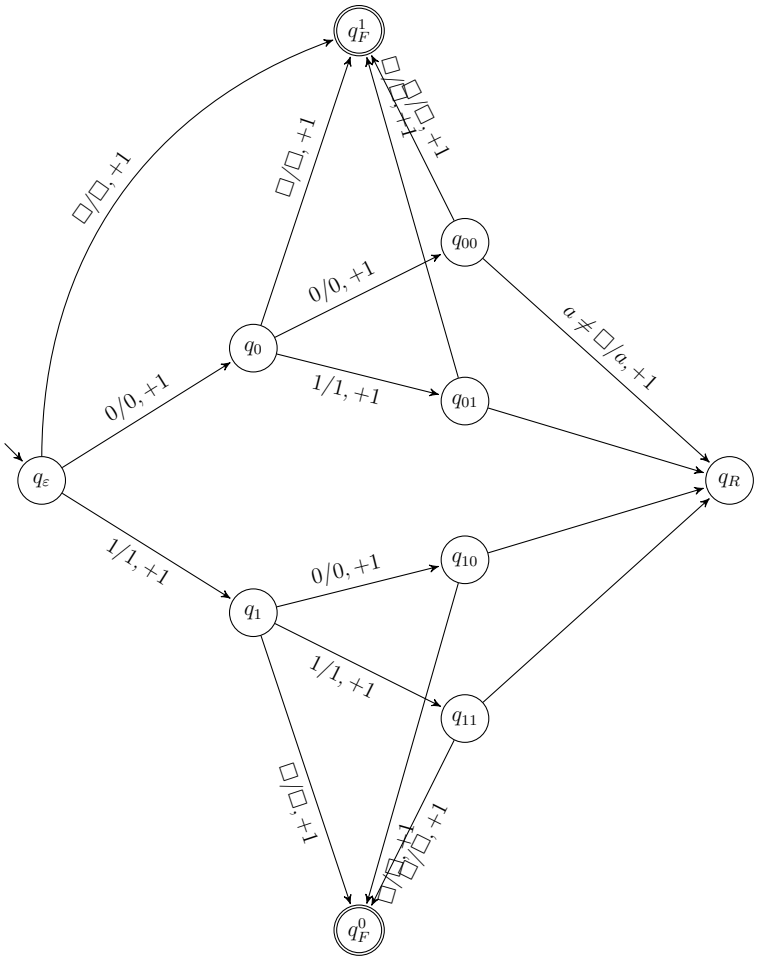


Figura 28: Stati q_w di μ' per $A = \{0, 1\}$, $N = 2$ e $X \cap A^{\leq 2} = \{\varepsilon, 0, 00, 01\}$: le parole in X portano in q_F^1, le altre in q_F^0.

Osservazione 8.12. Il nome "Look-up Table" nasce per il seguente motivo. La macchina che si costruisce nella dimostrazione è la macchina di partenza ma che classifica già la decidibilità delle parole di lunghezza N. Poprio come se la macchina, alla prima lettura della parola, dovesse controllare in una "tavola" se la parola c'è (allora è accettata) o meno (allora viene scartata). Se la parola invece è più lunga di N, una volta che vengono letti i primi N caratteri, la macchina capisce che ci sono altri caratteri e allora, facendo altri N passi, torna all'inizio della parola e comincia ad analizzare la parola usando la strategia della macchina di Turing di partenza.

Osservazione 8.13. Il significato di questo lemma è che ha senso analizzare la complessità (dell'algoritmo di decisione associato), soltanto relativamente ad un insieme X infinito, poiché nel caso di un X finito si può sempre decidere il problema in un numero di passi finito, pari alla lunghezza della sua codifica sull'alfabeto (associando alla parola uno stato "ad hoc" e facendo una transizione accettante in base alla parola). Inoltre, poiché è possibile decidere il complementare di un insieme decidibile (invertendo nella tabella lo stato accettante q_F^1 e quello non accettante q_F^0), in pratica la questione della decidibilità si pone solo quando X

è infinito e co-infinito (complementare infinito), mentre è banale quando l'insieme è finito oppure co-finito (complementare finito).

8.3 Relazioni tra le classi DTIME e DSPACE

Proposizione 8.14. Sia $T : \mathbb{N} \to \mathbb{N}$ una funzione di complessità; le inclusioni seguenti sono sempre vere:

$$\mathrm{DTIME}_1^{A,Q}(T) \subset \mathrm{DSPACE}_1^{A,Q}(T) \subset \bigcup_{\epsilon>0} \mathrm{DTIME}_1^{A,Q}(2^{\epsilon T}).$$

Dimostrazione. Visto che le definizioni delle due classi di complessità sono tutte "definitive" (ovvero, si richiede la condizione da un certo valore in poi) possiamo evitare di scegliere il valore da cui partire e discutere dirattemnte il caso "definitivo". Fissato l'alfabeto A, sia $X \in \mathrm{DTIME}_1^A(T)$. Poiché ad ogni passo di computazione la testina può spostarsi di al massimo una cella, la massima distanza raggiungibile a partire dalla posizione iniziale in $T(n)$ passi non può essere maggiore di $T(n)$. Di conseguenza, lo spazio utilizzato è banalmente limitato da $T(n)$, provando la prima inclusione.
Dimostriamo ora la seconda inclusione. Sia μ una macchina di Turing di alfabeto A e insieme degli stati Q, che decide un insieme in spazio $s = T(n)$. Poniamo $k = |A|$ e $d = |Q|$. Una configurazione completa della macchina di Turing è univocamente determinata da tre elementi:

1. la posizione della testina: muovendosi al massimo di uno spazio s a destra o a sinistra rispetto all'origine, la testina può visitare al massimo $2s + 1$ posizioni, le cella inziale corrisponendente a 0 e s caselle a destra oppure s a sinistra.

2. Il contenuto del nastro: da quanto detto sopra, le celle potenzialmente modificate sono al massimo $2s + 1$. Ognuna di esse può contenere uno dei k simboli dell'alfabeto, per un totale di k^{2s+1} combinazioni possibili.

3. Lo stato interno: La macchina, ad ogni passo, può trovarsi in uno qualsiasi dei d stati.

Il numero massimo di configurazioni distinte possibili durante la computazione è dato dal prodotto di queste scelte:

$$N_{\text{config}} = (2s + 1) \cdot k^{2s+1} \cdot d$$

Inoltre, se la macchina computasse per un numero di passi strettamente maggiore di N_{config}, per il principio dei cassetti visiterebbe necessariamente almeno due volte la stessa identica configurazione. Essendo μ deterministica, questo implicherebbe l'ingresso in un ciclo infinito. Tuttavia, poiché μ decide un insieme, essa per definizione si arresta sempre (sia nei casi accettanti sia nei casi non). Si deduce quindi che il tempo di arresto t è limitato dal numero di configurazioni:

$$t \leq (2s + 1) \cdot k^{2s+1} \cdot d$$

Passando in base 2, possiamo riscrivere il limite superiore come:

$$t \leq 2^{\log_2(2s+1)+(2s+1)\log_2(k)+\log_2(d)}$$

Poiché $\log_2(2s+1)$ cresce molto più lentamente di s, e i termini associati a k e d sono costanti dipendenti dalla macchina, per un s sufficientemente grande l'intero esponente cresce asintoticamente in modo lineare rispetto a s. Esisterà quindi certamente una costante $c > 0$ tale per cui l'esponente sia maggiorato da $c \cdot s$. Da cui si ottiene:

$$t \leq 2^{cs}$$

Sostituendo $s = T$ e ricordando che μ è arbitraria, si conclude che la classe spaziale è contenuta nell'unione delle classi temporali delimitate da 2^{cT}, ovvero $2^{\epsilon T}$ per un opportuno $\epsilon > 0$. Quindi, dato un $X \in \mathrm{DSPACE}_1^{A,Q}(T)$ dovrà esistere un $\epsilon \in \mathbb{N}$ tale che $X \in \mathrm{DTIME}_1^{A,Q}(2^{\epsilon T})$. Da cui segue la tesi. □

8.4 Pumping lemma

In questa sezione, vogliamo dare l'enunciato del Pumping Lemma che fornisce una condizione necessaria soddisfatta da ogni insieme decidibile in tempo lineare per macchina di Turing. Mostrare che la condizione è soddisfatta non permette di dedurre che un insieme appartiene alla classe di complessità lineare. D'altra parte permette di collocare ogni insieme che non la soddisfa al di fuori di essa.

Prima di dare l'enunciato, diamo una definizione e mostriamo un lemma tecnico cruciale per la dimostrazione del Pumping lemma.

Definizione 8.15 (Sequenza degli stati di passaggio)**.** Sia μ una macchina di Turing con insieme di stati Q, la cui testina parte dalla prima cella del nastro, e sia

$$(c) = (c_0, c_1, \ldots, c_T), \qquad c_i = (f_i, p_i, q_i),$$

la computazione di μ su una parola $w \in A^*$ con $|w| = n$, dove f_i è il contenuto del nastro, p_i la posizione della testina e $q_i \in Q$ lo stato all'istante i.

Fissata una cella $p \geq 1$, diciamo che l'istante i (con $0 \leq i < T$) è un *passaggio sulla cella* p se la testina attraversa il bordo tra la cella p e la cella $p+1$ nel passaggio da c_i a c_{i+1}, ovvero se

$$(p_i, p_{i+1}) = (p, p+1) \quad \text{oppure} \quad (p_i, p_{i+1}) = (p+1, p).$$

Siano $i_1 < i_2 < \cdots < i_r$ gli istanti di passaggio sulla cella p. La *sequenza degli stati di passaggio* di (c) nella cella p è

$$S_w(p) := (q_{i_1+1}, q_{i_2+1}, \ldots, q_{i_r+1}) \in Q^*,$$

dove q_{i_s+1} è lo stato in cui si trova la macchina subito dopo l's-esimo passaggio. La sua lunghezza $|S_w(p)| = r$ è il *numero di passaggi* nella cella p.

Lemma 8.16 (Ripetizione delle sequenze di passaggio)**.** Sia μ una macchina di Turing con insieme di stati Q, $d = |Q| \geq 2$, e sia $k \geq 1$ un intero. Sia $w \in A^*$ con $|w| = n$ tale che la computazione di μ su w abbia lunghezza al più kn e che

$$n \geq (k+1)\, d^{k+1}.$$

Allora esistono due posizioni $1 \leq p_1 < p_2 \leq n$ tali che

$$S_w(p_1) = S_w(p_2).$$

Dimostrazione. Supponiamo per assurdo che le sequenze $S_w(1), \ldots, S_w(n)$ siano a due a due distinte.
Sia N_i il numero di celle p con $1 \leq p \leq n$ in cui si effettuano esattamente i passaggi:

$$N_i = |\{p \mid 1 \leq p \leq n,\ |S_w(p)| = i\}|, \qquad n = \sum_{i=0}^{\infty} N_i.$$

Il prodotto iN_i conta i passi di computazione che attraversano il bordo di una cella con esattamente i passaggi. Sommando su tutti gli i si ottiene il numero totale di attraversamenti nelle celle $1, \ldots, n$, che non supera la lunghezza della computazione, cioè kn:

$$\sum_{i=0}^{\infty} iN_i \leq kn.$$

Minorando i con $k+1$ sulle celle con più di k passaggi si ottiene

$$(k+1) \sum_{i=k+1}^{\infty} N_i \leq \sum_{i=k+1}^{\infty} iN_i \leq \sum_{i=0}^{\infty} iN_i \leq kn. \tag{1}$$

Siano N^- il numero di celle con al più k passaggi e $N^+ = \sum_{i=k+1}^{\infty} N_i$ il numero delle altre. Per l'ipotesi di assurdo le celle contate da N^- hanno sequenze a due a due distinte, di lunghezza $0, 1, \ldots, k$. Poiché ci sono d^i sequenze di lunghezza i e $d \geq 2$,

$$N^- \leq \sum_{i=0}^{k} d^i = \frac{d^{k+1}-1}{d-1} \leq d^{k+1} - 1 < d^{k+1}.$$

Quindi

$$\sum_{i=k+1}^{\infty} N_i = N^+ = n - N^- > n - d^{k+1}. \tag{2}$$

Combinando la (1) con la (2) otteniamo

$$n - d^{k+1} < \frac{kn}{k+1},$$

cioè $(k+1)(n - d^{k+1}) < kn$, ovvero

$$n < (k+1)\, d^{k+1},$$

in contraddizione con l'ipotesi $n \geq (k+1)d^{k+1}$. Dunque l'ipotesi di assurdo è falsa e due posizioni $p_1 < p_2$ hanno la stessa sequenza di passaggio. □

Siamo ora pronti per la dimostrazione del lemma.

Lemma 8.17 (Pumping Lemma)**.** Supponiamo che $X \in \mathrm{DTIME}_1^{A,Q}(kn)$. Allora esiste $P > 0$ tale che per ogni $w \in X$ con $|w| = n \geq P$ esiste una fattorizzazione $w = uxv$ con $u, x, v \in A^*$ e $|x| > 0$ per cui, per ogni $m \geq 0$,

$$ux^m v \in X.$$

Dimostrazione. Per ipotesi esistono una macchina μ con insieme di stati Q che decide X e un intero N tali che, per ogni $n \geq N$ e ogni $w \in A^*$ con $|w| = n$, la computazione di μ su w ha lunghezza al più kn. Sia $d = |Q|$ (che si può supporre ≥ 2, aggiungendo se serve uno stato inutilizzato) e poniamo

$$P := \max\big((k+1)d^{k+1},\, N\big).$$

Sia $w \in X$ con $|w| = n \geq P$. Allora $n \geq N$, quindi la computazione di μ su w ha lunghezza al più kn, e $n \geq (k+1)d^{k+1}$. Sono esattamente le ipotesi del Lemma 8.16, che fornisce due posizioni $1 \leq p_1 < p_2 \leq n$ tali che

$$S := S_w(p_1) = S_w(p_2).$$

Siano u la parola formata dalle prime p_1 celle, x quella formata dalle celle da $p_1 + 1$ a p_2, e v quella formata dalle celle da $p_2 + 1$ a n. Allora $w = uxv$ e $|x| = p_2 - p_1 > 0$. Sia $r = |S|$ e siano $s_1, \ldots, s_r$ gli stati di S, nell'ordine. Mostriamo che $ux^m v \in X$ per ogni $m \geq 0$, distinguendo i casi.

$m = 1$: È la parola $w = uxv$, che appartiene a X per ipotesi.

$m = 2$: Più precisamente, supponiamo che a partire da uxv la computazione entri per la prima volta nel segmento di parola x (che coinvolge le celle con indici che vanno da $p_1 + 1$ a p_2) provenendo da sinistra: lo fa attraversando il bordo p_1, nello stato s_1. Dopo un certo numero di transizioni la testina uscirà dalla zona x tornando a sinistra attraverso p_1 oppure attraversando p_2; finché non attraversa p_2 per la prima volta, ogni uscita e rientro attraverso p_1 avviene come in uxv. Per ipotesi la prima volta che si attraversa p_2 lo si fa nello stato s_1, lo stesso con cui si era entrati in x la prima volta da p_1. Dunque, nel caso della parola di input $uxxv$, la macchina entra nella seconda copia di x nello stato s_1 e ripete gli stessi passi di transizione avvenuti nel passaggio precedente.

Quando finalmente si uscirà dalla seconda zona x ed entrerà nel fattore v (dato che la testina non può saltare celle, per passare da u a v deve attraversare sia p_1 sia p_2; inoltre, poiché $S_w(p_1) = S_w(p_2)$, il j-esimo passaggio avviene nella stessa direzione e nello stesso stato s_j su entrambi i bordi), lo farà nello stato s_1, cioè lo stesso con cui entrava in v durante la computazione su uxv, e la macchina effettuerà le stesse transizioni fatte nell'attraversamento di v in quella computazione. Analogamente si otterrà un comportamento identico quando la testina tornerà indietro: ogni volta che da v rientra nella seconda copia di x lo fa nello stato s_j con cui rientrava in x da destra durante la computazione su uxv; ogni volta che dalla seconda copia di x esce verso sinistra, entra nella prima copia nello stato s_j con cui da x si rientrava in u; e così via. In ogni attraversamento di ciascuno dei bordi p_1 e p_2 ciascuna parte riceve dall'altra gli stessi stati, nello stesso ordine, di quelli che riceveva in uxv, perché le due sequenze di passaggio coincidono.

Comunque, alla fine della computazione, lo stato terminale raggiunto dalla computazione che parte dalla parola uxv sarà lo stesso di quella con input $uxxv$. Infatti i passaggi sul bordo alternano da sinistra a destra e da destra a sinistra, a partire da quello verso destra: se r è pari la testina termina a sinistra del bordo p_1, dove il comportamento

è quello di uxv a sinistra di p_1; se r è dispari termina a destra del bordo p_2, dove il comportamento è quello di uxv a destra di p_2. In entrambi i casi la macchina si ferma nello stesso stato che in uxv, e dunque $uxxv \in X$.

$m \geq 3$: L'argomento del caso $m = 2$ è immediatamente applicabile alle parole con un numero arbitrario di ripetizioni della sottoparola x: ogni copia aggiuntiva di x viene attraversata con gli stessi stati, e la sequenza di passaggio su ciascun bordo tra due copie consecutive e sul bordo che segue u resta S. Lo stato terminale si determina con lo stesso ragionamento sulla parità di r, quindi $ux^m v \in X$.

$m = 0$: Si ottiene uv omettendo il segmento x. Il bordo tra u e v viene attraversato con gli stati $s_1, \ldots, s_r$, che sono tanto quelli ricevuti da u al bordo p_1 quanto quelli ricevuti da v al bordo p_2. Dunque u e v si comportano come in uxv, e per il ragionamento sulla parità di r lo stato terminale è ancora quello di uxv. Quindi $uv \in X$.

Questo prova l'affermazione, e dunque $ux^m v \in X$ per ogni $m \geq 0$. □

Osservazione 8.18. Il Pumping lemma è molto utile per mostrare che un insieme non è decidibile in tempo lineare (basta considerare la proposizione contronominale dell'enunciato). In particolare, visto che ogni insieme DFA-decidibile appartiene alla classe di complessità lineare, il lemma permette di mostrare che un insieme non è DFA-decidibile.

Finalmente possiamo dimostrare che l'iniseme $X = \{0^n 1^n \mid n \in \mathbb{N}\}$ non è DFA-decidibile.

Proposizione 8.19. Per ogni costante positiva c l'insieme $X = \{0^n 1^n \mid n \in \mathbb{N}\}$ non appartiene alla classe di complessità $\mathrm{DTIME}_1^{\{0,1\}}(cn)$.

Dimostrazione. Per il pumping-lemma, se esistesse una fattorizzazione $uxv = 0 \ldots 01 \ldots 1$, si dovrebbe avere che la parola $uxxv \in X$, ma questo non è possibile in quanto abbiamo solo tre possibilità per scegliere x:

- $x = 0 \ldots 0$, questo caso non è possibile poiché $uxxv$ non apparterrebbe a X avendo aumentato il numero di zero ma non quello di 1;
- $x = 1 \ldots 1$ analogo all'argomento precedente applicato al numero di 1;
- $x = 0 \ldots 01 \ldots 1$ in questo caso la parola xx alternerebbe una serie di 0 con una di 1 e questo non è possibile in X.

□

Corollario 8.20. L'inclusione

$$\mathrm{DTIME}_1^A(n) \subset \mathrm{DTIME}_1^A(n^2)$$

è un'inclusione stretta.

Dimostrazione. L'insieme $X = \{0^n 1^n \mid n \in \mathbb{N}\}$ appartiene a $\mathrm{DTIME}_1^A(n^2)$ ma non a $\mathrm{DTIME}_1^A(n)$. Quindi l'inclusione è stretta. □

Da questo risultato possiamo dedurre che la gerarchia di classi di complessità che abbiamo introdotto non è banale, in quanto le due classi non coincidono.

9 Macchine a seminastro e multinastro

9.1 Simulazione di algoritmi

In questa sottosezione, vogliamo fornire degli strumenti per poter mostrare l'equivalenza di diversi modelli di calcolo (ovvero, per mostrare che decidono gli stessi insiemi). Per fare questo, introduciamo la nozione di simulazione. Una simulazione si ha quando è possibile trovare un modo in cui ad ogni computazione effettuata con l'algoritmo per un certo modello di calcolo si possa associare la computazione, non necessariamente attraverso lo stesso numero di passi, svolta dall'altro sistema.

Definizione 9.1. Siano $\mathcal{A}_1 = (Conf_1, \varepsilon_1, \tau_1, \delta_1)$ e $\mathcal{A}_2 = (Conf_2, \varepsilon_2, \tau_2, \delta_2)$ due algoritmi con lo stesso alfabeto di ingresso A e stesso alfabeto di uscita B. Una *simulazione* di $\mathcal{A}_1$ tramite $\mathcal{A}_2$ è una funzione iniettiva

$$\varphi : Conf_1 \to Conf_2$$

tale che per ogni parola $w \in A^*$ si abbia

$$\tau_2^{k_0}(\varepsilon_2(w)) = \varphi(\varepsilon_1(w)) \quad \text{per qualche } k_0,$$

inoltre, per ogni passo della computazione i, si deve avere

$$\tau_2^{k_i}(\varphi(c_i)) = \varphi(\tau_1(c_i)) \quad \text{per qualche } k_i,$$

e per ogni configurazione terminale c_t (ovvero c_t nel dominio di δ_1)

$$\delta_2(\tau_2^{k_t}(\varphi(c_t))) = \delta_1(c_t) \quad \text{per qualche } k_t.$$

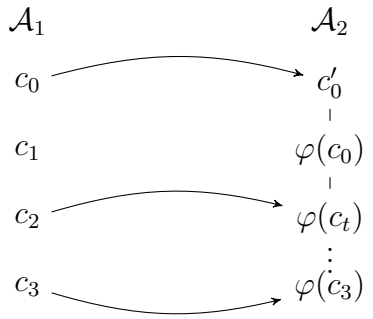


Figura 29: Schema di una simulazione di $\mathcal{A}_1$ tramite $\mathcal{A}_2$.

Osservazione 9.2. Significato di k_i: se $\mathcal{A}_2$ è l'algoritmo che simula $\mathcal{A}_1$ allora è possibile che, mentre in $\mathcal{A}_1$ si effettui una transizione dalla configurazione c_i alla configurazione successiva c_{i+1}, in $\mathcal{A}_2$ siano necessarie k_i transizioni per passare dalla configurazione immagine di c_i all'immagine di c_{i+1}.

Definizione 9.3. Si definisce la *testa di simulazione*, la computazione che $\mathcal{A}_2$ deve fare partendo dalla configurazione iniziale associata ad una parola $\varepsilon_2(w)$ per arrivare alla configurazione immagine della configurazione iniziale dell'algoritmo che viene simulato associata alla stessa parola w: $\varphi(\varepsilon_1(w))$.

In modo analogo si definisce la *coda di simulazione*, come quella computazione che $\mathcal{A}_2$ deve fare partendo dalla immagine della configurazione terminale per l'algoritmo $\mathcal{A}_1$, per arrivare ad una configurazione terminale per l'algoritmo $\mathcal{A}_2$.

Lemma 9.4. Se esiste la simulazione di un algoritmo $\mathcal{A}_1$ per mezzo di un algoritmo $\mathcal{A}_2$ e se $\mathcal{A}_1$ decide l'insieme X allora anche $\mathcal{A}_2$ decide X.

Dimostrazione. Segue dalla definizione. □

9.2 Macchine a seminastro

Superata la limitazione costituita, come nel caso degli automi finiti, dal fatto di avere a disposizione durante la computazione esclusivamente lo spazio occupato dai caratteri della parola da decidere, abbiamo ottenuto una famiglia di algoritmi più potenti che certamente escono dalla classe di complessità lineare $\mathrm{DSPACE}_1^A(n)$. Avendo aggiunto al modello la possibilità di gestire una memoria potenzialmente infinita vediamo che variando leggermente la definizione di macchina alcune proprietà appaiono più evidenti, ad esempio consideriamo ora il caso di una macchina di Turing in cui il nastro è infinito solo in una direzione, ed andiamo a considerare se a questa modifica al modello di calcolo corrisponde un diverso insieme di problemi decidibili.

Una macchina di Turing a seminastro è un modello più debole della macchina di Turing in quanto il nastro è indicizzato solo per interi positivi. Nella cella del nastro di indice 0 poniamo un simbolo particolare ($||$), detto *respingente*, in modo tale che ogni transizione corrispondente alla lettura sul nastro di tale simbolo, prevede lo spostamento della testina alla cella di indice successivo. In questo modo la cella di indice 0 non potrà mai essere superata verso gli indici negativi e la computazione si svolgerà forzatamente sul nastro con indici non negativi.

Definizione 9.5. Lo spazio delle configurazioni a seminastro di alfabeto A e insieme degli stati Q è l'insieme delle triple

$$Conf_{1/2}^{A,Q} = \{(f, p, q) \text{ dove } f : \mathbb{Z} \to A \cup \{\square, ||\},\ p \in \mathbb{Z} \text{ e } q \in Q\}$$

dove

$$f(k) = \begin{cases} \square & \text{se } k < 0 \\ || & \text{se } k = 0 \end{cases}$$

(per $k > 0$ la funzione f è definita perlomeno definta e non si hanno restrizioni). Denotiamo con $Conf_{1/2}^{A,Q}$ l'insieme delle configurazioni per le macchine a seminastro.

Le configurazioni delle macchine a seminastro sono in tutto analoghe a quelle delle macchine di Turing ad un nastro e pertanto possono essere rappresentate graficamente in modo analogo:

Definizione 9.6. Una macchina di Turing a seminastro μ di alfabeto A e insieme degli stati Q è una macchina di Turing di alfabeto $A_s := A \cup \{||\}$ e insieme degli stati Q

$$\mu : Q \times A_s' \to Q \times A_s' \times \{-1, +1\}$$

tale che per ogni stato $q \in Q$ abbiamo

$$\mu(q, ||) = (q', ||, +1) \qquad \text{dove } q' \in Q.$$

L'algoritmo formale associato ad una macchina a seminastro è quello dato per la macchina di Turing, con la sola differenza della funzione di input, che associa ad una parola di input $w \in A^*$ la configurazione $(f, 1, q_0) \in Conf^A_{1/2}$ dove

$$f(p) = \begin{cases} \square & p < 0 \text{ oppure } p > |w|, \\ || & p = 0, \\ w(p) & 1 \leq p \leq |w|. \end{cases}$$

In modo diretto, abbiamo che qualsiasi algoritmo associato ad una macchina a seminastro è un algoritmo associato ad una macchina di Turing standard modificata in modo tale che a partire dalla configurazione iniziale la computazione inizia aggiungendo il carattere speciale $||$ a sinistra della parola di input e poi prosegue la computazione utilizzando la tabella della macchina a seminastro. In questo caso, la funzione di simulazione è l'identità.

Proposizione 9.7. Un insieme decidibile per una macchina di Tuing a seminastro è decidibile anche per una macchina di Turing.

Dimostrazione. Segue dal lemma 9.4. □

Anche l'implicazione dall'altro lato è vera. Infatti abbiamo la seguente proposizione.

Proposizione 9.8. Data una macchina di Turing μ esiste una macchina di Turing a seminastro μ' e una funzione di simulazione $\varphi : Conf^A_1 \to Conf^A_{1/2}$ in modo che l'algoritmo associato a μ' simuli l'algoritmo associato a μ.

Dimostrazione. Poiché

$$\mu : Q \times A' \to Q' \times A' \times \{+1, -1\}$$

allora le sue configurazioni sono del tipo (f, p, q) con $p \in \mathbb{Z}$ e $f : \mathbb{Z} \to A'$. Le configurazioni di un algoritmo associato ad una macchina a seminastro sono del tipo $(f_{1/2}, p, q)$ con $f : \mathbb{Z} \to A'_s$ e $f_{1/2}(p) = \square$ se $p < 0$.

Dobbiamo, per prima cosa, costruire una biiezione $\psi : \mathbb{Z} \to \mathbb{Z}^+$ da utilizzare nella definizione della funzione di simulazione:

$$\psi(p) = \begin{cases} 2p + 1 & \text{se } p \geq 0, \\ -2p & \text{se } p < 0. \end{cases}$$

Consideriamo come insieme degli stati per la macchina μ' a semi-nastro

$$Q_{\mu'} = Q \times \{-1, +1\} \cup Q \times \{-1, +1\}^2 \cup Q_0$$

gli stati $(q, \epsilon) \in Q \times \{-1, +1\}$ servono ad interpretare le configurazioni che si trovano nello stato q nella macchina μ quando la posizione corrente p è nel seminastro di segno ϵ, gli stati $(q, \epsilon, m) \in Q \times \{-1, +1\}^2$ servono ad interpretare il passo di simulazione intermedio in cui

si trova la macchina μ' quando deve effettuare una transizione allo stato q' trovandosi nel seminastro di segno ϵ e dovendo μ effettuare uno spostamento della direzione m. Gli stati della forma (q, ϵ, m) sono gli stati in cui la macchina si trova quando sta attraversando una casella per raggiungere la casella successiva (quella corrispondente a $\psi(p+m)$). Poniamo $\varphi(f,p,q) = (f', p', q')$ dove $f'(k) = f(\psi^{-1}(k))$, $p' = \psi(p)$ e

$$q' = \begin{cases} (q,+1) & \text{se } p \geq 0, \\ (q,-1) & \text{se } p < 0 \end{cases}.$$

Quindi se $\mu(q,a) = (q', a', m)$ (con $m \in \{-1,+1\}$ possibile spostamento) allora, per $q \in Q_{\mu'}$ si hanno i seguenti casi

$$\mu'(q,a) = \begin{cases} ((q',\epsilon,m), a', \epsilon m) & \text{se } q = (q_\mu, \epsilon),\ a \neq || \text{ e } \mu(q_\mu, a) = (q'a', m) \\ ((q_\mu,\epsilon), a, \epsilon m) & \text{se } q = (q_\mu, \epsilon, m),\ a \neq || \\ ((q_\mu, -\epsilon, m), ||, +1) & \text{se } q = (q_\mu, \epsilon, m),\ a = || \\ ((q_\mu, -\epsilon), ||, +1) & \text{se } q = (q_\mu, \epsilon),\ a = || \end{cases}$$

la componente ϵ serve per "cambiare segno" allo spostamento, ad esempio quando la testina di μ deve passare dalla cella 0 alla cella -1 allora $\epsilon = -1$ mentre in μ' lo spostamento deve essere positivo.

Resta da costruire quella parte della macchina di Turing che a partire dalla configurazione per la macchina a seminastro associata alla parola in ingresso w computa l'immagine attraverso la funzione di simulazione della configurazione iniziale per la macchina ad un nastro (testa di simulazione). Tale macchina definita sull'insieme di stati Q_0 (con 12 elementi) è riportata in Figura 30. □

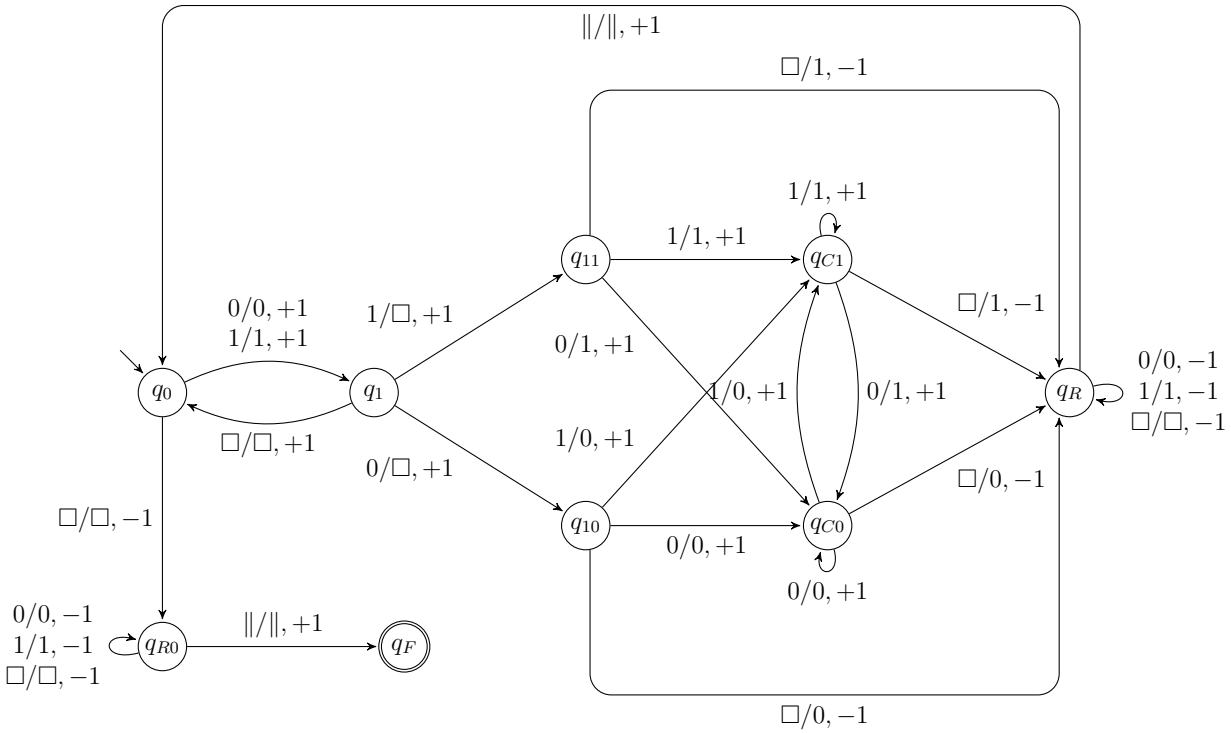


Figura 30: Macchina di Turing a seminastro che trasforma la configurazione iniziale nell'immagine della configurazione iniziale della macchina a nastro singolo. A ogni passata inserisce un blank tra due simboli consecutivi: q_0 e q_1 scorrono le coppie già separate, q_{10}, q_{11}, q_{C0} e q_{C1} spostano di una cella a destra il resto della parola trasportando un simbolo, e q_R riporta la testina al respingente. Quando q_0 trova un blank al posto di un simbolo, q_{R0} riporta la testina sulla cella 1 e la macchina termina in q_F.

Corollario 9.9. Se X a decidibile per macchina di Turing allora è decidibile anche per macchina di Turing a seminastro.

Dimostrazione. La tesi è conseguenza della proposizione 9.8 e del lemma 9.4. □

Notiamo che sul piano qualitativo le macchine di Turing ad un nastro e quelle a seminastro sono equivalenti, ma c'è un aspetto quantitativo che le distingue:

Lemma 9.10. Con le notazioni della Proposizione 9.8, per ogni $w \in A^*$ con $n = |w| \geq 1$, se poniamo t (risp. s) pari al costo in tempo (risp. in spazio) $T_\mu(w)$ (risp. $S_\mu(w)$), allora

$$T_{\mu'}(w) \leq 3t + 7n^2 \qquad (\text{risp. } S_{\mu'}(w) \leq 2s + 2n + 1).$$

Dimostrazione.

Tempo: Ogni passo della computazione di μ è simulato da 2 oppure 3 passi di calcolo di μ' (tre quando la testina attraversa il respingente e bisogna cambiare il segno dello spostamento), quindi la simulazione costa al più $3t$ passi.

Quanto all'inizializzazione, la trasformazione di una parola di lunghezza n in una di lunghezza $2n$, inserendo dei blank tra un simbolo e il successivo, necessita di $2n$ passi

per inserire il primo blank, $2(n-1)$ per il secondo, $2(n-2)$ per il terzo e così via, ovvero $\sum_{i=1}^{n} 2i = n^2 + n$ passi. Servono poi $2n$ passi per riportare la testina all'inizio della parola e ancora 2 per vedere la fine della parola spaziata (bisogna incontrare un doppio blank). In totale si hanno $n^2 + 3n + 2 \leq 7n^2$ passi, dove l'ultima disuguaglianza vale perché $n \geq 1$.

Spazio: Le celle visitate da μ hanno indici in un intervallo $[-a, b]$ con $a + b + 1 \leq s$, e ψ le manda nelle celle di indice al più $2\max(a, b) + 1 \leq 2s + 1$. Alla simulazione si aggiunge lo spazio usato in fase di inizializzazione, in cui la parola occupa $2n$ celle, per un totale di al più $2s + 2n + 1$.

□

Corollario 9.11. Per ogni S e T funzioni di complessità si ha:

$$\mathrm{DTIME}^A_{1/2}(T(n)) \subset \mathrm{DTIME}^A_1(T(n)) \subset \mathrm{DTIME}^A_{1/2}(3T(n) + 7n^2)$$

$$\mathrm{DSPACE}^A_{1/2}(S(n)) \subset \mathrm{DSPACE}^A_1(S(n)) \subset \mathrm{DTIME}^A_{1/2}(2S(n) + 2n + 1)$$

Dimostrazione. La tesi è conseguenza di quanto detto in questo paragrafo. □

Si noti come la costante 7 nella formula più sopra non abbia molto significato: infatti, per la definizione asintotica delle classi di complessità, questa costante può essere sostituita con una qualsiasi altra costante di valore maggiore strettamente di 1 poiché è il termine quadratico proveniente dalla maggiorazione di $n^2 + 3n + 3$.

9.3 Macchine multinastro

La definizione di macchina di Turing può essere estesa a una più versatile: in luogo di una funzione $f\colon \mathbb{Z} \to A'$ si può definire una funzione che a ogni coordinata associa una k-upla di simboli dell'alfabeto,

$$f\colon \mathbb{Z} \to A'^k,$$

che funziona come un array di nastri e che quindi va in direzione opposta rispetto alla restrizione applicata nelle macchine a seminastro: al modello generale con un nastro indicizzato dai numeri interi si sostituisce un seminastro indicizzato con i numeri naturali.

Introduciamo allora una generalizzazione della macchina di Turing che utilizzi più nastri. L'idea fisica è chiara: si ha una configurazione che utilizza k nastri (essendo k un numero fissato) e su ogni nastro soltanto una cella è accessibile. Ogni transizione avviene tenendo conto dello stato corrente, anche qui preso in un insieme finito, e dei k caratteri accessibili sull'array di nastri. La risposta consiste nei k caratteri da scrivere alla posizione corrente su ciascun nastro, negli spostamenti da effettuare su ciascun nastro (considereremo il caso più generale, in cui gli spostamenti non debbano essere paralleli) e infine nel nuovo stato in cui si troverà la macchina. Con questa definizione la formalizzazione è immediata:

Definizione 9.12. Una *configurazione a k nastri* di alfabeto A e insieme degli stati Q è una $(2k+1)$-pla

$$(f_1, \ldots, f_k, p_1, \ldots, p_k, q),$$

dove

1. ognuno dei nastri è una funzione $f_i\colon \mathbb{Z} \to A'$, dove $A' := A \cup \{\square\}$, per $i = 1, \dots, k$;
2. $p_i \in \mathbb{Z}$ per ogni $1 \leq i \leq k$;
3. $q \in Q'$, dove $Q' := Q \cup \{q_F^0, q_F^1\}$, essendo q_F^0 e q_F^1 due stati che non appartengono a Q.

Una macchina di Turing per k nastri potrebbe essere rappresentata come $2k+1$ tabelle a $k+1$ argomenti. È chiaro che una tale tabella è abbastanza illeggibile non appena il valore di k è maggiore di 1. Infatti le macchine di Turing multinastro, ancor più di quelle a un nastro, sono strumenti teorici non concepiti per essere utilizzati in pratica.

L'insieme delle configurazioni sarà denotato con

$$\mathrm{Conf}_k^{A,Q} = \{((f_1, \dots, f_k), (p_1, \dots, p_k), q) \text{ che verificano i punti 1)–3)}\}.$$

Definizione 9.13. Dato un alfabeto A e un insieme finito di stati Q, una *macchina di Turing a k nastri* di alfabeto A e insieme degli stati Q è un'applicazione

$$\mu\colon Q \times A'^k \to Q' \times A'^k \times \{-1, 0, +1\}^k,$$

dove $A' := A \cup \{\square\}$ e $Q' := Q \cup \{q_F^0, q_F^1\}$.

Osservazione 9.14. Nella definizione 9.13 si dà la possibilità a una delle varie testine di non muoversi. Si può mostrare che, obbligando le macchine a multinastro a muovere la testina di ogni nastro ad ogni passo della computazione, la classe degli insiemi decisi non cambierebbe (e la classe di complessità rimarrebbe la stessa a meno di un fattore costante). Scegliamo quindi per semplicità la definizione precedente.

In modo simile alle versioni precedenti bisogna associare a una macchina μ un algoritmo formale in grado di eseguire la computazione descritta dalla macchina. Per prima cosa, lo spazio delle configurazioni per una macchina a k nastri di alfabeto A e spazio degli stati Q è l'insieme $\mathrm{Conf}_k^{A,Q}$ appena definito. L'algoritmo formale $\mathcal{A}_\mu$ è dato dalla tripla $(\omega, \varepsilon, \delta)$, le cui tre funzioni sono le seguenti.

1. La *funzione di input* $\omega\colon A^* \to \mathrm{Conf}_k^{A,Q}$ associa a una parola w di alfabeto A la configurazione
$$\omega(w) = ((f_1, \dots, f_k), (1, \dots, 1), q_0),$$
dove la prima funzione nastro è definita come al solito come il nastro
$$f_j(p) = \begin{cases} w(p) & \text{se } 1 \leq p \leq |w| \text{ e } j = 1, \\ \square & \text{altrimenti} \end{cases}$$
(quindi tutte le componenti $j \neq 1$ hanno il nastro riempito con il simbolo blank: $f_j(p) = \square$ per ogni posizione p).
2. La *funzione di transizione*
$$\varepsilon\colon \mathrm{Conf}_k^{A,Q} \to \mathrm{Conf}_k^{A,Q}$$
è definita come
$$\varepsilon\big(((f_1, \dots, f_k), (p_1, \dots, p_k), q)\big) = ((f_1', \dots, f_k'), (p_1', \dots, p_k'), q')$$

quando si ha

$$\mu\big(q, (f_1(p_1), \ldots, f_k(p_k))\big) = (q', (a_1, \ldots, a_k), (m_1, \ldots, m_k))$$

e, per ogni $1 \leq i \leq k$,

- $f_i'(p) = \begin{cases} f_i(p) & \text{se } p \neq p_i, \\ a_i & \text{se } p = p_i, \end{cases}$
- $p_i' = p_i + m_i$.

3. La *funzione di uscita*

$$\delta \colon \mathrm{Conf}_k^{A,Q} \to \{0, 1\}$$

associa a una configurazione terminale data un output binario in accordo con lo stato finale:

$$\delta((f_1, \ldots, f_k), (p_1, \ldots, p_k), q) = \begin{cases} 0 & \text{se } q = q_F^0, \\ 1 & \text{se } q = q_F^1. \end{cases}$$

Esempio 9.15. Analizziamo se vi sono cambiamenti, con il passaggio alle macchine multinastro, nella decidibilità del problema $X = \{0^p 1^p \mid p \geq 0\}$. A partire da una parola $w \in \{0, 1\}^*$, il calcolo della macchina in Figura 31 consiste nel leggere da sinistra a destra i simboli di w. Supponiamo che w si scriva come $0^i 1^j w'$ con w' che non inizia per 1, la qual cosa si può sempre fare in una e una sola maniera. A partire dallo stato iniziale q_0, la macchina scriverà 0 sul secondo nastro per i volte, restando nello stato iniziale q_0. Quindi, al primo carattere 1 incontrato sul primo nastro, lo stato diventa q_1 e rimane tale fino a quando vengono letti degli 1 sul primo nastro. Allo stesso tempo, a ogni transizione viene cancellato uno 0 sul secondo nastro, ripercorrendo la sequenza di 0 che è stata scritta mentre veniva letta la sequenza di zero sul primo nastro. Se a un certo punto sul primo nastro c'è un carattere e sul secondo si trova un blank, cioè se abbiamo $j > i$ o $w' \neq \varepsilon$, la parola w viene rifiutata. Allo stesso modo, se c'è un blank sul primo nastro mentre rimane almeno uno 0 sul secondo, ovvero si ha $i < j$, la parola viene rifiutata. La parola è accettata solo se la lettura del suo ultimo simbolo coincide con la cancellazione dell'ultimo 0 sul secondo nastro, ovvero solo se $i = j$ e w' è la parola vuota.

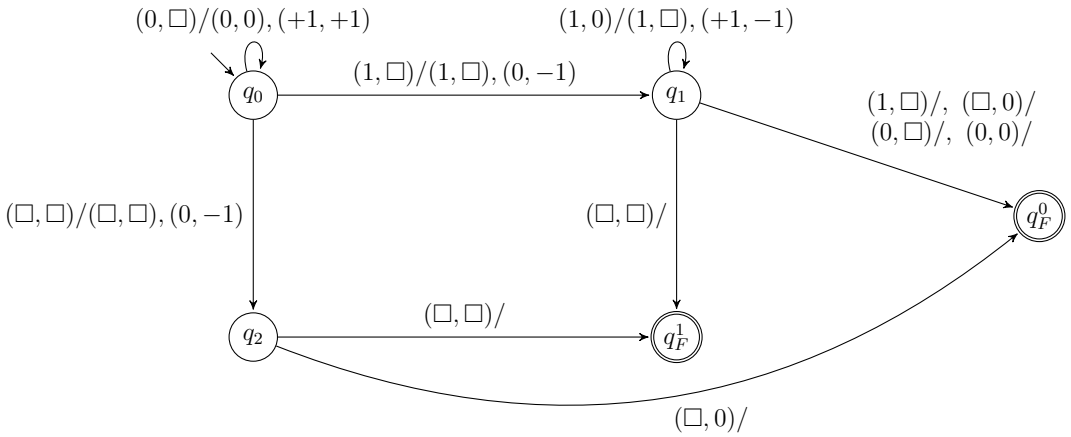


Figura 31: Macchina di Turing a 2 nastri che decide l'insieme $X = \{0^p 1^p \mid p \geq 0\}$ in tempo lineare.

L'algoritmo dell'esempio precedente decide quindi l'insieme $X = \{0^p 1^p \mid p \geq 0\}$. Introducendo la notazione per le classi di complessità per macchine di Turing multinastro, analoga a quella introdotta per le macchine mononastro, risulta chiaro dalla discussione precedente che la decisione per una parola binaria di lunghezza n viene presa entro $n+1$ passi di computazione, e dunque si può enunciare che:

Lemma 9.16. Dato l'insieme $X = \{0^p 1^p \mid p \geq 0\}$, allora $X \in \mathrm{DTIME}_2^{\{0,1\}}(n)$.

9.4 Equivalenza tra macchine a multinastro e macchine a nastro singolo

Vorremmo ora mostrare che il modello a multinastro decide gli stessi insiemi del modello a nastro singolo. Per farlo, forniremo una funzione di simulazione ϕ e una nuova macchina a nastro singolo μ_1 partendo da una a più nastri. L'insieme degli stati per la macchina μ_1 sarà un isnieme della forma $Q_1 := Q \sqcup (R \times Q)$, dove l'insieme $\overline{Q}$ sarà descritto più avanti. Cominciamo con la funzione di simulazione, questa sarà della forma

$$\varphi\colon Conf_k^{A,Q} \longrightarrow Conf_1^{A,Q_1}, \qquad \big((f_1,\ldots,f_k),(p_1,\ldots,p_k),q\big) \longmapsto (f,p,q),$$

Visto che sullo stato q, l'immagine rimane q stesso, dobbiamo solo specificare che forma hanno f (la configurazione del nastro singolo) e p (la posizione del puntatore sul nastro singolo).

La configurazione f del nastro singolo è la concatenazione delle colonne (numerate a partire da 1, così come le celle), in cui ogni lettera è seguita da un simbolo $*$ oppure $\square$: la colonna j occupa le celle da $2k(j-1)+1$ a $2kj$ e ha la forma

$$f_1(j)\, m_1\ f_2(j)\, m_2\ \cdots\ f_k(j)\, m_k,$$

dove $m_i = *$ se la testina del nastro i si trova sulla cella j (cioè $p_i = j$) e $m_i = \square$ altrimenti. In modo formale, per ogni $j \geq 1$ e $i \in \{1,\ldots,k\}$,

$$f\big(2k(j-1)+2i-1\big) = f_i(j), \qquad f\big(2k(j-1)+2i\big) = \begin{cases} * & \text{se } p_i = j, \\ \square & \text{altrimenti.} \end{cases}$$

Osservazione 9.17. Gli asterischi occupano sempre posizioni pari: per questo si può usare un simbolo fissato di A senza bisogno di ingrandire l'alfabeto, visto che nelle posizioni pari ci saranno solamente $\square$ oppure il simbolo fissato. Lo stato tiene un contatore modulo $2k$ della posizione nella colonna, e così distingue le celle delle lettere da quelle dei marcatori e riconosce a quale nastro i appartiene un asterisco.

Se le testine dei nastri si trovano in posizioni diverse, come in figura,

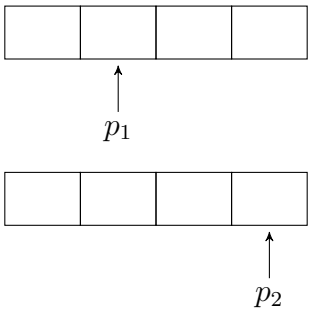


Figura 32: Due nastri con le testine in posizioni diverse.

la testina del nastro singolo va posta nella prima cella della colonna più piccola tra quelle in cui si trova una testina. Posto $p_{\min} := \min\{p_1, \dots, p_k\}$, si ha dunque

$$p = 2k\,(p_{\min} - 1) + 1.$$

Ora costruiamo la macchina μ_1 a partire dalla funzione di transizione della macchina a k nastri μ

$$\mu\colon Q \times A^k \longrightarrow Q \times A^k \times \{-1, 0, 1\}^k.$$

Gli stati di μ_1 saranno

$$Q_1 := Q \sqcup (R \times Q), \qquad R = \{R_i\}_{i \in I}, \quad |I| < \infty,$$

dove gli R_i sono stati ausiliari che permettono a μ_1 di eseguire un passo di μ: in uno stato (R_i, q) la macchina μ_1 ricorda lo stato q di μ da cui è partita la simulazione del passo. Un passo di μ viene simulato da μ_1 con un giro di andata e uno di ritorno.

1. *Andata.* Si parte con la testina in $p = 2k\,(p_{\min} - 1) + 1$, ricordando nello stato il valore $q \in Q$ di μ. Si scorre il nastro verso destra tenendo nello stato la posizione nella colonna (modulo $2k$) e l'ultima lettera letta. Quando si trova un $*$ in una cella x, appartenente al nastro i, la lettera marcata è quella della cella $x - 1$, già memorizzata, e si pone a_i uguale a essa. Raggiunto il k-esimo asterisco la testina si trova su di esso, e nello stato è memorizzata la k-upla $(a_1, \dots, a_k)$.

2. *Calcolo.* Si calcola

   $$\mu(q, a_1, \dots, a_k) = (q',\, b_1, \dots, b_k,\, s_1, \dots, s_k) \in Q \times A^k \times \{-1, 0, 1\}^k,$$

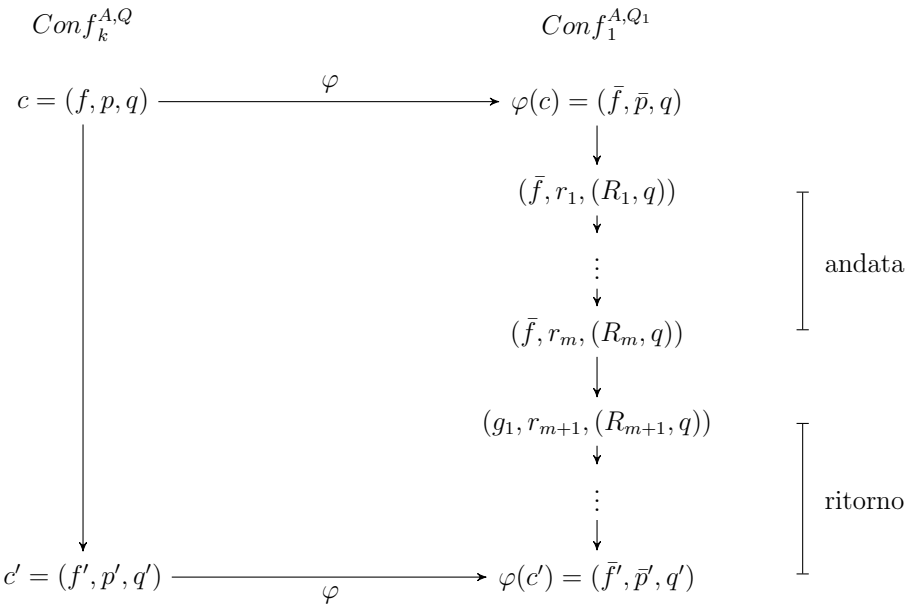
   che è un'informazione finita e quindi può essere ricordata dallo stato.

3. *Ritorno.* Si scorre il nastro verso sinistra. Quando si legge un $*$ nella cella x, appartenente al nastro i, la lettera marcata è in $x-1$ e si procede così:

   - se $s_i = 0$ si va in $x-1$, si scrive b_i e si prosegue;
   - se $s_i = -1$ si cancella il $*$ (scrivendo $\square$), si va in $x-1$, si scrive b_i e si prosegue, ricordando nello stato che bisogna scrivere $*$ nella cella dello slot i della colonna precedente (cioè in $x-2k$), dove si scriverà appena la scansione vi arriva;
   - se $s_i = +1$ si cancella il $*$, si va in $x+2k$ e vi si scrive $*$, si torna in $x-1$, si scrive b_i e si prosegue.

4. *Riposizionamento.* Dopo aver processato tutti i k asterischi si prosegue verso sinistra fino all'inizio della colonna precedente a quella del vecchio asterisco più a sinistra (scrivendo gli eventuali asterischi in sospeso), poi si scorre verso destra fino al primo $*$ e si torna all'inizio della sua colonna: la testina è sul nuovo $p = 2k\,(p_{\min} - 1) + 1$. Si entra nello stato $q' \in Q$.

In termini di configurazioni, se $c = (f, p, q)$ e $c' = (f', p', q')$ sono configurazioni consecutive di μ, allora μ_1 passa da $\varphi(c)$ a $\varphi(c')$ attraversando un numero finito di configurazioni intermedie, nelle quali lo stato è della forma (R_h, q). Qui R_h codifica l'informazione finita che μ_1 deve ricordare durante il passo: la fase (andata, ritorno o riposizionamento), la posizione nella colonna modulo $2k$, le lettere lette finora e, dopo il calcolo, la $(2k+1)$-upla $(q', b_1, \ldots, b_k, s_1, \ldots, s_k)$ ottenuta da μ, insieme agli asterischi ancora da scrivere. Durante l'andata il nastro resta invariato e si muove solo la testina; durante il ritorno vengono modificati il nastro e la posizione degli asterischi.



Per stimare il costo osserviamo che, su un input di lunghezza n (con $T(n) \geq n$), dopo $t \leq T(n)$ passi di μ le testine e gli asterischi si trovano nelle colonne $1, \ldots, T(n)+1$, quindi tutto ciò che μ_1 deve attraversare è contenuto nelle prime $2k\,(T(n)+1)$ celle del nastro.

Il numero di passi di μ_1 non è però limitato dal numero di celle occupate, ma dal modo in cui la testina si muove. Nella fase di andata la testina si sposta sempre verso destra, di una cella per passo, fino all'ultimo asterisco, quindi compie al più $2k\,(T(n)+1)$ passi. Nella fase di ritorno la testina si sposta verso sinistra, con la sola eccezione di una deviazione per ciascun asterisco con $s_i = +1$ (andare a scrivere il nuovo asterisco in $x + 2k$ e tornare in $x - 1$), che richiede $4k$ passi in più rispetto alla scansione. Il ritorno costa perciò al più $2k\,(T(n)+1)+4k^2$ passi ($4k^2$ perchè si fanno $4k$ operazioni per k asterischi). Il riposizionamento finale sul nuovo $p_{\min}$, che sta nelle colonne $j_{\min} - 1$, $j_{\min}$ o $j_{\min} + 1$, richiede al più $12k$ passi: al più $4k$ per arrivare all'inizio della colonna $j_{\min} - 1$, al più $6k$ per scorrere a destra fino al primo $*$ e al più $2k$ per tornare all'inizio della sua colonna. Un passo di μ viene dunque simulato in al più

$$2 \cdot 2k\,(T(n) + 1) + 4k^2 + 12k$$

passi di μ_1. Poiché μ esegue al più $T(n)$ passi, la macchina μ_1, una volta inizializzata, compie

$$(2 \cdot 2k\,(T(n) + 1) + 4k^2 + 12k)T(n)$$

passi. Resta da stimare il costo della codifica iniziale φ dell'input $w = w_1 \cdots w_n$. La configurazione iniziale di μ ha la parola w scritta su tutti i nastri, con le testine in cella 1, cioè $f_i(j) = w_j$ per ogni i e $p_i = 1$. La colonna j della configurazione codificata è dunque

$$w_j\, m\; w_j\, m\; \cdots\; w_j\, m \qquad (k \text{ coppie}),$$

con $m = *$ se $j = 1$ e $m = \square$ se $j \geq 2$. Partendo dalla parola w scritta all'inizio del nastro di μ_1, si procede con n passate, una per lettera. Posto $d := 2k - 1$, nella passata j la testina si trova su w_j, in cella $2k(j-1)+1$:

- nelle d celle successive si scrive $m\, w_j\, m\, w_j \cdots m$, memorizzando nello stato, che avrà la struttura di una coda di d simboli, le lettere cancellate, si compiono così d passi;
- si prosegue verso destra: in ogni cella si legge il simbolo, si scrive il simbolo più vecchio della coda e si inserisce in coda il simbolo letto, finché nella coda restano solo blank. In questo modo le $n - j$ lettere rimanenti scorrono di d celle verso destra, e la testina percorre $(n - j)$ celle, che coincide con il numero di operazioni per questa fase;
- si torna a sinistra fino al primo $\square$ o $*$ (l'ultimo marcatore inserito, in cella $2kj$) e si fa un passo a destra, posizionandosi su w_{j+1}: al più $(n - j) + 1$ passi.

La passata j richiede quindi al più $d + (n - j) + (n - j) + 1 = 2(n - j) + 2k$ passi, e le n passate richiedono quindi

$$\sum_{j=1}^{n} \big(2(n - j) + 2k\big) = n^2 + (2k - 1)\, n.$$

Infine la testina deve tornare in cella 1, per al più $2kn$ passi. Il costo della codifica è dunque al più $n^2 + (4k - 1)\, n \leq 4k\, n^2$ per $n \geq 1$. Otteniamo quindi che

$$\begin{aligned} \mathrm{DTIME}_k^A\big(T(n)\big) &\subseteq \mathrm{DTIME}_1^A\Big(\big(2k\,(2T(n) + 2) + 4k(k + 3)\big)\, T(n) + 4k\, n^2\Big) \\ &\subseteq \mathrm{DTIME}_1^A\big(4k(k + 6)\, T^2(n)\big). \end{aligned} \tag{3}$$

visto che vale le seguenti relazioni

$$\begin{aligned}\big(2k\,(2T(n)+2)+4k(k+3)\big)\,T(n)+4k\,n^2 &= 4k\,T^2(n)+4k(k+4)\,T(n)+4k\,n^2\\ &\le 4k\,T^2(n)+4k(k+4)\,T^2(n)+4k\,T^2(n)\\ &= 4k(k+6)\,T^2(n),\end{aligned}$$

(abbiamo usato $n \le T(n)$ dalla definizione di funzione di complessità, e $T(n) \le T^2(n)$ visto che $T(n)$, sempre dalla definziione di funzione di complesità, deve essere non decrescente).

9.5 Cambiamenti di alfabeto

In questa sezione, vogliamo studiare le cosneguenze relativo al cambiamento dell'alfabeto: la classe di insiemi decisi rimane invariata? come cambiano le classi di complessità?

Definizione 9.18. Definiamo $\mathrm{DTIME}^{A,Q}(T(n)) := \cup_{k=1}^{\infty}\mathrm{DTIME}_k^{A,Q}(T(n))$.

Se aumentiamo l'alfabeto di riferiemnto, chiaramente non abbiamo grosse modifche:

Lemma 9.19. Supponiamo che $A \subseteq B$ e che $T(n)$ sia una funzione di complessità. Allora la classe

$$\mathrm{DTIME}^{A}(T(n)) \subseteq \mathrm{DTIME}^{B}(T(n)).$$

Dimostrazione. Basta prendere una macchina di Turing μ che decide $X \in \mathrm{DTIME}^{B}(T(n))$, con $X \subseteq A^*$, e proiettarla sull'alfabeto A, nel senso che si definisce una macchina μ' che coincide con μ sulle coppie (q, c) solo se il carattere letto c è in A:

$$\mu'(q,c) = \begin{cases}\mu(q,c) & \text{se } c \in A,\\ (q_F^0,\, c,\, +1) & \text{se } c \in B \text{ e } c \notin A.\end{cases}$$

□

D'altra parte ci si può chiedere se, passando da un alfabeto più ampio B ad uno con meno simboli A, si perda in qualche modo capacità computazionale, o se come è ovvio si perde solo in efficienza.
Una prima osservazione è che il problema $Y \subseteq B^*$ deve essere codificato in un problema $X \subseteq A^*$. La codifica ω si ottiene come omomorfismo iniettivo di monoidi (essendo l'insieme delle parole su un alfabeto un monoide libero) e quindi se per ogni $b \in B$, $\omega(b) = w_b \in A^*$ estendiamo ω alle parole tramite le proprietà:

$$\omega(\varepsilon_B) = \varepsilon_A \qquad \text{e} \qquad \omega(w_1 w_2) = \omega(w_1)\,\omega(w_2).$$

Associamo ad una codifica di alfabeto ω un intero $d(\omega)$ che fornisce la lunghezza massima della parola associata ad un elemento di B:

$$d(\omega) := \max_{b \in B} |\omega(b)|.$$

Esempio 9.20. Ad esempio $B = \{1, 2, 3\}$ ed $A = \{0, 1\}$ allora definiamo $\omega\colon B \to A^*$ come $\omega(1) = 00$, $\omega(2) = 01$ e $\omega(3) = 10$ così tramite l'estensione $\omega\colon B^* \to A^*$ si ottiene $\omega(11213) = 0000010010$.

Con *codifica* indicheremo un omomorfismo iniettivo tra monoidi liberi. Potremo considerare sia codifiche della forma

$$\omega\colon B^* \to A^*,$$

sia codifiche della forma

$$\omega\colon B'^* \to A'^*,$$

dove $B' = B \sqcup \{\square\}$ e $A' = A \sqcup \{\square\}$. Quest'ultimo caso serve, ad esempio, quando vogliamo scrivere un blank su un nastro usando più celle (come faremo nella dimostrazione del lemma 9.23): in tal caso $\omega(\square)$ sarà una parola di soli blank (cfr. la sezione 11.6.1 per approfondimenti sulla costruzione di codifiche a partire da funzioni sui generatori).

Lemma 9.21. Data una codifica $\omega\colon B^* \to A^*$, se $d(\omega) = |\omega(b)|$ per ogni $b \in B$ e se $Y \subseteq B^*$ è tale che

$$Y \in \mathrm{DTIME}_k^B(T(n))$$

per un'opportuna funzione di complessità T, allora

$$\omega(Y) \in \mathrm{DTIME}_k^A\big(3d\,T(\lceil n/d \rceil)\big).$$

Dimostrazione. Fissiamo $d = d(\omega)$. Assumendo il numero dei nastri $k = 1$, l'adattamento a qualsiasi numero è immediato. Supponiamo che μ sia una macchina di Turing di alfabeto B che decide Y. Si tratta di costruire a partire da μ una macchina μ' di alfabeto A che ancora decide l'insieme $\omega(Y)$.
L'idea è che ogni carattere di B sia codificato da d caratteri di A, quindi la nuova macchina, per poter simulare μ, deve elaborare i caratteri in blocchi di d. Questo è lo stesso schema per la simulazione di d nastri su uno solo, ma qui nel caso semplificato dal fatto che le testine sui nastri fanno sempre gli stessi movimenti. Per effettuare una transizione, la nuova macchina deve prima tenere conto dei d caratteri che costituiscono un blocco. Questa volta non occorre cercarli lungo tutto il nastro, ma solo leggerli nelle caselle immediatamente vicine. Formalmente, possiamo introdurre la codifica ψ delle configurazioni (a un nastro) di alfabeto B e insieme degli stati Q (insieme di stati di μ) nelle configurazioni di alfabeto A definita come:

$$\psi(f, p, q) := (f', dp, q) \quad \text{dove} \quad f'(dk)\, f'(dk+1) \cdots f'(dk+d-1) = \omega(f(k)),\ \forall k \in \mathbb{Z}$$

quindi in modo tale che le d posizioni a partire da dk per qualche k contengano la parola $\omega(f(k))$ di lunghezza d che codifica il carattere $f(k)$.
Le istruzioni di μ' sono quindi definite per garantire la simulazione delle transizioni effettuate da μ. Si inizia leggendo d caratteri verso destra (i caratteri letti vengono via via memorizzati nello stato), si procede poi a sostituire i d caratteri letti con i d caratteri che codificano il nuovo carattere di alfabeto B scritto dalla macchina μ (quindi d movimenti a sinistra), e infine bisogna posizionare il puntatore per la transizione seguente effettuando una transizione di d posizioni a sinistra o a destra. È quindi chiaro che ogni transizione effettuata da μ sarà correttamente simulata da $3d$ transizioni della nuova macchina. Poiché non è stata richiesta la suriettività di ω, la tabella μ' è costruita in modo che, se un blocco di caratteri non è l'immagine di un carattere di B', la macchina passa allo stato di rifiuto. Per quanto riguarda la fase di inizializzazione, bisgona gestire Notiamo inoltre che non c'è niente da fare nella fase

di inizializzazione, poiché si tratta di decidere $\omega(Y)$ e non Y, dunque la parola in ingresso sarà direttamente scritta come sequenza di blocchi di d caratteri di alfabeto A. Si noti che il calcolo a partire da una parola di lunghezza n simula il calcolo di μ a partire da una parola di lunghezza $\lceil n/d \rceil$, che termina in al massimo $T(\lceil n/d \rceil)$ passi, e quindi con una simulazione di al più $3dT(\lceil n/d \rceil)$ passi la macchina μ' deciderà in modo simile a quanto farebbe la macchina μ. □

Osservazione 9.22. Il lemma 9.21 vale anche per codifiche $\omega\colon B'^* \to A'^*$ con $|\omega(b)| = d$ per ogni $b \in B'$ e $\omega(\square) = \square^d$, purché $\omega(Y) \subseteq A^*$. Infatti, nella dimostrazione la codifica agisce sulle celle dei nastri, che contengono anche il blank, quindi si usa comunque l'estensione $\omega(\square) = \square^d$ (compatibile con l'iniettività, perché $\square^d$ non è immagine di nessuna lettera di B). Di ω servono solo tre fatti: ogni lettera è codificata da un blocco di lunghezza d; una cella vuota di μ corrisponde a d celle vuote di μ', così che ψ sia ben definita su $\mathbb{Z}$ e mandi la configurazione iniziale su w in quella su $\omega(w)$; l'input $\omega(w)$ di μ' è una parola di A^*. I blank interni ai blocchi sono solo contenuto delle celle durante la simulazione e non compaiono nell'input. La macchina μ' può leggerli e scriverli, perché il blank è sempre un simbolo di nastro, e rifiuta i blocchi che non sono immagine di una lettera. La dimostrazione resta quindi invariata.

Ci servirà un ulteriore lemma, il quale fornisce una stima in termini di complessità temporale del costo che bisogna pagare per simulare la computazione di una macchina su un alfabeto B utilizzando una macchina definita su un alfabeto $A \subseteq B$.

Lemma 9.23. Sia $A \subseteq B$ con

$$|B| + 1 \leq (|A| + 1)^d. \tag{4}$$

Allora, per ogni funzione di complessità $T\colon \mathbb{N} \to \mathbb{N}$,

$$2^{A^*} \cap \mathrm{DTIME}_k^B(T(n)) \subseteq \mathrm{DTIME}_k^A\big(3dT(n) + 2n^2\big) \qquad \text{se } k \geq 1,$$

e, se $k \geq 2$ e $T(n) \geq 3n$ per ogni n,

$$2^{A^*} \cap \mathrm{DTIME}_k^B(T(n)) \subseteq \mathrm{DTIME}_k^A\big(4dT(n)\big).$$

Dimostrazione. Sia $\square$ il carattere blank, che non appartiene né ad A né a B, e poniamo $A' = A \cup \{\square\}$ e $B' = B \cup \{\square\}$. Indichiamo con

$$P = \{u \in A'^* : |u| = d\}$$

l'insieme delle parole di lunghezza d su A'; esso ha $(|A| + 1)^d$ elementi. Per ogni $c \in A'$ sia $\hat{c} \in P$ la parola formata da d copie di c. Le parole $\hat{c}$, al variare di $c \in A'$, sono a due a due distinte. La condizione (4) dà

$$|B \setminus A| \;=\; |B| - |A| \;\leq\; (|A| + 1)^d - 1 - |A| \;=\; |P \setminus \{\hat{c} : c \in A'\}|.$$

Possiamo quindi scegliere una funzione iniettiva

$$\omega\colon B' \to A'^*$$

tale che

- $\omega(c) = \hat{c}$ per ogni $c \in A$ e $\omega(\square) = \hat{\square}$;
- ω manda $B \setminus A$ in modo iniettivo in $P \setminus \{\hat{c} : c \in A'\}$.

Per costruzione $|\omega(b)| = d$ per ogni $b \in B'$, e ω è iniettiva perché le immagini sono a due a due distinte.
Indichiamo ancora con $\omega\colon B'^* \to A'^*$ l'unico omomorfismo di monoidi che la estende, cioè $\omega(b_1 \cdots b_n) = \omega(b_1) \cdots \omega(b_n)$. Poiché tutte le immagini delle lettere hanno la stessa lunghezza d, ω è iniettiva su B'^* (la decodifica si fa leggendo blocchi di d caratteri) e $|\omega(w)| = d\,|w|$. Inoltre, essendo $\omega(c) = \hat{c} \in A^*$ per $c \in A$, si ha $\omega(A^*) \subseteq A^*$: sugli input la codifica non introduce blank, e $\omega(X) \subseteq A^*$ per ogni $X \subseteq A^*$.
Sia ora $X \subseteq A^*$ un insieme che appartiene a $\mathrm{DTIME}_k^B(T(n))$, e sia μ una macchina di Turing a k nastri di alfabeto B che decide X in tempo $T(n)$. Per il lemma 9.21 e l'osservazione 9.22, esiste una macchina μ' a k nastri di alfabeto A che decide l'insieme $\omega(X) \subseteq A^*$ in tempo $3dT(m/d)$ su input di lunghezza m; poiché $|\omega(w)| = d\,|w|$, su $\omega(w)$ con $|w| = n$ essa impiega tempo $3dT(n)$.
Sia μ'' una macchina di alfabeto A che, a partire dalla configurazione iniziale associata alla parola $w \in A^*$, porta a una configurazione che, a meno della sostituzione dello stato iniziale con uno stato terminale, è la configurazione iniziale associata alla parola $\omega(w)$. Allora la macchina ottenuta per concatenazione di μ'' e μ', vale a dire riunendo le due tabelle e sostituendo ovunque in μ' lo stato iniziale con lo stato terminale di μ'', decide l'insieme X su alfabeto A. Il suo tempo è la somma dei tempi di μ'' e di μ'.
Resta da costruire μ'' e valutarne il tempo, per una parola w di lunghezza n.

Caso $k \geq 2$: Ricordiamo che la configurazione iniziale associata a una parola ha la parola scritta su tutti i k nastri, con le testine nella prima casella. La macchina μ'' opera in tre fasi.

1. Si scandisce w sul nastro 1 da sinistra a destra; per ogni carattere a letto, le testine dei nastri $2, \ldots, k$ scrivono (ciascuna sul proprio nastro, in parallelo) il blocco $\omega(a) = \hat{a}$. Servono dn passi. Poiché $dn \geq n$, i blocchi sovrascrivono interamente la copia di w presente sui nastri $2, \ldots, k$, che al termine contengono $\omega(w)$ con le testine in posizione dn; il nastro 1 contiene ancora w, con la testina in posizione n.
2. La testina del nastro 1 si sposta a destra fino alla posizione dn, in $(d-1)n$ passi.
3. Tutte le testine si spostano a sinistra fino alla posizione 0, in dn passi, e a ogni passo il carattere letto sul nastro 2 viene copiato sul nastro 1. Poiché $dn \geq n$, la copia sovrascrive tutto w.

Al termine tutti i nastri contengono $\omega(w)$, con le testine nelle caselle di partenza: è la configurazione iniziale associata a $\omega(w)$. Il tempo di μ'' è $dn + (d-1)n + dn = (3d-1)n \leq 3dn$, quindi il tempo totale è

$$3dT(n) + 3dn \leq 3dT(n) + dT(n) = 4dT(n),$$

dove si è usato $T(n) \geq 3n$.

Caso $k = 1$. Con un solo nastro bisogna procedere carattere dopo carattere, spostando ogni volta a destra la parte restante della parola per fare posto al blocco. Se l'i-esimo carattere ($1 \leq i \leq n$) è seguito da $n - i$ caratteri ancora da elaborare, la sua sostituzione con il blocco di d caratteri richiede al più $d + 2(n - i + 1)$ passi (inclusi gli spostamenti avanti e indietro). Sommando,

$$\sum_{i=1}^{n} \big(d + 2(n - i + 1)\big) = dn + n(n + 1) = n^2 + (d + 1)n,$$

a cui va aggiunto il ritorno della testina alla posizione iniziale, che richiede al più dn passi. In totale

$$n^2 + (2d + 1)n \leq 2n^2 \qquad \text{per } n \geq 2d + 1.$$

Per gli $n \leq 2d$ le parole di tale lunghezza sono in numero finito: la macchina può leggere l'input memorizzandolo negli stati e decidere direttamente l'appartenenza a X, in al più $n + 1 \leq 2n^2$ passi (per $n \geq 1$; il caso $n = 0$ è banale). Dunque in ogni caso il tempo di μ'' è al più $2n^2$, e il tempo totale è al più $3d\,T(n) + 2n^2$.

□

Si ricordi che con la notazione 2^{A^*} stiamo indicando l'insieme delle parti di A^*, ovvero tutti i possibili insiemi di parole di alfabeto A; allora, quando scriviamo

$$2^{A^*} \cap \mathrm{DTIME}_k^B(T(n))$$

stiamo indicando gli insiemi $X \subseteq A^*$ decidibili da una macchina di Turing di alfabeto B, e quindi le parole di X sono di alfabeto A ma la macchina può utilizzare durante la computazione anche i simboli aggiuntivi che sono in B, che pertanto talvolta viene detto *alfabeto di lavoro*. Come vedremo, la possibilità di utilizzare simboli aggiuntivi (cambiando l'alfabeto di lavoro di una macchina) permetterà di accelerarne la computazione. Così come nel valutare la complessità non teniamo conto del numero di stati utilizzati disporre di più stati (infatti, usare un insieme di stati più grande può rendere più veloce una macchina solo di un fattore costante), così non sembra utile restringere l'alfabeto utilizzato durante la computazione all'alfabeto del problema:

Definizione 9.24. Siano T ed S due funzioni di complessità ed A un alfabeto qualunque. Un insieme $X \subseteq A^*$ si dice appartenere alla classe $\mathrm{DTIME}(T(n))$ (rispettivamente $\mathrm{DSPACE}(S(n))$) se esiste un alfabeto B con $A \subseteq B$ tale che

$$X \in \mathrm{DTIME}^B(T(n)) \qquad (\text{rispettivamente } X \in \mathrm{DSPACE}^B(S(n))).$$

Grazie al lemma precedente è possibile mettere in relazione le classi di complessità appena definite con quelle cui siamo abituati:

Proposizione 9.25. Se l'insieme $X \subseteq A^*$ e $X \in \mathrm{DTIME}(T(n))$ con $T(n) \geq 3n$ allora esiste $c > 0$ tale che $X \in \mathrm{DTIME}^A(c\,T(n))$. Esiste inoltre un'ulteriore costante $c_1 > 0$ tale che $X \in \mathrm{DTIME}_1^A(c_1\,T^2(n))$.

Dimostrazione. Per definizione $X \in \mathrm{DTIME}(T(n))$ implica l'esistenza di B tale che $X \in \mathrm{DTIME}_k^B(T(n))$. Possiamo supporre $k > 1$, altrimenti si può indebolire immergendo la macchina mononastro in una a 2 nastri. Per il lemma 9.23

$$X \in \mathrm{DTIME}_k^A\big(4d\,T(n)\big)$$

dove d è preso in modo tale che $|B| + 1$ è al più $(|A| + 1)^d$. Un tale intero d esiste perché necessariamente $|A| + 1 \geq 2$.
Per il secondo enunciato, usando la relazione (3), otteniamo che

$$X \in \mathrm{DTIME}_1^B\big(c'\,T^2(n)\big).$$

per qualche costante c'. Applichiamo ora il Lemma 9.23 con $k = 1$ e funzione di complessità $c'\,T^2(n)$, con lo stesso d di prima:

$$X \in \mathrm{DTIME}_1^A\big(3d\,c'\,T^2(n) + 2n^2\big).$$

Poiché $T(n) \geq n$, si ha $n^2 \leq T^2(n)$, quindi

$$3d\,c'\,T^2(n) + 2n^2 \leq (3d\,c' + 2)\,T^2(n),$$

e la tesi vale con $c_1 = 3d\,c' + 2$. □

Osservazione 9.26. L'ipotesi $T(n) \geq 3n$ nel caso $k \geq 2$ del Lemma 9.23 serve soltanto per assorbire il costo della macchina μ'' nel termine $d\,T(n)$. Nella dimostrazione, infatti, per $k \geq 2$ si è ottenuto, senza alcuna ipotesi su T,

$$2^{A^*} \cap \mathrm{DTIME}_k^B(T(n)) \subseteq \mathrm{DTIME}_k^A\big(3d\,T(n) + 3dn\big) \qquad \text{se } k \geq 2,$$

che è l'analogo, per $k \geq 2$, della prima inclusione del lemma (dove compare il termine $2n^2$ per $k = 1$). Da questa inclusione si ricava l'enunciato con costante moltiplicativa sotto ipotesi più deboli:

- se $T(n) \geq n$ per ogni n, allora $3dn \leq 3d\,T(n)$, e quindi

$$2^{A^*} \cap \mathrm{DTIME}_k^B(T(n)) \subseteq \mathrm{DTIME}_k^A\big(6d\,T(n)\big) \qquad \text{se } k \geq 2;$$

- se $T(n) \geq 3n$, si ottiene la costante migliore $4d$, come nell'enunciato del lemma, perché allora $3dn \leq d\,T(n)$.

In entrambi i casi la tesi è della stessa forma, cioè $\mathrm{DTIME}_k^A\big(c\,T(n)\big)$ con c dipendente solo da d; cambia soltanto il valore della costante.

Corollario 9.27. Sia $X \subseteq \Sigma^*$ e $T(n)$ una funzione di complessità. Se $X \in \mathrm{DTIME}(T(n))$, allora esistono una codifica $\omega\colon \Sigma^* \to \{0,1\}^*$ e una costante $c > 0$ tali che

$$\omega(X) \in \mathrm{DTIME}_1^{\{0,1\}}\big(c\,T^2(n)\big).$$

Dimostrazione. Con lo stesso ragionamento fatto nell'osservazione 9.26, possiamo assumere $T(n) \geq 3n$. Per definizione esistono un alfabeto $B \supseteq \Sigma$ e $k \geq 1$ tali che $X \in \mathrm{DTIME}_k^B(T(n))$. Poniamo $d := \max\{1, \lceil \log_2 |B| \rceil\}$, così che $|B| \leq 2^d$, e scegliamo una funzione iniettiva $\omega\colon B \to \{0,1\}^d$. Per la Definizione 11.20 e la Remark 11.22, poiché tutte le immagini hanno la stessa lunghezza d e ω è iniettiva sulle lettere, l'omomorfismo esteso $\omega\colon B^* \to \{0,1\}^*$ è iniettivo, quindi è una codifica con $|\omega(w)| = d\,|w|$. In particolare la sua restrizione a Σ^* è una codifica $\Sigma^* \to \{0,1\}^*$ e $w \in X \iff \omega(w) \in \omega(X)$.

Passo 1: Poniamo $\omega(\square) = \square^d$. Poiché $\omega(X) \subseteq \{0,1\}^*$, il Lemma 9.21, nella forma dell'Osservazione 9.22, applicato a $Y = X$ e ad $A = \{0,1\}$, dà

$$\omega(X) \in \mathrm{DTIME}_k^{\{0,1\}}\big(S(n)\big), \qquad S(n) := 3d\,T(\lceil n/d \rceil).$$

Passo 2: La funzione S è non decrescente perché lo è T, e, poiché $T(m) \geq 3m$,

$$S(n) \geq 3d \cdot 3\lceil n/d \rceil \geq 9n \geq n,$$

quindi S è una funzione di complessità. Applicando la relazione (3) sull'alfabeto $\{0,1\}$ e con S al posto di T si ottiene

$$\omega(X) \in \mathrm{DTIME}_1^{\{0,1\}}\big(4k(k+6)\,S^2(n)\big).$$

Passo 3: Poiché $d \geq 1$ si ha $\lceil n/d \rceil \leq n$, e T è non decrescente, quindi $T(\lceil n/d \rceil) \leq T(n)$ e

$$S^2(n) = 9d^2\,T^2(\lceil n/d \rceil) \leq 9d^2\,T^2(n).$$

Dunque

$$\omega(X) \in \mathrm{DTIME}_1^{\{0,1\}}\big(c\,T^2(n)\big), \qquad c := 36\,d^2\,k\,(k+6).$$

□

Questo ci dice che, un qualsiasi insieme decidibile, una volta codificato in binario, può essere deciso da una macchina ad un nastro che utilizza come alfabeto un insieme con soli due elementi.

9.6 Teorema di Speed-up

I risultati precedenti mostrano che quando si estende l'alfabeto si può passare, alla peggio, da una complessità $T(n)$ a una complessità $cT(n)$ per una qualche costante c. Andiamo ora a mostrare che, partendo da un insieme X decidibile in tempo $T(n)$ e da una costante c qualsiasi, si può, cambiando l'alfabeto, costruire una nuova macchina che decide lo stesso insieme in tempo $cT(n)$. Dunque possiamo vedere una corrispondenza bidirezionale tra i cambi di complessità del tipo $T(n) \leftrightarrow cT(n)$ e i cambi di alfabeto.

Theorem 9.28 (Speed-Up)**.** Sia T una funzione di complessità. Allora per ogni costante $c > 0$ si ha che

$$\mathrm{DTIME}(T(n)) = \mathrm{DTIME}(c\,T(n)).$$

Prima della dimostrazione isoliamo il fatto che permette di simulare d passi con soli 3 passi. Sia Γ l'alfabeto di lavoro di una macchina μ a un nastro (con $\square \in \Gamma$) e fissiamo $d \geq 1$. Dividiamo il nastro in *blocchi* di d celle consecutive: il blocco i è formato dalle celle id, $id+1, \ldots, id+d-1$, e il suo contenuto è una parola $\beta_i \in \Gamma^d$. Se la testina è nella cella p, scriviamo $p = id + r$ con $0 \leq r < d$: diciamo che si trova nel blocco $i = \lfloor p/d \rfloor$, in posizione $r = p \bmod d$.

Lemma 9.29. Sia (f, p, q) una configurazione di μ con la testina nel blocco i. Allora, nei successivi d passi di μ:

1. la testina visita, oltre al blocco i, al più *uno* tra i blocchi $i-1$ e $i+1$;
2. il lato dal quale la testina esce per la prima volta dal blocco i (se esce) è determinato da q, r e β_i.

Di conseguenza la configurazione dopo d passi differisce dalla precedente al più nei blocchi i e j, dove $j \in \{i-1, i+1\}$ è il blocco vicino visitato, e la testina si trova in i o in j. Inoltre il nuovo stato, la nuova posizione r' nel blocco e i nuovi contenuti β_i', β_j' sono funzione soltanto di (q, r, β_i, β_j).

Dimostrazione. Per raggiungere il blocco $i-1$ dalla cella $p = id + r$ servono almeno $r+1$ passi, e da lì per raggiungere il blocco $i+1$ (cella $(i+1)d$) ne servono almeno altri $d+1$: in totale almeno $r + d + 2 > d$ passi. Analogamente, per raggiungere prima $i+1$ servono almeno $d - r \geq 1$ passi, e poi almeno $d+1$ per arrivare al blocco $i-1$: in totale almeno $d+2 > d$. Quindi in d passi i due vicini non possono essere visitati entrambi, e questo prova il punto 1.

Finché la testina resta nel blocco i, la computazione di μ legge solo celle di β_i, quindi dipende solo da (q, r, β_i) e in particolare lo è il lato dal quale esce, se esce: questo prova il punto 2. L'ultima affermazione segue perché, una volta noti β_i e β_j, tutte le celle lette nei d passi sono note, e la simulazione è un calcolo su dati finiti. □

Dimostrazione del Teorema 9.28. Supponiamo $0 < c < 1$ (in caso contrario applicare il teorema a $c' = 1/c$): il teorema prende allora il significato che ogni computazione di lunghezza t può essere accelerata per avere lunghezza ct.

La prima inclusione

$$\mathrm{DTIME}(cT(n)) \subset \mathrm{DTIME}(T(n))$$

è banale: se un algoritmo termina in al più $cT(n)$ passi, termina a maggior ragione in al più $T(n)$ passi, essendo $cT(n) < T(n)$.

Per l'altra inclusione

$$\mathrm{DTIME}(T(n)) \subset \mathrm{DTIME}(cT(n))$$

dobbiamo associare alla macchina μ che decide X in tempo $T(n)$ una macchina μ', con alfabeto esteso, che simula μ in tempo $cT(n)$. Per definizione esiste un alfabeto di lavoro $\Gamma \supset A$ tale che $X \in \mathrm{DTIME}_k^\Gamma(T(n))$. Trattiamo il caso $k = 1$; il caso generale si ottiene in modo analogo (a meno di una costante diversa nel numero di passi per ciclo, che non cambia la conclusione perché d è arbitrario). La costruzione procede in quattro passi.

1. *Scelta di d e alfabeto di μ'.* Fissiamo

$$d := \left\lceil \frac{12}{c} \right\rceil \qquad \text{e} \qquad B := A \cup \Gamma^d.$$

Un simbolo di Γ^d rappresenta un intero blocco di d celle di μ. La macchina μ' ha un nastro in più rispetto a μ ($k' = k + 1$): sul secondo nastro viene scritta la versione compressa del nastro di μ, in cui la cella i contiene il simbolo $\beta_i \in \Gamma^d$.

2. *Compressione dell'input.* Leggendo w dal primo nastro, μ' ricorda negli stati gli ultimi (al più d) simboli letti e, ogni volta che ne ha letti d, scrive sul secondo nastro il simbolo di Γ^d corrispondente (l'ultimo blocco viene completato con $\square$). Poi riporta la testina del secondo nastro all'inizio. Questo costa al più

$$n + \lceil n/d \rceil \leq n + \frac{n}{d} + 1 \leq 2n + 1 \quad \text{passi.}$$

3. *Simulazione a cicli.* Gli stati di μ' sono del tipo (q, r, β) (più quelli intermedi del ciclo, in numero finito), con il significato:

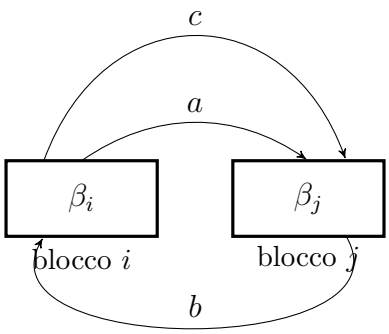
   - q è lo stato di μ;
   - r è la posizione della testina di μ nel blocco corrente;
   - $\beta \in \Gamma^d$ è il *contenuto vero* del blocco corrente, che può essere diverso da quello scritto sul nastro di μ'.

   Così lo stato di μ' "ricorda" il blocco sotto la testina, e non serve rileggerlo. Sia μ' nella cella i dello stato (q, r, β_i). Per il Lemma 9.29, calcolando internamente (cioè senza spendere passi) le prime transizioni di μ, μ' conosce il lato $s \in \{-1, +1\}$ da cui μ uscirebbe dal blocco, e pone $j := i + s$. Se μ non esce mai dal blocco in d passi, sceglie $j := i + 1$ (il vicino non verrà modificato). Un ciclo di μ' è il seguente.

   (a) In i: scrive β_i (cioè il contenuto vero, aggiornando il nastro) e si sposta in j.

   (b) In j: la transizione legge β_j. Ora μ' conosce (q, r, β_i, β_j) e, per il Lemma 9.29, calcola q', r', β_i', β_j' e il blocco in cui finisce la testina di μ. Scrive β_j' in j e si sposta in i. Se la testina di μ finisce nel blocco i, il ciclo termina qui, nel nuovo stato (q', r', β_i'). Il nastro contiene ancora il vecchio β_i in i, ma non è un problema: sarà aggiornato al passo (a) del ciclo successivo.

   (c) Solo se la testina di μ finisce nel blocco j: in i scrive β_i' e si sposta in j, che già contiene β_j' ed è il blocco in cui ci si trova ora, nello stato (q', r', β_j').



Ogni ciclo dura al più 3 passi (il passo (c) c'è solo se la testina finisce nel vicino) e simula esattamente d passi di μ (oppure meno, se μ si arresta: in tal caso μ' entra nello stato di arresto corrispondente). Per costruzione, dopo ogni ciclo la configurazione di μ' è l'immagine, tramite la codifica dei blocchi, di quella di μ dopo d passi; quindi μ' e μ decidono lo stesso insieme X.

4. *Conteggio dei passi.* Per simulare i $T(n)$ passi di μ servono al più $\lceil T(n)/d \rceil \leq T(n)/d+1$ cicli, quindi al più $\frac{3T(n)}{d} + 3$ passi. Contando anche la lettura iniziale del primo blocco (un passo), il tempo totale di μ' su un input di lunghezza n è

$$t_{\mu'}(n) \leq \underbrace{2n+1}_{\text{compressione}} +1 + \frac{3T(n)}{d} + 3 = 2n + 5 + \frac{3T(n)}{d}.$$

Dalla scelta $d \geq 12/c$ segue

$$\frac{3T(n)}{d} \leq \frac{c}{4}\, T(n).$$

Per ipotesi $\lim_{n\to\infty} n/T(n) = 0$; in particolare $T(n) \to \infty$. Quindi per n abbastanza grande valgono

$$2n \leq \frac{c}{2}\, T(n) \qquad \text{e} \qquad 5 \leq \frac{c}{4}\, T(n),$$

da cui

$$t_{\mu'}(n) \leq \frac{c}{2} T(n) + \frac{c}{4} T(n) + \frac{c}{4} T(n) = c\, T(n).$$

Quindi $X \in \text{DTIME}(cT(n))$.

□

Grazie al teorema di speed-up possiamo, nel valutare le classi di complessità, considerare solo l'ordine di grandezza della funzione di complessità $T(n)$. In particolare, per le classi di complessità polinomiali sarà sufficiente considerare l'esponente del termine dominante del polinomio, risultano così descritte da un unico parametro k: $\text{DTIME}(n^k)$. Il teorema di Speed-up ha anche come conseguenza che risulta determinata la relazione tra una classe di complessità espressa in funzione di un alfabeto A e la classe di complessità che le corrisponde senza aver specificato l'alfabeto:

Proposizione 9.30. Sia A un alfabeto con almeno due simboli e sia $T(n) \geq 1$ una funzione di complessità. Allora

$$2^{A^*} \cap \mathrm{DTIME}(T(n)) = \bigcup_{c>0} \mathrm{DTIME}^A(cT(n)).$$

Dimostrazione. Applichiamo il teorema 9.28 a T.

($\supseteq$) Sia $c > 0$ e $X \in \mathrm{DTIME}^A(cT(n))$. Allora $X \subset A^*$ e, a maggior ragione, $X \in \mathrm{DTIME}(cT(n))$ (basta prendere $\Gamma = A$ come alfabeto di lavoro). Per il Teorema 9.28, $\mathrm{DTIME}(cT(n)) = \mathrm{DTIME}(T(n))$, dunque $X \in 2^{A^*} \cap \mathrm{DTIME}(T(n))$.

($\subseteq$) Sia $X \subset A^*$ con $X \in \mathrm{DTIME}(T(n))$. Esistono un alfabeto $\Gamma \supset A$, un intero $k \geq 1$ e una macchina μ a k nastri con alfabeto di lavoro Γ che decide X in tempo $T(n)$. Dobbiamo trovare una macchina μ' con alfabeto di lavoro A che decide X in tempo $cT(n)$ per un opportuno $c > 0$.

Scegliamo $m \in \mathbb{N}$ con $|A|^m \geq |\Gamma|$ e una codifica iniettiva $e\colon \Gamma \to A^m$ con $e(\square) = \square^m$. Ogni simbolo di Γ viene così rappresentato da un blocco di m celle. La macchina μ' ha $k+1$ nastri: il nastro 1 contiene l'input, i nastri $2, \ldots, k+1$ contengono le versioni codificate dei nastri di μ.

1. *Codifica dell'input.* Leggendo $w = a_1 \cdots a_n$ dal nastro 1, μ' scrive $e(a_1) \cdots e(a_n)$ sul nastro 2 (per ogni a_i servono m passi, durante i quali la testina del nastro 1 avanza di una cella), poi riporta la testina del nastro 2 all'inizio. I nastri $3, \ldots, k+1$ vengono svuotati, e il vuoto è già codificato correttamente da $\square^m$. Costo: al più $2mn+1$ passi.

2. *Simulazione di un passo.* Con le testine all'inizio dei blocchi corrispondenti alle celle osservate da μ, μ' procede in tre fasi, muovendo in parallelo tutte le testine.

   - Legge i blocchi spostandosi di $m-1$ celle a destra, memorizzando negli stati i simboli letti. Ora conosce i k simboli letti da μ.
   - Applica internamente la transizione di μ, che è determinata dallo stato corrente e dai simboli letti. Quindi scrive i blocchi nuovi tornando indietro di $m-1$ celle.
   - Sposta ciascuna testina di m celle a sinistra, a destra o la lascia ferma, a seconda del movimento prescritto da μ.

   Un passo di μ costa così al più $(m-1)+(m-1)+m \leq 3m$ passi di μ'. Se μ entra in uno stato di arresto, μ' entra nello stato di arresto corrispondente. Per costruzione μ' accetta w se e solo se lo fa μ, quindi μ' decide X.

3. *Conteggio dei passi.* Il tempo di μ' su input di lunghezza n è

   $$t_{\mu'}(n) \leq 2mn + 1 + 3m\,T(n).$$

   Poiché $n/T(n) \to 0$ e $T(n) \to \infty$, esiste n_0 tale che per $n \geq n_0$ vale $2mn+1 \leq T(n)$, e quindi $t_{\mu'}(n) \leq (3m+1)\,T(n)$. Per i finiti input con $n < n_0$, μ' può leggere l'input in $n+1 \leq n_0\,T(n)$ passi (usando $T(n) \geq 1$) e rispondere in base a una tabella memorizzata negli stati. Posto $c := \max\{3m+1,\, n_0\}$ si ha allora $t_{\mu'}(n) \leq cT(n)$ per ogni n.

Dunque $X \in \mathrm{DTIME}^A(cT(n))$, il che conclude la dimostrazione. □

9.7 Macchine Speciali

Una macchina di Turing μ di alfabeto unario $A = \{1\}$ si dice *speciale* quando per auto-confinamento utilizza la porzione di nastro con indice $p > 0$:

Definizione 9.31. Una macchina di Turing di alfabeto A e insieme degli stati Q

$$\mu : Q \times A' \to Q' \times A' \times \{+1, -1\}$$

si dice *speciale* quando $A = \{1\}$ e per ogni parola $w = 1^n \in A^*$ la computazione di μ associata a w ha configurazioni di nastro positive:

$$\tau^k(\iota(w)) = (f, p, q) \text{ con } p > 0 \text{ per ogni } k \text{ per cui } \tau \text{ è definita.}$$

Proposizione 9.32. Data una macchina di Turing μ per seminastro di alfabeto $A = \{1\}$ esiste sempre una macchina di Turing speciale che la simula.

Dimostrazione. Osserviamo che se l'alfabeto di input è A la macchina a seminastro μ sarà definita utilizzando l'alfabeto $A'_s = A \cup \{\square, \|\}$, mentre la macchina speciale μ' di cui dobbiamo mostrare l'esistenza sarà definita sull'insieme $A' = A \cup \{\square\}$. Per avere la tesi basta creare una simulazione, per farlo basta creare da un'applicazione iniettiva $\phi : A'*_s \to A'^*$. Osservando che la mappa seguente

$$\begin{cases} \varphi(1) = 11 \\ \varphi(\|) = \square 1 \\ \varphi(\square) = \square\square \end{cases}$$

è suffix-free, abbiamo che φ induce una mappa iniettiva ϕ come voluto. Abbiamo quindi la tesi. □

In realtà, si può fare di più.

Osservazione 9.33. La Proposizione vale anche se la macchina a seminastro μ ha un alfabeto A qualunque, non necessariamente unario, mentre la macchina speciale μ' ha sempre alfabeto unario $\{1\}$ e alfabeto di lavoro $\{1, \square\}$. Basta cambiare la codifica φ, e il resto della dimostrazione resta identico.

Sia $A'_s = A \cup \{\square, \|\}$ l'alfabeto di lavoro di μ. Ciò che serve è una applicazione iniettiva a lunghezza costante

$$\varphi : A'_s \to \{1, \square\}^k,$$

per un certo $k \geq 1$. Un tale φ esiste se e solo se $|A'_s| \leq 2^k$, cioè basta prendere

$$k := \lceil \log_2(|A| + 2) \rceil.$$

Concretamente, numeriamo i simboli $A'_s = \{s_0, s_1, \ldots, s_{m-1}\}$ con $s_0 = \square$, e poniamo $\varphi(s_j)$ uguale alla scrittura di j in base 2 su k cifre, dove la cifra 1 è il simbolo 1 e la cifra 0 è il simbolo $\square$. In particolare $\varphi(\square) = \square^k$, quindi un nastro vuoto di μ corrisponde a un nastro vuoto di μ'.

Per $A = \{1\}$ si ha $|A'_s| = 3 \leq 2^2$ e $k = 2$: si ritrova la codifica $\varphi(1) = 11$, $\varphi(\|) = \square 1$, $\varphi(\square) = \square\square$ usata nella dimostrazione.

Poiché tutte le parole $\varphi(x)$ hanno la stessa lunghezza k, l'estensione ϕ alle parole è iniettiva, e la simulazione (leggere un blocco di k celle, riscriverlo, spostarsi di k celle a destra o a sinistra) non dipende da A. Il seminastro ha ancora il simbolo $\|$ nella cella 0, quindi μ' non esce mai dalle celle di indice > 0 ed è speciale.

Quindi, vale il seguente corollario:

Corollario 9.34. Data una macchina di Turing μ di alfabeto A qualunque, esiste sempre una macchina di Turing speciale μ' che la simula.

Dimostrazione.

1. *Seminastro.* Per la Proposizione 9.8, la macchina μ si simula con una macchina a seminastro μ_s di alfabeto A.

2. *Macchina speciale.* Per la Proposizione 9.32 e la Remark 9.33, la macchina a seminastro μ_s si simula con una macchina speciale μ' di alfabeto unario.

3. *Codifica.* L'input $w \in A^*$ è sostituito dalla parola 1^{n+1}, dove n è il numero rappresentato da w in base $|A|$, e analogamente per l'output; per la Proposizione 9.35 questa sostituzione non cambia la calcolabilità né la decidibilità.

□

9.8 Rappresentazione degli interi

Rappresentazione di un intero $x \in \mathbb{N}$ in base $b \geq 2$:

$$x = \sum_{i=0}^{m} d_i \, b^i \quad \text{con } d_i \in \{0, 1, \ldots, b-1\} \text{ e } d_m \neq 0 \;\; (\text{oppure } x = 0,\; m = 0,\; d_0 = 0).$$

Tra gli oggetti matematici su cui si riferiscono i problemi di decidibilità, gli interi hanno un posto importante, sia per il loro ruolo diretto che per il loro utilizzo per una ulteriore codifica di altri dati. Poiché ci sono diversi sistemi di rappresentazione mediante parole, la questione della robustezza rispetto ai cambiamenti della rappresentazione delle nozioni di decidibilità e di complessità sorge spontanea.

Le rappresentazioni più comuni di numeri interi tramite parole sono gli sviluppi relativi a una base fissata. Sia $b \geq 2$ un numero intero e sia $B = \{0, 1, \ldots, b-1\}$. Per ogni parola $d_n \cdots d_0 \in B^*$ non vuota indichiamo con $(d_n \cdots d_0)_b$ il numero intero

$$\sum_{i=0}^{n} d_i \, b^i.$$

L'applicazione $w \mapsto (w)_b$, che ad una parola associa il numero intero corrispondente, è suriettiva ma non iniettiva (le parole 5 e 05 rappresentano lo stesso numero). Diciamo che una parola è *canonica* se è la parola 0 oppure non inizia per 0. Ogni intero n è l'immagine

di una *unica* parola canonica: la chiamiamo *rappresentazione* di n in base b e la indichiamo con $(n)_b$. Quindi $w \mapsto (w)_b$ ristretta alle parole canoniche è una bigezione con $\mathbb{N}$, con inversa $n \mapsto (n)_b$, e vale $((n)_b)_b = n$.

Possiamo immaginare molte altre rappresentazioni di numeri interi, ovvero molte altre funzioni iniettive dell'insieme $\mathbb{N}$ nel monoide libero A^*. In particolare, si può ottenere una rappresentazione con un alfabeto unario (con un solo elemento) associando ad ogni numero intero $n \in \mathbb{N}$ la parola formata da una sequenza di $n+1$ uno:

$$(n)_1 := 1^{n+1}.$$

Si usano $n+1$ simboli, e non n, perché così lo zero ha una rappresentazione non vuota. Anche in base 1 poniamo $B = \{1\}$ e $(w)_1 := |w| - 1$ per ogni parola non vuota $w \in B^*$; in questo caso *ogni* parola non vuota è canonica, e $w \mapsto (w)_1$ è una bigezione tra le parole non vuote e $\mathbb{N}$, con inversa $n \mapsto (n)_1$. D'ora in avanti, salvo diversa indicazione, b indica un intero $b \geq 1$.

Questa notazione viene estesa a insiemi di interi $X \subset \mathbb{N}^k$: se

$$X = \{(x_1, \ldots, x_k) \mid x_i \in \mathbb{N}\} \subset \mathbb{N}^k$$

è un insieme di vettori di numeri interi, poniamo

$$(X)_b := \{(x_1)_b \,\square\, (x_2)_b \,\square \cdots \square\, (x_k)_b \mid (x_1, \ldots, x_k) \in X\} \subset B'^*$$

con $B' = B \cup \{\square\}$.

Analogamente, se F è una funzione da $\mathbb{N}$ in $\mathbb{N}$, indichiamo con $(F)_b$ la funzione che alla parola canonica w associa la parola $(F((w)_b))_b$.

Andiamo quindi a confrontare dal punto di vista della decidibilità i diversi insiemi $(X)_b$ ottenuti variando la base b (e di conseguenza l'alfabeto B).

Proposizione 9.35. Sia $X \subset \mathbb{N}$. Allora sono equivalenti:

(i) esiste $b \geq 1$ tale che $(X)_b$ è TM-decidibile;

(ii) per ogni $b \geq 1$ l'insieme $(X)_b$ è TM-decidibile.

Dimostrazione. L'implicazione (ii)⇒(i) è ovvia. Per (i)⇒(ii), sia $b \geq 1$ tale che $(X)_b$ sia decidibile da una macchina di Turing μ_b, e sia $b' \geq 1$ qualsiasi (anche $b' = b$ o una base uguale a 1). Costruiamo una macchina M a due nastri che decide $(X)_{b'}$; poiché ogni macchina a più nastri si simula con una macchina a un nastro, questo basta. La costruzione procede in quattro passi.

1. *Operazioni elementari sui nastri.* Per ogni base $\beta \geq 1$ consideriamo un nastro che contiene una parola non vuota in base β, e indichiamo con v il numero rappresentato. Descriviamo tre operazioni, ciascuna realizzabile da una macchina con un numero finito di stati.

   - *Test di zero* ($v = 0$?). Se $\beta \geq 2$: si scorre la parola e si controlla che tutte le cifre siano 0 (la parola può avere zeri iniziali dopo i decrementi). Se $\beta = 1$: si controlla che la parola sia formata da un solo simbolo 1.

- *Decremento* ($v \mapsto v-1$, definito se $v > 0$). Se $\beta \geq 2$: si porta la testina sull'ultima cifra; finché la cifra letta è 0 la si sostituisce con $\beta - 1$ e ci si sposta a sinistra; alla prima cifra $d \geq 1$ la si sostituisce con $d - 1$ e ci si ferma. Il ciclo termina perché $v > 0$ garantisce l'esistenza di una cifra non nulla. Se $\beta = 1$: si cancella l'ultimo 1 (la parola ha almeno due simboli perché $v > 0$, quindi resta non vuota).
- *Incremento* ($v \mapsto v + 1$). Se $\beta \geq 2$: si porta la testina sull'ultima cifra; finché la cifra letta è $\beta - 1$ la si sostituisce con 0 e ci si sposta a sinistra; alla prima cifra $d < \beta - 1$ la si sostituisce con $d + 1$ e ci si ferma; se si esce dalla parola a sinistra si scrive 1 nella casella vuota. Se $\beta = 1$: si aggiunge un 1 in fondo alla parola.

In ciascun caso la nuova parola rappresenta effettivamente $v - 1$ oppure $v + 1$.

Se $\beta \geq 2$, l'incremento di una parola canonica produce ancora una parola canonica. La cifra più a sinistra viene modificata solo se tutte le altre cifre sono uguali a $\beta - 1$, e ci sono tre casi:

- se la parola è 0, diventa 1 (perché $\beta - 1 \geq 1$);
- se la cifra iniziale $d \geq 1$ è minore di $\beta - 1$, diventa $d + 1 \neq 0$;
- se la cifra iniziale è uguale a $\beta - 1$, diventa 0 e si scrive un 1 davanti.

Se $\beta = 1$, ogni parola non vuota è canonica, e le operazioni non producono mai la parola vuota.

2. *Alfabeto di M.* Sia Γ_b l'alfabeto di lavoro di μ_b, che contiene l'alfabeto dell'input di μ_b e il simbolo $\square$. Poniamo

$$\Gamma := \Gamma_b \cup \{0, 1, \ldots, b' - 1\} \cup \{1, \square\}.$$

La macchina M ha alfabeto di lavoro Γ. Non si richiede quindi che durante la computazione si usino solo i simboli dell'input. Inoltre ogni transizione di μ_b è anche una transizione di M, perché $\Gamma \supset \Gamma_b$.

3. *Definizione di M.* Il nastro 1 contiene l'input $w' \in \{0, \ldots, b' - 1\}^*$ se $b' \geq 2$, oppure $w' \in \{1\}^*$ se $b' = 1$. Il nastro 2 viene usato come nastro di lavoro di μ_b. Indichiamo con v_1 il numero rappresentato sul nastro 1 in base b' e con v_2 quello rappresentato sul nastro 2 in base b. La macchina M procede in quattro fasi.

Fase 0. M scorre il nastro 2 sostituendo ogni simbolo con $\square$ e riporta la testina all'inizio, in modo che il nastro 2 sia vuoto.

Fase 1. (controllo e inizializzazione) M controlla con una scansione che w' sia una parola canonica in base b' (se $b' \geq 2$: non vuota e uguale a 0 oppure non iniziante per 0; se $b' = 1$: non vuota); in caso contrario rifiuta. Poi scrive sul nastro 2 la parola $(0)_b$, cioè 0 se $b \geq 2$ e 1 se $b = 1$. Detto $n := (w')_{b'}$, all'inizio $v_1 = n$ e $v_2 = 0$.

Fase 2. (conversione) M ripete i seguenti passi.

i. Esegue il test di zero sul nastro 1 (in base b'): se $v_1 = 0$ passa alla Fase 3.

ii. Altrimenti esegue il decremento sul nastro 1 (in base b').

iii. Esegue l'incremento sul nastro 2 (in base b) e torna al punto (a).

Ad ogni ripetizione v_1 diminuisce di 1 e v_2 aumenta di 1, quindi all'inizio di ogni ripetizione vale $v_1 + v_2 = n$. Poiché v_1 diminuisce strettamente, dopo n ripetizioni $v_1 = 0$ e M passa alla Fase 3, con $v_2 = n$. Per quanto osservato sulle operazioni, la parola sul nastro 2 è sempre canonica in base b, quindi al termine della Fase 2 il nastro 2 contiene proprio $(n)_b$.

Fase 3. (simulazione di μ_b) M riporta la testina del nastro 2 sulla prima cifra di $(n)_b$ ed esegue da lì le transizioni di μ_b, usando il nastro 2 come nastro di μ_b (gli stati di μ_b sono tra gli stati di M). Accetta se e solo se μ_b accetta.

4. *Correttezza.* M si ferma su ogni input: le Fasi 1 e 2 terminano per quanto visto, e la Fase 3 termina perché μ_b decide $(X)_b$. Inoltre, per l'input w',

$$w' \in (X)_{b'} \iff w' \text{ è canonica e } (w')_{b'} = n \in X \iff (n)_b \in (X)_b \iff \mu_b \text{ accetta } (n)_b,$$

dove la seconda equivalenza segue dal fatto che $n \mapsto (n)_b$ è una bigezione tra $\mathbb{N}$ e le parole canoniche in base b. Quindi M decide $(X)_{b'}$.

□

10 Funzioni ricorsive

10.1 Funzioni Turing-computabili

L'oggetto tecnico di questa sezione consiste nella sostituzione del concetto di *insieme decidibile per macchina di Turing* con quello di *funzione calcolabile per macchina di Turing*. Finora, abbiamo considerato gli algoritmi di decisione in cui l'input è una parola w formata su un certo alfabeto A, e dove l'output è un booleano "vero" o "falso" a seconda che la parola w appartenga o meno all'insieme X che bisogna decidere. Quando si tratta di determinare i valori di una funzione F da A^* in B^*, diremo che un algoritmo calcola la funzione F se, dalla configurazione la parola iniziale associata a una parola w di A^*, si arriva in un numero finito di passi di computazione, a una configurazione terminale (accettante) associata alla parola $F(w)$.

Al fine di trattare contemporaneamente i casi delle funzioni a più variabili e/o i casi delle funzioni a più valori, diamo le seguenti definizioni:

Definizione 10.1. Sia A un alfabeto qualunque, e siano k ed r due interi positivi. Per ogni vettore di r elementi in A^*: $(w_1, w_2, \ldots, w_r) \in (A^*)^r$, la configurazione iniziale a k nastri associata al vettore $(w_1, w_2, \ldots, w_r)$ è data dalla tripla (f, p, q) dove $f\colon \mathbb{Z} \to (A')^k$, $p = (1, \ldots, 1) \in \mathbb{Z}^k$ e $q = q_0$ è lo stato iniziale in Q,

$$f(k) = \begin{cases} w_j\left(k - \sum_{i=1}^{j-1}(|w_i|+1)\right) & \text{se } \sum_{i=1}^{j-1}(|w_i|+1) < k < \sum_{i=1}^{j}(|w_i|+1) \\ \square & \text{altrimenti} \end{cases}$$

Diremo poi che una configurazione terminale a k nastri associata al vettore $(w_1, w_2, \ldots, w_r)$ è data dalla tripla (f, p, q) dove f e p sono presi come nel caso della configurazione iniziale e $q = q_F^1$ è lo stato finale della macchina.

In pratica la configurazione iniziale per una data macchina corrisponde alla seguente situazione: sul primo nastro è memorizzato l'input a partire dalla casella 1 e lasciando un blank per separare le parole w_i, mentre gli altri nastri sono vuoti; ogni testina è posizionata sulla casella 1 di ciascun nastro. La configurazione finale è del tutto analoga: gli ultimi $k-1$ nastri sono vuoti e l'output è memorizzato sul primo nastro a partire dalla casella 1 e le testine sono tutte allineate in posizione 1; coincide con la configurazione iniziale multinastro associata alla parola di ingresso.

Come nel caso della decidibilità sarà possibile che una computazione non termini, questa situazione corrisponderà al caso di una funzione parziale, ovvero di una funzione il cui dominio non è tutto l'insieme di partenza.

Definizione 10.2. Siano A e B due alfabeti e F una funzione, non necessariamente definita dappertutto, da $(A^*)^r$ a $(B^*)^s$. Si dice che la funzione F è calcolabile per macchina di Turing o *Turing calcolabile*, se esiste un numero intero k e una macchina di Turing μ a k nastri il cui alfabeto include A e B tale che, per ogni sequenza finita $(w_1, \ldots, w_r) \in (A^*)^r$

i) se $F(w_1, \ldots, w_r)$ è definito allora la computazione di μ a partire dalla configurazione iniziale a k nastri associata a $(w_1, \ldots, w_r)$ arriva alla configurazione terminale associata a $F(w_1, \ldots, w_r)$; ovvero, la parola nella configurazione finale coincide con $F(w_1, \ldots, w_r)$;

ii) se $F(w_1, \ldots, w_r)$ non è definito, allora il calcolo di μ dalla configurazione iniziale a k nastri associata a $(w_1, \ldots, w_r)$ non termina mai.

È importante discutere il fatto che una configurazione terminale corrisponda al fatto che tutti i nastri tranne il primo sono vuoti (nel senso che contengono solo il $\square$): questa condizione nella definizione di configurazione terminale è dettata dalla possibilità di comporre in modo naturale la computazione di due macchine di Turing, cosa che non sarebbe possibile nel caso in cui la configurazione finale non corrispondesse alla configurazione iniziale per la seconda macchina. D'altra parte la condizione non è restrittiva in quanto se esiste una macchina di Turing che computa una funzione senza "cancellare" i nastri è possibile definire una macchina μ' che fa la stessa computazione essendo pronta a ripulire i nastri diversi dal primo nella parte utilizzata, più precisamente:

Lemma 10.3. Per ogni macchina di Turing μ a k nastri di alfabeto A e insieme degli stati Q, esiste una macchina μ' che per ogni configurazione iniziale associata ad un vettore $(w_1, \ldots, w_r)$ termina la computazione con lo stesso contenuto della μ sul primo nastro ed i rimanenti $k-1$ nastri posti identicamente uguali al $\square$, in particolare se il primo nastro contiene la sequenza corrispondente al vettore $(w'_1, \ldots, w'_m)$ allora la macchina μ' terminerà nella configurazione terminale associata a $(w'_1, \ldots, w'_m)$.

Dimostrazione. Per dimostrare questa affermazione bisogna introdurre un nuovo simbolo nell'alfabeto ed utilizzarlo come marcatore di inizio e fine della zona di nastro che è stata utilizzata durante la computazione: quindi la macchina μ' sarà una macchina di Turing di alfabeto $A' = A \cup \{*\}$ supposto che $*$ non appartiene ad A. Analogamente bisognerà modificare l'insieme degli stati per tenere conto delle operazioni da fare al momento in cui si incontra il marcatore proveniendo da sinistra o da destra. Si può utilizzare ad esempio un certo carattere speciale su ogni nastro che marca le estremità destra e sinistra della parte di nastro effettivamente usata. Quando si incontra un carattere che corrisponde all'estremità destra, si spinge il carattere speciale di una casella a destra e si torna per continuare il calcolo; analogamente nell'estremità a sinistra. Alla fine del calcolo ci si sposta fino al carattere speciale di destra, e mentre si torna verso sinistra fino al carattere speciale di sinistra si cancellano tutti i nastri (tranne nel caso di un carattere speciale della casella numero 1). Finalmente torneremo alla casella 1 riconoscibile per il suo carattere speciale. $\square$

Avremmo potuto chiedere che qualora la funzione F non fosse definita per un certo valore, il calcolo della macchina di Turing termini, ad esempio, in uno stato di rifiuto. Questa sarebbe stata una definizione non equivalente. È davvero ovvio come trasformare una macchina che si ferma in uno stato di rifiuto in una macchina che non si ferma mai. D'altra parte, il passaggio inverso non è necessariamente possibile, e riguarda l'"indecidibilità" del problema dell'arresto per una macchina di Turing, problema che sarà discusso più avanti nel corso.

Con immediato adattamento dei risultati delle lezioni precedenti sulla rappresentazione degli interi e la decidibilità di sotto-insiemi di numeri interi, si dimostra facilmente che

qualsiasi funzione calcolabile da una macchina di Turing è calcolabile con una macchina di Turing ad un nastro (o anche per un semi-nastro si può adattare la definizione).

Se $(n_1, \ldots, n_r)$ è una sequenza finita di numeri interi e $b > 1$ una qualsiasi base, si denoti con $((n_1, \ldots, n_r))_b$ la sequenza finita di parole $((n_1)_b, \ldots, (n_r)_b)$. Quindi, se F è una funzione di $\mathbb{N}^r$ in $\mathbb{N}^s$, denotiamo con $(F)_b$ la funzione da $\{0, 1, \ldots, b-1\}^*$ in $\{0, 1, \ldots, b-1\}^*$ che associa alla sequenza $(n_1, \ldots, n_r)_b$ la sequenza $(F(n_1, \ldots, n_r))_b$. I risultati della Sezione 15.1 mostrano che una qualsiasi funzione $(F)_b$ è Turing calcolabile non appena esiste una base b rispetto alla quale lo è. Diremo in questo caso (quando esiste una qualsiasi base b rispetto a cui la funzione è Turing computabile) semplicemente che la funzione F è Turing calcolabile.

Definizione 10.4. Il grafico di una funzione F, non necessariamente ovunque definita, da $(A^*)^r$ a $(B^*)^s$ è l'insieme

$$G_F := \{(a_1, \ldots, a_r, b_1, \ldots, b_s) \mid F(a_1, \ldots, a_r) \text{ è definita e } (b_1, \ldots, b_s) = F(a_1, \ldots, a_r)\}.$$

Il collegamento tra Turing decidibilità e Turing calcolabilità è semplice. Ricordiamo che la funzione caratteristica di un insieme X è

$$\mathbb{1}_X(x) = \begin{cases} 1 & \text{se } x \in X \\ 0 & \text{se } x \notin X \end{cases}$$

allora si ha che

Proposizione 10.5.

i) Un insieme X è Turing decidibile se e solo se la sua funzione caratteristica $\mathbb{1}_X$ è Turing calcolabile.

ii) Supponiamo che F sia ovunque definita su $(A^*)^r$, allora avremo che F è Turing calcolabile se e solo se il grafico di F è un insieme Turing decidibile.

Dimostrazione.

i) La funzione caratteristica di un insieme X (che si presume incluso in A^*) è una funzione F tale che $F(w)$ è 1 per w in X e 0 altrimenti. Se la funzione F è Turing calcolabile, è immediato trasformare una macchina di Turing che la calcola in una macchina di Turing che decide X: basta sostituire la configurazione $(0, q_F)$ nella configurazione $(0, q_F^0)$ e la configurazione $(1, q_F)$ in $(1, q_F^1)$. Il passaggio inverso è più delicato perché, con le definizioni che abbiamo posto, la Turing calcolabilità di F richiede non solo un algoritmo che, a partire dalla configurazione iniziale associata a w, raggiunga a una configurazione di nastro con 1 o 0 nella casella appropriata di un nastro prestabilito dedicato all'output, ma più specificatamente ad una configurazione analoga a quella iniziale, che dunque termina nella configurazione iniziale associata a 1 o 0, il che presuppone che tutti i nastri siano stati cancellati e i puntatori siano tornati nella posizione iniziale. Utilizzando il Lemma 70 possiamo trasformare una qualsiasi macchina che decide un insieme X in una macchina che ancora decide X ma che alla fine del calcolo cancella il contenuto di tutti i nastri tranne il primo e che sul primo ritrova il risultato e posiziona il puntatore nella prima colonna per ogni nastro.

ii) Supponiamo che F sia una funzione di A^* in se stesso. Se F è Turing calcolabile possiamo decidere se una coppia (w, v) è nel grafico di F calcolando $F(w)$ e testando l'uguaglianza $F(w) = v$. Supponiamo invece che il grafico di F sia Turing decidibile. Esiste un'enumerazione $v_1, v_2, \ldots$ di parole di A^* e una macchina di Turing che passa dalla configurazione iniziale associata a v_i alla configurazione iniziale associata a v_{i+1}. Otterremo una macchina di Turing che calcola F come segue. A partire dalla parola in ingresso w, la macchina scrive $w\Box v_1$ e verifica se (w, v_1) è nel grafico di F. Se lo è, cancella tutto (come sopra) e passa alla configurazione terminale associata v_1. Altrimenti, sostituisce v_1 con v_2 e verifica se (w, v_2) è nel grafico di F, e così via. Il calcolo si fermerà nella configurazione terminale associata a quell'unico v_i tale che (w, v_i) sia grafico di F (la computazione termina poiché F è totale).

□

Se non assumiamo più che le funzioni siano definite ovunque, allora esiste ancora un rapporto tra il concetto di decidibilità e quello di calcolabilità che questa volta si precisa tra la Turing calcolabilità (di funzioni parziali) e la Turing semi-decidibilità:

Proposizione 10.6. Sia F una funzione qualunque (non necessariamente ovunque definita). Quindi F è Turing calcolabile se e solo se il grafico di F è un insieme Turing semi-decidibile.

Dimostrazione. Supponiamo la funzione F Turing calcolabile. L'argomento è lo stesso di sopra. Dovendo decidere se (w, y) è nel grafico di F, calcoliamo $F(w)$; se il calcolo risulta in un valore y', verifichiamo se y' è uguale a y: se sì, accettiamo, altrimenti entriamo in uno stato che provoca un calcolo infinito. L'algoritmo così ottenuto semi-decide il grafico di F: si ferma su (w, y) esattamente se (w, y) è in questo grafico.

Supponiamo invece che la macchina μ decida il grafico di F. Di nuovo impostiamo un'enumerazione Turing calcolabile $y_1, y_2, \ldots$ di tutte le parole di A^*; il problema è che, essendo il grafico di F assunto solo Turing semi-decidibile, non possiamo aspettare l'ipotetica fine della decisione di μ in merito a (w, y_1) per iniziare il calcolo relativo a (w, y_2), seguito da quello (w, y_3) e così via. L'idea è la seguente: eseguiremo un passo di computazione del calcolo di μ a partire da (w, y_1), seguito dalla parte di calcolo che prepara l'input (w, y_2), quindi riprenderemo il calcolo iniziato di (w, y_1) (che si troverà nella seconda configurazione della computazione), il primo passo di computazione a partire da (w, y_2) e la preparazione dell'input (w, y_3), seguito dal terzo passo della computazione di (w, y_1), il secondo per (w, y_2), il primo per (w, y_3) e la preparazione dell'input (w, y_4).

Ora se w è nel dominio di F, deve esistere un indice i in modo tale che (w, y_i) è nel grafico di F, e quindi dopo un certo numero t di passi di calcolo μ a partire da (w, y_i) arriva ad una configurazione accettante, e possiamo produrre il valore corrispondente y_i e fermare l'algoritmo.

Nel caso in cui w non sia nel dominio di F si avrà che per ogni y, la coppia (w, y) non sarà nel grafico di F, e quindi, qualunque sia il valore di t, dopo t passi del calcolo di μ a partire da (w, y) non porteranno a una configurazione finale che accetta e l'algoritmo non si fermerà. In conclusione, l'algoritmo così descritto calcola la funzione F. Per quanto riguarda la fattibilità delle simulazioni successive di un numero crescente di passi di calcolo di μ sulle parole y_1, y_2, ecc. sarà realizzabile con nastri aggiuntivi in numero sufficiente (uno per la parola y, uno per scrivere t in base unaria). □

Grazie alle proposizioni precedenti si vede che la teoria delle funzioni calcolabili e quella degli insiemi decidibili sono equivalenti.

Proposizione 10.7. La funzione composta da due funzioni Turing calcolabili è Turing calcolabile.

Dimostrazione. Siano G e H due funzioni Turing calcolabili. Si supponga che il codominio di G corrisponda al dominio di H (altrimenti il risultato è banale poiché la funzione composta ha dominio vuoto). Supponiamo che la macchina di Turing

$$\mu_1 \colon Q_1 \times A'^{k_1} \to Q_1' \times A'^{k_1} \times \{-1, +1\}^{k_1}$$

calcoli G e la macchina di Turing

$$\mu_2 \colon Q_2 \times A'^{k_2} \to Q_2' \times A'^{k_2} \times \{-1, +1\}^{k_2}$$

calcoli H. Possiamo ipotizzare che μ_1 e μ_2 siano macchine per lo stesso numero di nastri, ovvero $k_1 = k_2$ e che i loro insiemi di stati siano, non contando lo stato q_0 e lo stato q_F, disgiunti $Q_1 \cap Q_2 = \emptyset$. Sia μ_1' la macchina ottenuta sostituendo in μ_1 lo stato q_F^1 con un nuovo stato q_A, e sia μ_2' la macchina ottenuta da μ_2 sostituendo lo stato iniziale con questo stesso stato q_A. Finalmente sia μ' l'unione di μ_1' e μ_2'. Sia il vettore $(w_1, \ldots, w_r) \in (A^*)^r$ nel dominio di G. Sono possibili tre casi:

- $(w_1, \ldots, w_r)$ non è nell'insieme di definizione di G. Quindi il calcolo di μ_1 non finisce, ed è uguale a quello di μ_1', quindi quello di μ' non termina.
- Oppure G è definito in $(w_1, \ldots, w_r)$ ma $G(w_1, \ldots, w_r)$ non è nel dominio di H. Quindi μ_1 passa dalla configurazione terminale, ma μ_1' passa allo stato q_A che coincide con lo stato iniziale di μ_2'. Quindi, per ipotesi, il calcolo di μ_2' dalla configurazione iniziale non si ferma, quindi il calcolo di μ' a partire da $(w_1, \ldots, w_r)$ non termina.
- Supponiamo infine che l'applicazione $H(G(w_1, \ldots, w_r))$ sia definita. La macchina μ_1 computa il risultato dell'applicazione $G(w_1, \ldots, w_r)$ e così fa μ_1', passando dalla configurazione terminale per μ_1 a quella iniziale per μ_2 che, applicata a $G(w_1, \ldots, w_r)$, calcolerà il risultato della composizione $H(G(w_1, \ldots, w_r))$.

Quindi μ' calcola bene la funzione composta di H e G. □

10.2 Funzioni ricorsive

Useremo le seguenti funzione che chiameremo funzioni base indicando con $A = B = (\mathbb{N})_1$, cioè $\mathbb{N}$ in base 1, abbiamo

- Addizione: $S(x, y) = x + y$.
- Predecessore (decremento): $D(x) = \begin{cases} x - 1 & x > 0 \\ 0 & \text{altrimenti.} \end{cases}$
- Schemi per funzioni:

– costanti: $C_{x_0,n}\colon \mathbb{N}^n \to \mathbb{N}$, $(-) \mapsto x_0$;

– proiezione: $P_{i,n}\colon \mathbb{N}^n \to \mathbb{N}$, $(x_1, \ldots, x_n) \mapsto x_i$.

Definizione 10.8. Supponiamo che G e H siano due funzioni rispettivamente da $\mathbb{N}^{r+1}$ a $\mathbb{N}$ e da $\mathbb{N}^{r+2}$ a $\mathbb{N}$. La funzione definita per ricorsione primitiva di base G e passo H è la funzione F da $\mathbb{N}^{r+1}$ a $\mathbb{N}$ tale che

$$F(x_1, \ldots, x_r, m) = \begin{cases} G(x_1, \ldots, x_r) & \text{se } m = 0 \\ H(x_1, \ldots, x_r, m, F(x_1, \ldots, x_r, m-1)) & \text{se } m > 0. \end{cases}$$

Dal punto di vista dei tipi sulle parole: $G\colon (A^*)^r \to B^*$, $H\colon (A^*)^r \times \mathbb{N} \times B^* \to B^*$ e $F\colon (A^*)^r \times \mathbb{N} \to B^*$.

Definizione 10.9. Supponiamo che G sia una funzione di $\mathbb{N}^{r+1}$ in $\mathbb{N}$. La funzione definita per minimalizzazione di G è la funzione F da $\mathbb{N}^r$ in $\mathbb{N}$, che soddisfa la seguente condizione:

$$F(n_1, \ldots, n_r) = k$$

se e solo se

$$\text{per ogni } i < k \text{ si ha che } G(n_1, \ldots, n_r, i) \neq 0 \text{ e } G(n_1, \ldots, n_r, k) = 0.$$

In particolare G è definita (e non nulla) per ogni $i < k$. Con le notazioni sulle parole: $G\colon (A^*)^r \times \mathbb{N} \to \mathbb{N}$ e $F\colon (A^*)^r \to \mathbb{N}$.

Introduciamo una famiglia di funzioni sugli interi tramite la prossima definizione "astratta" (o piuttosto induttiva):

Definizione 10.10. L'insieme delle *funzioni ricorsive* $\mathcal{R}$ è il più piccolo insieme di funzioni su numeri interi

- che contiene la funzione "somma", la funzione "predecessore", le funzioni costanti e le funzioni proiezione,
- e che è chiuso per concatenazione, composizione, ricorsione primitiva e minimalizzazione.

Le funzioni ricorsive sono gli elementi dell'insieme precedente. Gli *insiemi ricorsivi* sono insiemi di numeri interi la cui funzione caratteristica è una funzione ricorsiva.

Espressioni ricorsive e interpretazione

Definizione 10.11. Le espressioni ricorsive di profondità 0 sono S, D, $\mathsf{C}_{x_0,n}$ e $\mathsf{P}_{i,n}$. Per $d \geq 1$, una espressione ricorsiva è di profondità d se è di profondità $d-1$ oppure se può essere scritta per composizione, concatenazione, ricorsione o minimalizzazione di espressioni di profondità $d-1$.

Osservazione 10.12. Dal punto di vista sintattico, la costruzione di tutte le espressioni mostra in modo induttivo che i "costrutti" introdotti sopra possono essere ottenuti come interpretazione delle espressioni ricorsive generate dalla grammatica con simbolo iniziale R le cui regole di produzione sono

$$\mathsf{R} := \mathsf{S} \mid \mathsf{D} \mid \mathsf{C}_{x_0,n} \mid \mathsf{P}_{i,n} \mid \mathsf{COMP}(\mathsf{R},\mathsf{R}) \mid \mathsf{CONCAT}(\mathsf{R},\mathsf{R}) \mid \mathsf{REC}(\mathsf{R},\mathsf{R}) \mid \mathsf{MIN}(\mathsf{R}).$$

Definizione 10.13. La classe delle funzioni ricorsive può essere definita come l'insieme delle funzioni che sono interpretazione di un'espressione ricorsiva:

$$\mathcal{R} := \{[[\mathsf{e}]] \mid \mathsf{e} \in \mathsf{R}\}.$$

Definizione 10.14. L'insieme delle espressioni ricorsive sarà indicato con R. Più precisamente l'interpretazione sarà una funzione

$$[[\ ]] \colon \mathsf{R} \to \left(\bigcup_{r,s} (\mathbb{N}^r \to \mathbb{N}^s) \right),$$

ad esempio se $\mathsf{e} \in \mathsf{R}$ è un'espressione ricorsiva $[[\mathsf{e}]] = F$ nel senso che per ogni $(n_1, \ldots, n_r)$

$$[[\mathsf{e}]](n_1, \ldots, n_r) = F(n_1, \ldots, n_r).$$

È anche possibile avere presentazioni alternative della stessa classe di funzioni: ad esempio si potrebbe cambiare l'interpretazione del simbolo S dalla somma alla funzione successore, $[[\mathsf{S}]](x) := x + 1$, senza cambiare il linguaggio:

$$\mathsf{R} := \mathsf{S} \mid \mathsf{D} \mid \mathsf{C}_{x_0,n} \mid \mathsf{P}_{i,n} \mid \mathsf{COMP}(\mathsf{R},\mathsf{R}) \mid \mathsf{CONCAT}(\mathsf{R},\mathsf{R}) \mid \mathsf{REC}(\mathsf{R},\mathsf{R}) \mid \mathsf{MIN}(\mathsf{R}).$$

Si vede facilmente che la classe delle funzioni che sono interpretazione delle (stesse) espressioni ricorsive non cambia, in quanto la funzione successore si può ottenere dalla somma e dalla funzione costante a 1 per composizione, e la funzione somma si può ottenere come iterazione del successore per ricorsione primitiva di base la funzione identità e il successore come passo di ricorsione. In pratica, denotando con $[[\ ]]_1$ la prima interpretazione e con $[[\ ]]_2$ la seconda, quella che associa a S il successore, abbiamo le seguenti equazioni:

$$[[\mathsf{S}]]_2 = [[\mathsf{COMP}(\mathsf{CONCAT}(\mathsf{P}_{1,1}, \mathsf{C}_{1,1}), \mathsf{S})]]_1 \qquad [[\mathsf{S}]]_1 = [[\mathsf{REC}(\mathsf{P}_{1,1}, \mathsf{COMP}(\mathsf{P}_{3,3}, \mathsf{S}))]]_2$$

Turing calcolabilità delle funzioni base

Lemma 10.15. La funzione somma $S \colon \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ è Turing calcolabile.

Dimostrazione. Usiamo la rappresentazione in base 1. È quindi necessario passare dalla configurazione iniziale associata a $1^{n+1}\square 1^{m+1}$ (rappresentazione della coppia di numeri interi (n, m)) alla configurazione terminale associata alla parola 1^{n+m+1} (rappresentazione dell'intero $n+m$). In altre parole, si tratta di cancellare due 1 alla fine e sostituire il blank centrale con un 1. La macchina in Figura 33 esegue i passi che abbiamo descritto. □

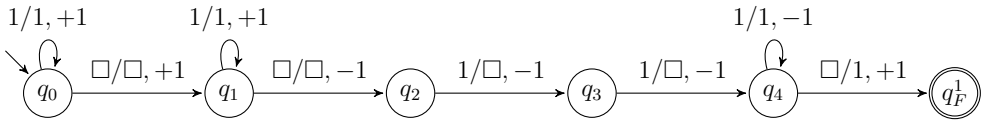


Figura 33: Macchina di Turing che calcola la somma di due interi rappresentati in unario.

Utilizzeremo l'espressione S e la sua interpretazione $[[\mathsf{S}]](n, m) = S(n, m) = n + m$.

Lemma 10.16. La funzione predecessore $D\colon \mathbb{N} \to \mathbb{N}$ che associa all'intero $N + 1$ il valore N è Turing calcolabile.

Dimostrazione. Più semplice del lemma precedente ... □

Utilizzeremo l'espressione D e la sua interpretazione

$$[[\mathsf{D}]](n) = D(n) = \begin{cases} n - 1 & \text{se } n > 0 \\ 0 & \text{altrimenti.} \end{cases}$$

Lemma 10.17. Per ogni numero intero n, e per ogni numero intero x_0, la funzione da $\mathbb{N}^n$ in $\mathbb{N}$ di valore costante uguale a x_0 è Turing calcolabile.

Dimostrazione. Cerca l'n-esimo blank a destra, ritorna cancellando tutti i caratteri fino a trovare il primo blank, quindi si sposta nella casella a destra e scrive la rappresentazione di x_0 e infine torna alla posizione 1. Nella Figura 34 è rappresentata la macchina nel caso $n = 2$ e $x_0 = 2$. □

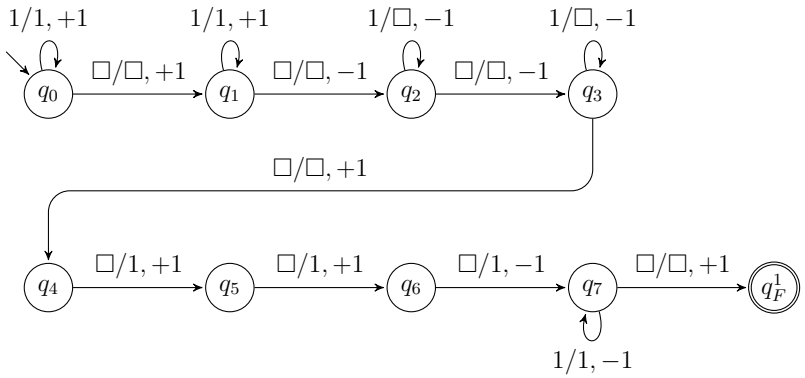


Figura 34: Macchina di Turing che calcola la funzione costante $\mathsf{C}_{2,2}$ ($x_0 = 2$) su due argomenti rappresentati in unario.

Osservazione 10.18. La macchina che calcola $F_1 = [[\mathsf{C}_{m,1}]]$ è diversa da quella che calcola $F_2 = [[\mathsf{C}_{m,2}]]$: l'elenco delle istruzioni necessarie per cancellare un blocco di 1 non è affatto lo stesso di quello che cancella due blocchi di 1. In effetti non possiamo costruire la macchina di Turing che, qualunque sia il numero intero n, calcoli la funzione $[[\mathsf{C}_{m,n}]]$, perché, necessariamente il valore del parametro n deve essere codificato in un modo o nell'altro negli stati

della macchina. L'impossibilità viene quindi dal fatto che n non è un valore limitato, quindi non è possibile codificarlo in un insieme finito di stati.

Tuttavia, possiamo facilmente costruire una macchina che calcola ciascuna delle funzioni $[[\mathsf{C}_{m,n}]]$ per ogni valore di n inferiore a un valore $n_{\max}$. Infatti, una macchina che cancella $n_{\max}$ blocchi di 1 procede allo stesso modo anche quando alcuni blocchi possono essere mancanti. Inoltre, vedremo questo tipo di proprietà molto più in generale più avanti.

D'altra parte potremmo anche scrivere una macchina che, a partire dal valore di n e di qualsiasi tupla, porta alla configurazione terminale associata alla costante m: assumendo il valore di n codificato in rappresentazione unaria, la macchina cancellerà tanti blocchi di 1 quanti ce ne sono nel primo di questi blocchi (che verrà cancellato per ultimo).

Lemma 10.19. Per ogni coppia di interi n ed i con $i \leq n$, la proiezione che associa ad una n-upla di interi l'i-esimo elemento è Turing calcolabile.

Dimostrazione. Ancora una volta, si tratta di cancellare n blocchi di numeri. Durante il passaggio in corrispondenza dell'i-esimo blocco, questo viene copiato sul secondo nastro e quindi, quando la cancellazione di tutti i blocchi è terminata, il puntatore torna alla casella iniziale, e infine il blocco dei numeri del secondo nastro viene copiato sul primo (e cancellato dal secondo). □

Utilizzeremo l'espressione $\mathsf{P}_{i,n}$ e la sua interpretazione $[[\mathsf{P}_{i,n}]](x_1, \ldots, x_n) = x_i$.

Con questa nozione possiamo riassumere i risultati di chiusura sopra stabiliti per funzioni Turing calcolabili nella forma delle seguenti inclusioni:

Proposizione 10.20.

i) Qualsiasi funzione ricorsiva su numeri interi è calcolabile mediante macchina di Turing.

ii) Qualsiasi insieme ricorsivo di numeri interi è decidibile per macchina di Turing.

Esempi di funzioni ricorsive

Lemma 10.21. Le funzioni "prodotto" ed "esponenziale" sono ricorsive.

Dimostrazione. Abbiamo visto sopra che il prodotto può essere definito per ricorsione primitiva a partire dalla somma. Precisamente, se denotiamo con M la funzione "prodotto", abbiamo che soddisfa le seguenti equazioni di ricorsione:

$$M(n,m) := \begin{cases} 0 & \text{se } m = 0 \\ n + M(n, m-1) & \text{se } m > 0 \end{cases}$$

quindi si può applicare la definizione per ricorsione e, ponendo l'espressione ricorsiva

$$\mathsf{MULT} := \mathsf{REC}(\mathsf{C}_{0,1}, \mathsf{COMP}(\mathsf{CONCAT}(\mathsf{P}_{3,3}, \mathsf{P}_{1,3}), \mathsf{S})),$$

si ha che $[[\mathsf{MULT}]] = M \colon \mathbb{N}^2 \to \mathbb{N}$.

In modo analogo la funzione esponenziale $E(n, m) := n^m$ soddisfa l'equazione di ricorsione primitiva:

$$E(n, m) := \begin{cases} 1 & \text{se } m = 0 \\ n \cdot E(n, m-1) & \text{se } m > 0 \end{cases}$$

quindi si può applicare la definizione per ricorsione e, ponendo l'espressione ricorsiva

$$\mathsf{EXP} := \mathsf{REC}(\mathsf{C}_{1,1}, \mathsf{COMP}(\mathsf{CONCAT}(\mathsf{P}_{3,3}, \mathsf{P}_{1,3}), \mathsf{MULT})),$$

si ha che $[[\mathsf{EXP}]] = E \colon \mathbb{N}^2 \to \mathbb{N}$. Quindi otteniamo una definizione di E in termini di sole funzioni ricorsive di base e costrutti mediante sostituzione dell'espressione MULT con l'espressione ricorsiva che la rappresenta. □

Domanda. f Turing calcolabile $\overset{?}{\Longrightarrow} \exists\, \mathsf{e} \in \mathsf{R}$ tale che $[[\mathsf{e}]] = f$.

Per $X \subset \mathbb{N}^r$, la funzione caratteristica di X sia $\mathbb{1}_X$ e sia Δ la funzione per cui $\Delta(x, y) = \sup(x - y, 0)$.

Lemma 10.22. Le funzioni Δ, sup, inf, $\mathbb{1}_{\{0\}}$ e $\mathbb{1}_{\{<\}}$ sono ricorsive.

11 Prerequisiti

Per tutto il documento, la definizione di funzione sarà quella di funzione parziale. Pertanto, data una funzione $f : A \to B$, indicheremo con $\mathrm{dom}(f)$ il dominio della funzione f. Inoltre, se $x \in A \setminus \mathrm{dom}(f)$ (quindi f non è definita in x), allora scriveremo che $f(x) := \perp$.

Inoltre, quando scriveremo *alfabeto* intenderemo un insieme. Useremo la notazione $f^0 := \mathrm{Id}_X$ per ogni funzione $f : X \to X$.

11.1 Definizioni Preliminari

Definizione 11.1 (Funzione caratteristica)**.** La funzione $\mathbf{1}_X$ è la *funzione caratteristica* (o indicatrice) dell'insieme X, definita come

$$\mathbf{1}_X(w) = \begin{cases} 1 & \text{se } w \in X \\ 0 & \text{se } w \notin X. \end{cases}$$

Definizione 11.2 (Partizione)**.** Sia X un insieme. Una *partizione* di X è una famiglia $\{X_i\}_{i \in I}$ di sottoinsiemi di X tali che:

1. $X_i \neq \emptyset$ per ogni $i \in I$ (i sottoinsiemi non sono vuoti);
2. $X_i \cap X_j = \emptyset$ per ogni $i \neq j$ (i sottoinsiemi sono a due a due disgiunti);
3. $\bigcup_{i \in I} X_i = X$ (l'unione di tutti i sottoinsiemi ricopre l'intero insieme X).

11.2 Grafi e Matrici di Adiacenza

Definizione 11.3 (Grafo orientato)**.** Un *grafo orientato* (o diretto) è una coppia $G = (V, E)$, dove V è un insieme i cui elementi sono detti *vertici* (o nodi) ed $E \subseteq V \times V$ è una relazione binaria su V, i cui elementi sono detti *archi* orientati. Un arco che punta da un vertice u a un vertice v è dunque rappresentato esplicitamente dalla coppia ordinata (u, v).

Definizione 11.4 (Matrice di adiacenza)**.** Sia $G = (V, E)$ un grafo orientato finito, con i vertici ordinati come $V = \{v_1, v_2, \ldots, v_n\}$. La *matrice di adiacenza* di G è la matrice M di dimensione $n \times n$ definita da:

$$M_{i,j} = \begin{cases} 1 & \text{se } (v_i, v_j) \in E \\ 0 & \text{se } (v_i, v_j) \notin E \end{cases}$$

per ogni indice $i, j \in \{1, \ldots, n\}$.

Proposizione 11.5. Se M è la matrice di adiacenza per un grafo G, allora $(M^k)_{i,j}$ è il numero di cammini di lunghezza k che connettono i vertici v_i e v_j.

11.3 Grafi Etichettati

Definizione 11.6 (Grafo etichettato)**.** Sia A un insieme, detto *alfabeto*. Un *grafo etichettato* su A è una terna

$$G = (V, E, L),$$

dove $E \subset V \times A \times V$ e

- chiamando con $\pi_{1,3}$ la proiezione sulla prima e terza componente, $(V, \pi_{1,3}(E))$ è un grafo orientato;
- $L : E \to A$ è una funzione, detta *funzione di etichettatura*, che associa ad ogni arco un elemento di A chiamato *etichetta* associata all'arco.

Proposizione 11.7 (Cammini sui grafi etichettati)**.** Sia $G = (V, E, L)$ un grafo etichettato finito con $V = \{v_1, \ldots, v_n\}$, e supponiamo di poter moltiplicare e sommare le etichette A all'interno di un opportuno semianello con zero 0. Definiamo la matrice di adiacenza etichettata (o pesata) M di dimensione $n \times n$ come:

$$M_{i,j} = \begin{cases} L(v_i, a, v_j) & \text{se } (v_i, a, v_j) \in E \\ 0 & \text{altrimenti.} \end{cases}$$

Allora, l'elemento $(M^k)_{i,j}$ della potenza k-esima della matrice M rappresenta la *somma* (nel semianello) dei pesi di tutti i cammini di lunghezza k che connettono il vertice v_i al vertice v_j. (Il peso di un singolo cammino è il prodotto delle etichette degli archi che lo compongono).

11.4 Semianelli e Serie Formali

Per estendere ai grafi etichettati l'interpretazione combinatoria delle potenze successive della matrice di adiacenza, è necessario dotare l'insieme delle etichette di una struttura algebrica con somma e prodotto. Introduciamo qui, in modo autonomo, gli strumenti algebrici necessari: semianelli, matrici a coefficienti in un semianello, serie formali e algebre di Kleene.

Definizione 11.8 (Semigruppo)**.** Un *semigruppo* è una coppia $(S, *)$ dove S è un insieme e

$$* : S \times S \to S$$

è un'operazione binaria (cioè una funzione che ad ogni coppia ordinata $(x, y) \in S \times S$ associa un elemento $x * y \in S$) tale che valga la proprietà associativa:

$$(x * y) * z = x * (y * z) \qquad \text{per ogni } x, y, z \in S.$$

Esempio 11.9. $(\mathbb{N}, +)$ è un semigruppo: la somma di due numeri naturali è un numero naturale ed è associativa. Anche $(\mathbb{N}, \cdot)$ è un semigruppo. Un altro esempio è $(A^*, \cdot\,)$, l'insieme delle stringhe finite su un alfabeto A con l'operazione di concatenazione, che è associativa ma non commutativa se $|A| > 1$.

Definizione 11.10 (Semianello)**.** Un *semianello* è una struttura algebrica

$$(S, +, \cdot)$$

dove S è un insieme e

$$+ : S \times S \to S, \qquad \cdot : S \times S \to S$$

sono due operazioni binarie su S, dette rispettivamente *somma* e *prodotto*, tali che:

- $(S, +)$ è un semigruppo (Definizione 11.8), cioè $+$ è associativa;
- $(S, \cdot)$ è un semigruppo, cioè $\cdot$ è associativa;
- valgono le leggi di distribuzione tra somma e prodotto:

  $$x \cdot (y + z) = (x \cdot y) + (x \cdot z), \qquad (y + z) \cdot x = (y \cdot x) + (z \cdot x)$$

  per ogni $x, y, z \in S$.

Osservazione 11.11. $(\mathbb{N}, +, \cdot)$, con le usuali operazioni di somma e prodotto tra numeri naturali, è un semianello: non è un anello perché $(\mathbb{N}, +)$ non ha elementi inversi (ad esempio non esiste $n \in \mathbb{N}$ tale che $2 + n = 0$). Un secondo esempio, centrale per quanto segue, è il *semianello booleano* $\mathbb{B} = (\{0, 1\}, \vee, \wedge)$, dove $\vee$ è l'operazione di "or" logico (che gioca il ruolo della somma modulo 2) e $\wedge$ è l'operazione di "and" logico (che gioca il ruolo del prodotto modulo 2).

Definizione 11.12 (Semianello additivamente commutativo, zero, identità)**.** Sia $(S, +, \cdot)$ un semianello (Definizione 11.10).

- $(S, +, \cdot)$ si dice *additivamente commutativo* se

  $$x + y = y + x \qquad \text{per ogni } x, y \in S.$$

- Un elemento $0 \in S$ si dice *zero* di $(S, +, \cdot)$ se

  $$x + 0 = 0 + x = x \quad \text{e} \quad x \cdot 0 = 0 \cdot x = 0 \qquad \text{per ogni } x \in S.$$

- Un elemento $1 \in S$ si dice *identità* (o *unità*) di $(S, +, \cdot)$ se

  $$x \cdot 1 = 1 \cdot x = x \qquad \text{per ogni } x \in S.$$

Si noti che, quando esistono, lo zero e l'identità di un semianello sono unici: se 0 e $0'$ fossero entrambi zeri, si avrebbe $0 = 0 + 0' = 0'$; analogamente per l'identità.

Osservazione 11.13. Nel semianello booleano $\mathbb{B} = (\{0, 1\}, \vee, \wedge)$ dell'esempio precedente, 0 è lo zero (l'elemento neutro di $\vee$) e 1 è l'identità (l'elemento neutro di $\wedge$): infatti $x \vee 0 = x$, $x \wedge 0 = 0$ e $x \wedge 1 = x$ per ogni $x \in \{0, 1\}$. Il semianello $\mathbb{B}$ è inoltre additivamente commutativo, poiché $x \vee y = y \vee x$.

Definizione 11.14 (Semianello idempotente)**.** Un semianello $(S, +, \cdot)$ additivamente commutativo, con zero 0 e identità 1 (Definizione 11.12), si dice *idempotente* se l'operazione di somma è idempotente, ossia

$$x + x = x \qquad \text{per ogni } x \in S.$$

Osservazione 11.15. Il semianello booleano $\mathbb{B} = (\{0, 1\}, \vee, \wedge)$ è idempotente: $x \vee x = x$ per ogni $x \in \{0, 1\}$ (ad esempio $1 \vee 1 = 1$). Al contrario, $(\mathbb{N}, +, \cdot)$ non è idempotente, dato che ad esempio $1 + 1 = 2 \neq 1$.

11.5 Matrici su Semianelli

Definizione 11.16 (Matrici su un semianello)**.** Sia S un insieme munito di struttura di semianello commutativo[2] con zero 0 e identità 1, e sia n un intero positivo. Si definisce

$$M_n(S) := \{A : \{1, \dots, n\} \times \{1, \dots, n\} \to S\}$$

l'insieme di tutte le matrici $n \times n$ a coefficienti in S (cioè le funzioni che ad ogni coppia di indici (i, j) associano un elemento $A_{ij} \in S$), munito delle operazioni:

$$(A + B)_{ij} := A_{ij} + B_{ij} \qquad \text{(somma elemento per elemento)},$$

$$(A \cdot B)_{ij} := \sum_{k=1}^{n} A_{ik} \cdot B_{kj} \qquad \text{(prodotto righe per colonne)}.$$

Con tali operazioni $(M_n(S), +, \cdot)$ è a sua volta un semianello (le proprietà associative e distributive si verificano direttamente dalle corrispondenti proprietà in S), ed è in particolare un semigruppo additivamente commutativo se S lo è. La matrice $0_n \in M_n(S)$ con tutti gli elementi uguali a 0 è l'elemento neutro per l'operazione additiva, e la matrice identità

$$I_n := \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \in M_n(S)$$

è l'identità del semianello $M_n(S)$.

Se S contiene più di un elemento ed $n > 1$, allora $M_n(S)$ non può essere moltiplicativamente commutativo (come nel caso classico delle matrici su un anello, è sufficiente esibire due matrici A, B con $AB \neq BA$).

Osservazione 11.17. Per $n = 2$ ed $S = \mathbb{B}$ il semianello booleano, si ha ad esempio

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \in M_2(\mathbb{B}),$$

con prodotto (righe per colonne, usando $\vee, \wedge$ al posto di $+, \cdot$)

$$A \cdot B = \begin{pmatrix} (1 \wedge 0) \vee (0 \wedge 1) & (1 \wedge 1) \vee (0 \wedge 0) \\ (1 \wedge 0) \vee (1 \wedge 1) & (1 \wedge 1) \vee (1 \wedge 0) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Per $A \in M_n(S)$ e $i, j \in \{1, \dots, n\}$, ricordiamo che A_{ij} denota l'elemento di A alla riga i-esima e alla colonna j-esima.

Definizione 11.18 (Trasposta)**.** La *trasposta* di una matrice $A \in M_n(S)$ è la matrice $A^t \in M_n(S)$ definita da

$$(A^t)_{ij} := A_{ji} \qquad \text{per ogni } i, j \in \{1, \dots, n\}.$$

Per ogni coppia di matrici $A, B \in M_n(S)$ valgono le identità

$$(A^t)^t = A, \qquad (A + B)^t = A^t + B^t, \qquad (AB)^t = B^t A^t.$$

[2] Con *commutativo*, quando non altrimenti specificato, si intende qui additivamente commutativo (Definizione 11.12).

11.6 Monoidi, Serie Formali e Algebre di Kleene

11.6.1 Monoidi

Definizione 11.19 (Monoide)**.** Un *monoide* è una terna $(M, \cdot, 1)$ dove M è un insieme,

$$\cdot : M \times M \to M$$

è un'operazione binaria associativa (cioè $(M, \cdot)$ è un semigruppo, Definizione 11.8), e $1 \in M$ è un elemento tale che

$$1 \cdot m = m \cdot 1 = m \qquad \text{per ogni } m \in M.$$

Definizione 11.20. Una funzione $\omega\colon B \to A^*$ è *prefix-free* se $\omega(b) \neq \varepsilon$ per ogni $b \in B$ e, per $b \neq b'$, la parola $\omega(b)$ non è un prefisso di $\omega(b')$ (in particolare ω è iniettiva sulle lettere). Analogamente, ω è *suffix-free* se vale la stessa condizione con "suffisso" al posto di "prefisso".

Osservazione 11.21. Per la proprietà universale dei monoidi liberi, ogni funzione $\omega\colon B \to A^*$ definita sui generatori si estende in modo unico a un omomorfismo $\omega\colon B^* \to A^*$, ponendo

$$\omega(b_1 \cdots b_n) = \omega(b_1) \cdots \omega(b_n), \qquad \omega(\varepsilon) = \varepsilon.$$

In generale l'iniettività di ω sui generatori non basta a garantire quella dell'omomorfismo esteso: ad esempio, se $B = \{b, c\}$ e $A = \{0\}$, la scelta $\omega(b) = 0$, $\omega(c) = 00$ è iniettiva sulle lettere, ma $\omega(bb) = \omega(c)$.

Osservazione 11.22. Valgono i seguenti fatti.

1. Se $\omega\colon B \to A^*$ è prefix-free oppure suffix-free, allora l'omomorfismo esteso $\omega\colon B^* \to A^*$ è iniettivo, quindi è una codifica.
2. Se $|\omega(b)| = d \geq 1$ per ogni $b \in B$ e ω è iniettiva sulle lettere, allora ω è sia prefix-free sia suffix-free, quindi l'omomorfismo esteso è una codifica. Questo è il caso delle codifiche a blocchi usate nel Lemma 9.23.
3. Le condizioni prefix-free e suffix-free sono sufficienti ma non necessarie: l'iniettività su B^* equivale al fatto che $\omega(B)$ sia un codice (decifrabilità unica), e esistono codici né prefix-free né suffix-free.

11.6.2 Serie formali

Definizione 11.23 (Serie formali)**.** Dato un monoide $(M, \cdot, 1)$ (Definizione 11.19) e un semianello $\mathbb{K}$, si consideri l'insieme delle *serie formali* a coefficienti in $\mathbb{K}$ sul monoide M:

$$\mathbb{K}[M] := \left\{ \sum_{m \in M} \alpha_m \, m \;\middle|\; \alpha_m \in \mathbb{K} \text{ per ogni } m \in M \right\},$$

cioè l'insieme delle funzioni $\alpha : M \to \mathbb{K}$, scritte formalmente come somme $\sum_{m \in M} \alpha_m m$ dove $\alpha_m := \alpha(m)$.

Su $\mathbb{K}[M]$ resta definita un'operazione di somma, elemento per elemento sui coefficienti,

$$\sum_{m\in M} \alpha_m m \;+\; \sum_{m\in M} \beta_m m \;:=\; \sum_{m\in M} (\alpha_m + \beta_m)\, m,$$

e un'operazione di prodotto (detta *prodotto di convoluzione*)

$$\left(\sum_{m'\in M} \alpha_{m'}\, m'\right)\left(\sum_{m''\in M} \beta_{m''}\, m''\right) = \sum_{m\in M} \left(\sum_{\substack{m',m''\in M\\ m'm''=m}} \alpha_{m'}\beta_{m''}\right) m.$$

Si noti che nella definizione di prodotto tra serie formali compaiono due nozioni distinte di prodotto:

- il prodotto tra coefficienti $\alpha_{m'}\beta_{m''}$, che è il prodotto del semianello $\mathbb{K}$ (Definizione 11.10);
- il prodotto tra elementi del monoide $m'm''$, che è il prodotto del monoide M (Definizione 11.19).

Con queste due operazioni, $(\mathbb{K}[M], +, \cdot)$ è a sua volta un semianello.

Definizione 11.24 (Ordine indotto da un semianello idempotente)**.** Sia $(S, +, \cdot)$ un semianello idempotente (Definizione 11.14). Si definisce su S la relazione binaria $\leq \subseteq S \times S$ data da

$$a \leq b \quad \text{se e solo se} \quad a + b = b.$$

Si verifica facilmente che $\leq$ è un ordine parziale, ossia soddisfa:

- *riflessività*: $a \leq a$ per ogni $a \in S$ (segue da $a + a = a$, cioè dall'idempotenza);
- *antisimmetria*: se $a \leq b$ e $b \leq a$ allora $a = b$ (da $a + b = b$ e $b + a = a$, usando la commutatività additiva, si ottiene $a = b + a = a + b = b$);
- *transitività*: se $a \leq b$ e $b \leq c$ allora $a \leq c$ (da $a + b = b$ e $b + c = c$ segue $a + c = a + (b + c) = (a + b) + c = b + c = c$).

Definizione 11.25 (Estremo superiore)**.** Sia $(S, \leq)$ un insieme parzialmente ordinato (ad esempio quello indotto come in Definizione 11.24) e sia $T \subseteq S$. Si dice che $y \in S$ è l'*estremo superiore* di T, e si scrive $y = \sup T$, se e soltanto se:

1. $x \leq y$ per ogni $x \in T$ (cioè y è un maggiorante di T);
2. per ogni $y' \in S$ che verifica la proprietà al punto (1), si ha $y \leq y'$ (cioè y è il più piccolo dei maggioranti).

Per un insieme finito, l'estremo superiore è la somma degli elementi:

$$\sup\{a_1, a_2\} = a_1 + a_2, \qquad \sup\{a_1, \ldots, a_n\} = a_1 + \cdots + a_n.$$

(Questo segue induttivamente dalla definizione di $\leq$: $a_1 \leq a_1 + a_2$ poiché $a_1 + (a_1 + a_2) = (a_1 + a_1) + a_2 = a_1 + a_2$ per idempotenza, e analogamente per a_2; la minimalità segue dalla definizione di $+$ come somma nel semianello.) Si noti che l'estremo superiore di un insieme infinito potrebbe non esistere in generale.

11.6.3 Algebra di Kleene

Definizione 11.26 (Algebra di Kleene)**.** L'algebra delle serie formali $\mathbb{K}[M]$ (Definizione 11.23) su un semianello idempotente $\mathbb{K}$ si dice *algebra di Kleene* se ad ogni elemento $a \in \mathbb{K}$ è associato un elemento $a^* \in \mathbb{K}$, detto *chiusura di Kleene* di a, tale che

$$a\, b^*\, c = \sup_{n \geq 0} a\, b^n\, c, \qquad \text{dove } b^n = \begin{cases} 1 & \text{se } n = 0 \\ b \cdot b^{n-1} & \text{se } n \geq 1, \end{cases} \tag{5}$$

per ogni $a, b, c \in \mathbb{K}$, con il sup preso rispetto all'ordine parziale di Definizione 11.24.

L'equazione (5) asserisce, in particolare, la proprietà di chiusura mediante l'operatore $*$: postula l'esistenza dell'estremo superiore per famiglie infinite della forma $\{a\, b^n\, c\}_{n \geq 0}$, anche quando l'esistenza del sup non è garantita in generale per un semianello idempotente qualsiasi.

12 Wolfram

12.1 Introduzione

Osservazione 12.1. Nel linguaggio Wolfram non ci sono tipi: l'unico tipo è il tipo *funzione*. Non c'è memoria, ma solo *stati* (come per gli automi). Si possono aggiungere stati e/o regole (queste ultime all'interno del "kernel" di Wolfram).

Definizione 12.2. Una *espressione* è un oggetto della forma

$$E[E_1, \dots, E_n]$$

dove, a loro volta, gli E_i sono espressioni.

Definizione 12.3. Data una funzione f e un'espressione $E[E_1, \dots, E_n]$, si può costruire

$$\texttt{Map}\,[f, E[E_1, \dots, E_n]],$$

che applica f a ciascun argomento E_i, mantenendo la testa E.

Osservazione 12.4. Ad esempio,

$$\texttt{Map}\,[f, \{a_1, \dots, a_n\}] = \texttt{Map}\,[f, \texttt{List}\,[a_1, \dots, a_n]] = \texttt{List}\,[f[a_1], \dots, f[a_n]].$$

Definizione 12.5. Ci sono due modi per introdurre regole nel kernel.

1. *Assegnazione immediata* (`=`): il membro destro viene *valutato* e la regola di sostituzione così ottenuta viene inserita per tutte le espressioni che hanno la stessa forma:

   ```
   f[x_] = x + 2*a
   ```

2. *Assegnazione ritardata* (`:=`): il membro destro *non* viene valutato, ma si memorizza soltanto la regola, che resta scritta così com'è:

   ```
   f[x_] := x + 2*a
   ```

Osservazione 12.6. Nel secondo caso la valutazione avviene solo quando si chiama, ad esempio, `f[3]` : in quel momento vengono ricalcolate tutte le operazioni usando l'ultimo valore assegnato anche a tutti i parametri presenti nella regola.

Osservazione 12.7. Esempio di codice:

```
In[1]:=  f[x_] = 2x + a
Out[1]=  2x + a
In[2]:=  f[3]
Out[2]=  6 + a
In[3]:=  a = 2
Out[3]=  2
In[4]:=  f[3]
Out[4]=  8
```

Con `=` il membro destro è stato valutato subito (con a ancora indeterminato), mentre a viene sostituito solo alla chiamata successiva.

Osservazione 12.8. Per vedere le regole definite su un simbolo f basta digitare

```
?f
```

Definizione 12.9. Con `List[1,2,3]` (o `{1,2,3}`) si crea l'insieme ordinato $\{1, 2, 3\}$.

Definizione 12.10. Il comando

```
Apply[k, {1,2,3}]
```

sostituisce la testa `List` con k, dando come output

$$k[1, 2, 3].$$

Osservazione 12.11. `Plus` fa la somma degli elementi, quindi

`Apply[Plus, {1,2,3}]` $\longmapsto$ 6.

Cambiando `Plus` in `Times` si ottiene la produttoria:

`Apply[Times, {1,2,3}]` $\longmapsto$ 6.

Osservazione 12.12. Per eseguire più istruzioni in un certo ordine, le si separa con un "`;`". Il punto e virgola sopprime anche l'output dell'istruzione che precede.

Osservazione 12.13. Esempio di codice:

```
lista = Range[0, 1000]; Map[h, lista]
```

Il comando `Range` crea un'insieme ordinato (in questo caso da 0 a 1000). L'output è la lista $\{h[0], h[1], \ldots, h[1000]\}$.

Definizione 12.14. Per estrarre elementi da una lista (o più in generale i rami dell'albero di un'espressione) si usa il comando `Part`, comunemente abbreviato con le doppie parentesi quadre `[[ ]]`. Ad esempio, se `lista = \{a, b, c\}`, il comando `lista[[2]]` restituisce `b`. È possibile usarlo in modo annidato: `matrice[[1, 2]]` estrae il secondo elemento della prima riga. In `TM-lib.m`, `automa[[1]]` estrae la tabella delle transizioni.

Definizione 12.15. In Wolfram è possibile definire *funzioni pure* (o anonime) senza assegnare loro un nome. Si utilizza il simbolo `\#` per indicare l'argomento (o gli argomenti, es. `\#1, \#2`) e il simbolo `\&` alla fine per indicare la chiusura della funzione. Ad esempio:

`Map[\#^2 \&, \{1, 2, 3\}]` $\longmapsto$ $\{1, 4, 9\}$.

Questo costrutto è estremamente comune per applicare trasformazioni rapide all'interno di un `Map`.

Definizione 12.16. Oltre all'assegnazione globale (`=`), esistono le *regole di trasformazione* locali, create con `->` (`Rule`). Per applicare una o più regole a un'espressione si usa l'operatore di sostituzione `/.` (`ReplaceAll`). Ad esempio:

`x^2 + y /. x -> 3` $\longmapsto$ $9 + y$.

Negli automi, le singole transizioni sono modellate proprio come liste di regole del tipo `\{stato, simbolo\} -> nuovo\_stato`.

Osservazione 12.17. Hai già visto che il comando `Apply[f, espressione]` sostituisce la testa dell'espressione con `f`. Esiste una forma sintattica abbreviata e molto diffusa per questa operazione, che usa il simbolo `@@`. Pertanto:

`Apply[Plus, \{1, 2, 3\}]` è equivalente a `Plus @@ \{1, 2, 3\}`.

Nel codice della libreria, ad esempio, si usa `Union @@ ...` per estrarre e unire in un unico insieme gli elementi di una lista di liste.

Definizione 12.18. In Wolfram Language, il costrutto `Module` viene utilizzato per definire variabili locali. La sintassi generale è:

`Module[\{var1, var2, \dots\}, corpo\_della\_funzione]`

Questo ambiente garantisce che le variabili dichiarate nella lista iniziale (ad esempio `var1`, `var2`) abbiano visibilità solo all'interno del modulo, evitando che vadano a sovrascrivere o interferire con variabili globali omonime definite altrove nel kernel.

Osservazione 12.19. Il blocco `Module` raggruppa più istruzioni separate da punto e virgola (`;`) e restituisce come output il valore dell'ultima espressione valutata. Ad esempio:

```
calcola[x_] := Module[{y, z},
  y = x^2;
  z = y + 1;
  z * 2
]
```

In questo caso, `y` e `z` sono variabili temporanee locali. L'output della chiamata `calcola[3]` sarà il risultato dell'ultima operazione, ovvero `20`.

12.2 Entropia

Definizione 12.20. Entropia ...

Definizione 12.21. Sia $(\Omega, \mathcal{F}, \mathbb{P})$ uno spazio di probabilità e sia $\mathcal{X}$ un insieme finito (l'alfabeto), munito della σ-algebra $2^{\mathcal{X}}$. Una variabile aleatoria è una funzione misurabile $X\colon \Omega \to \mathcal{X}$. La sua legge è la misura di probabilità $\mathbb{P}_X = \mathbb{P} \circ X^{-1}$ su $\mathcal{X}$, e si pone

$$p(x) = \mathbb{P}_X(\{x\}) = \mathbb{P}(X = x), \qquad x \in \mathcal{X}.$$

Definizione 12.22. Il *contenuto informativo* (grezzo) di un esito $x \in \mathcal{X}$ con $p(x) > 0$ è

$$h(x) = \log \frac{1}{p(x)} = -\log p(x).$$

Con il logaritmo in base 2 l'unità di misura è il bit. Più un esito è improbabile, maggiore è la sua informazione: $h(x) = 0$ se e solo se $p(x) = 1$.

Definizione 12.23. Date due variabili aleatorie $X\colon \Omega \to \mathcal{X}$ e $Y\colon \Omega \to \mathcal{Y}$, la loro legge congiunta è $p(x, y) = \mathbb{P}(X = x,\, Y = y)$. Il contenuto informativo della coppia è

$$h(x, y) = \log \frac{1}{p(x, y)}.$$

Definizione 12.24. Le variabili X e Y sono *indipendenti* se $\sigma(X)$ e $\sigma(Y)$ sono indipendenti, cioè $\mathbb{P}(X \in A,\, Y \in B) = \mathbb{P}(X \in A)\,\mathbb{P}(Y \in B)$ per ogni $A \subseteq \mathcal{X}$ e $B \subseteq \mathcal{Y}$. Poiché gli alfabeti sono finiti, ciò equivale a

$$p(x, y) = p(x)\,p(y) \qquad \forall\, x \in \mathcal{X},\ y \in \mathcal{Y}.$$

Osservazione 12.25. Se X e Y sono indipendenti, allora l'informazione della coppia è la somma delle informazioni:

$$h(x, y) = \log \frac{1}{p(x)\,p(y)} = \log \frac{1}{p(x)} + \log \frac{1}{p(y)} = h(x) + h(y).$$

Questa è la ragione per cui si usa il logaritmo: trasforma il prodotto delle probabilità (indipendenza) in una somma di informazioni.

Definizione 12.26. L'*entropia* di X è il valore atteso del contenuto informativo:

$$H(X) = \mathbb{E}\big[h(X)\big] = \int_\Omega h\big(X(\omega)\big)\, d\mathbb{P}(\omega).$$

Per il teorema di cambio di variabili rispetto alla legge $\mathbb{P}_X$ si ottiene

$$H(X) = \int_{\mathcal{X}} h(x)\, d\mathbb{P}_X(x) = \sum_{x\in\mathcal{X}} p(x)\, h(x) = \sum_{x\in\mathcal{X}} p(x) \log \frac{1}{p(x)},$$

con la convenzione $0 \log \frac{1}{0} = 0$, che permette di sommare anche sugli x con $p(x) = 0$.

Osservazione 12.27. L'entropia misura l'informazione media, o equivalentemente l'incertezza, associata a X. Dipende solo dalla legge $\mathbb{P}_X$ e non dai valori assunti da X. Vale sempre $H(X) \geq 0$, poiché $h(x) \geq 0$ quando $p(x) > 0$. Inoltre, essendo $\mathcal{X}$ finito, $h(X)$ è una funzione semplice e quindi integrabile, dunque $H(X) < \infty$.

Osservazione 12.28. Se X e Y sono indipendenti, $H(X,Y) = H(X) + H(Y)$. Infatti, per la linearità del valore atteso e l'additività di h,

$$H(X,Y) = \mathbb{E}\big[h(X,Y)\big] = \mathbb{E}\big[h(X) + h(Y)\big] = \mathbb{E}\big[h(X)\big] + \mathbb{E}\big[h(Y)\big] = H(X) + H(Y).$$

Osservazione 12.29. Esempio di calcolo dell'entropia (in bit) di una distribuzione $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$:

```
p = {1/2, 1/4, 1/4};
Total[p * Log[2, 1/p]]
```

L'output è $\frac{3}{2}$, infatti $\frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{4}\cdot 2 = \frac{3}{2}$.

Definizione 12.30. Appendice: fondamenti di probabilità. Uno *spazio di probabilità* è una terna $(\Omega, \mathcal{F}, \mathbb{P})$ dove:

- $\mathcal{F} \subseteq 2^\Omega$ è una σ-algebra, cioè $\Omega \in \mathcal{F}$, è chiusa per complementare e per unioni numerabili;
- $\mathbb{P}\colon \mathcal{F} \to [0,1]$ è una misura con $\mathbb{P}(\Omega) = 1$, cioè è σ-additiva sugli elementi disgiunti di $\mathcal{F}$.

Una funzione $X\colon (\Omega,\mathcal{F}) \to (E,\mathcal{E})$ è *misurabile* se $X^{-1}(A) \in \mathcal{F}$ per ogni $A \in \mathcal{E}$. La *legge* di X è $\mathbb{P}_X(A) = \mathbb{P}(X^{-1}(A))$ per $A \in \mathcal{E}$.

Definizione 12.31. Sia $g\colon \Omega \to \mathbb{R}$ misurabile. Il *valore atteso* di g è l'integrale di Lebesgue

$$\mathbb{E}[g] = \int_\Omega g\, d\mathbb{P},$$

definito prima per le funzioni semplici $g = \sum_i c_i \mathbb{1}_{A_i}$ come $\sum_i c_i\, \mathbb{P}(A_i)$, poi per $g \geq 0$ come estremo superiore sulle funzioni semplici $0 \leq s \leq g$, e infine per g integrabile come $\mathbb{E}[g^+] - \mathbb{E}[g^-]$.

Osservazione 12.32. Vale il *cambio di variabili*: se $X\colon \Omega \to E$ è misurabile e $f\colon E \to \mathbb{R}$ è misurabile e integrabile rispetto a $\mathbb{P}_X$, allora

$$\mathbb{E}\big[f(X)\big] = \int_\Omega f \circ X \, d\mathbb{P} = \int_E f \, d\mathbb{P}_X.$$

Se E è finito o numerabile, l'ultimo integrale si riduce alla somma $\sum_{x\in E} f(x)\,\mathbb{P}_X(\{x\})$. Questa è la formula usata per scrivere $H(X) = \sum_x p(x)h(x)$.

Osservazione 12.33. Calcoliamo `Integrate[x^2 Sin[x], x]` e vediamo il risultato:

$$\int x^2 \sin x \, dx = (2 - x^2)\cos x + 2x \sin x + C.$$

12.3 Libreria TM-lib

Definizione 12.34. Per implementare gli automi in Wolfram si usa il pacchetto `TM-lib.ma`. Si procede in tre passi.

1. Assegnare la directory corrente, cioè la cartella che contiene il pacchetto: `SetDirectory["/percorso/d`
2. Caricare la libreria: `Get["TM-lib.ma"]`
3. Usare le funzioni implementate dalla libreria (elencate sotto).

Osservazione 12.35. Esempio di codice per i primi due passi:

```
SetDirectory["/percorso/della/cartella"]
Get["TM-lib.ma"]
```

Definizione 12.36. La libreria implementa le seguenti funzioni.

- `ShowConfiguration[c_]` visualizza la configurazione (di una macchina di Turing).
- `ManipulateComputation[mu_, init_]` visualizza la computazione di una macchina di Turing.
- `InitAutomaton[automaton_, w_]` inizializza l'automa, cioè restituisce la configurazione iniziale associata alla parola w.
- `DFA2TM[automaton_]` trasforma un automa nella macchina di Turing equivalente.
- `States[automaton_]` restituisce la lista degli stati dell'automa.
- `ProductFinalStates[A1_, A2_]` restituisce l'automa prodotto a partire da due automi (composizione parallela).

- `TuringMachineGraph[mu_]` restituisce il grafo della macchina di Turing.

Osservazione 12.37. Nelle firme delle funzioni il carattere `_` indica un *pattern*, cioè un parametro formale. Quando si chiama la funzione si scrive solo l'argomento, senza il trattino basso.

12.4 Automi

Definizione 12.38. Un automa si specifica con tre oggetti:

- la lista `T` delle transizioni, ciascuna della forma `{stato, simbolo} -> nuovo_stato`;
- lo stato iniziale `q0`;
- l'insieme `F` degli stati finali.

L'automa è la terna `{T, q0, F}`.

Osservazione 12.39. Esempio di automa con due stati, `pari` e `dispari`, sull'alfabeto $\{0, 1\}$. Lo stato cambia a ogni simbolo 1 letto e resta invariato a ogni simbolo 0, quindi l'automa accetta le parole con un numero pari di 1.

```
T = {
  {pari, 0} -> pari,
  {dispari, 0} -> dispari,
  {pari, 1} -> dispari,
  {dispari, 1} -> pari
  }
q0 = pari
F = {pari}
automa = {T, q0, F}
```

Per mostrare l'esecuzione:

```
init = InitAutomaton[
    automa2, {0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1}
    ] ;
mu = DFA2TM[automa2] ;
TuringMachineGraph[mu]
ManipulateComputation[mu, init]
```

Osservazione 12.40. Una volta definito `automa`, lo si può inizializzare su una parola w con `InitAutomaton[automa, w]`, che restituisce la configurazione iniziale. Con `DFA2TM[automa]`

si ottiene la macchina di Turing equivalente, di cui si può visualizzare la computazione con `ManipulateComputation` e il grafo con `TuringMachineGraph`.

Osservazione 12.41. Composizione di automi: Per studiare sistemi complessi o l'intersezione di linguaggi regolari, la libreria mette a disposizione il prodotto cartesiano tramite le funzioni `ProductTable[A1, A2]` e `ProductFinalStates[A1, A2]`. La prima genera le transizioni incrociate combinando gli stati con il comando `Tuples`, mentre la seconda definisce gli stati finali dell'automa prodotto.

Definizione 12.42. La funzione di visualizzazione interattiva

```
ManipulateComputation[mu, init]
```

combina il motore di animazione di Wolfram (`Manipulate`) con la generazione sequenziale delle configurazioni della macchina di Turing, permettendo di scorrere passo-passo l'intera computazione fino al limite di passi predefinito (`maxsteps`).

13 Esercizi

13.1 Automi

Esercizio 13.1.1. Fissiamo l'alfabeto binario $A = \{0, 1\}$, dimostrare che l'insieme

$$X = \{w0 \mid w \in A^*\}$$

è decidibile per automa a stati finiti.

Soluzione esercizio 13.1.1. Si richiede di mostrare un automa il cui algoritmo associato accetta le parole che terminano con uno zero. È dunque sufficiente memorizzare nello stato l'ultimo simbolo letto e alla fine della lettura della parola decidere in base allo stato: poniamo $Q = \{q_0, q_1\}$, e definiamo

$$T(q, a) := \begin{cases} q_0 & \text{se } a = 0, \\ q_1 & \text{se } a = 1, \end{cases}$$

prendiamo q_0 come stato iniziale e $F = \{q_0\}$ come insieme degli stati finali accettanti. . Sia

$$X = \{w0 \mid w \in A^*\}$$

l'insieme delle parole sull'alfabeto binario che terminano con il simbolo 0.

Costruiamo un automa finito che decide X. Consideriamo l'insieme degli stati

$$Q = \{q_0, q_1\},$$

dove intuitivamente:

- q_0 rappresenta la situazione in cui l'ultimo simbolo letto è 0;
- q_1 rappresenta la situazione in cui l'ultimo simbolo letto è 1.

Definiamo la funzione di transizione

$$T : Q \times A \to Q$$

come

$$T(q, a) = \begin{cases} q_0 & \text{se } a = 0, \\ q_1 & \text{se } a = 1. \end{cases}$$

Scegliamo come stato iniziale

$$q_{\text{init}} = q_0$$

e come insieme degli stati accettanti

$$F = \{q_0\}.$$

Consideriamo ora l'algoritmo associato all'automa. Sia

$$w = a_1 a_2 \cdots a_n \in A^*.$$

La configurazione iniziale è

$$\epsilon(w) = (w, 1, q_0).$$

Dopo k transizioni, per induzione su k, la configurazione raggiunta è

$$(w, k+1, q_k),$$

dove

$$q_k = \begin{cases} q_0 & \text{se } a_k = 0, \\ q_1 & \text{se } a_k = 1. \end{cases}$$

Infatti, ad ogni passo la funzione di transizione sostituisce lo stato corrente con lo stato corrispondente all'ultimo carattere letto.

Dopo aver letto tutta la parola w, la computazione termina nella configurazione

$$(w, n+1, q_n).$$

La funzione di uscita restituisce quindi

$$\delta(w, n+1, q_n) = \begin{cases} 1 & \text{se } q_n \in F, \\ 0 & \text{se } q_n \notin F. \end{cases}$$

Poiché

$$F = \{q_0\},$$

si ha

$$\delta(w, n+1, q_n) = \begin{cases} 1 & \text{se l'ultimo simbolo di } w \text{ è } 0, \\ 0 & \text{se l'ultimo simbolo di } w \text{ è } 1. \end{cases}$$

Quindi

$$f_{\mathcal{A}}(w) = \begin{cases} 1 & \text{se } w \in X, \\ 0 & \text{se } w \notin X. \end{cases}$$

Pertanto

$$f_{\mathcal{A}} = \mathbf{1}_X$$

e quindi l'algoritmo associato all'automa decide X.

Concludiamo che

$$X = \{w0 \mid w \in A^*\}$$

è DFA-decidibile.

Esercizio 13.1.2. Si consideri l'alfabeto $A = \{0, 1\}$ e sia $B = \{1\}$ la restrizione all'insieme che contiene il solo simbolo 1; si considerino l'insieme delle parole di alfabeto A di lunghezza pari:

$$X_1 = \{w \in A^* \mid |w| = 2n,\ n \geq 0\}$$

e l'insieme delle parole di alfabeto A che contengono un numero pari di 1:

$$X_2 = \{w \in A^* \mid |w_{|B}| = 2k,\ k \geq 0\}$$

qui si è utilizzata la notazione $w_{|B}$ per indicare la parola w ristretta ai soli elementi di alfabeto B. Costruire l'automa che decide $X_1 + X_2$.

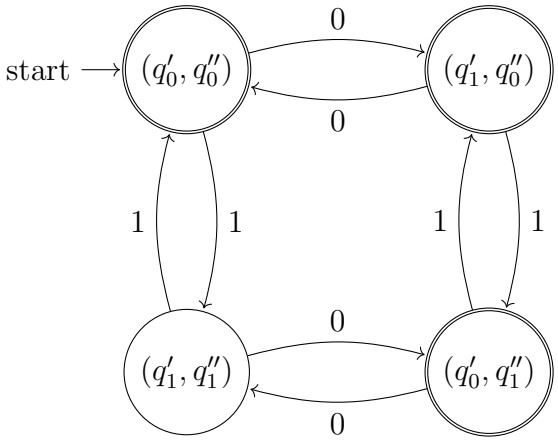
Soluzione esercizio 13.1.2. Utilizzando il teorema 4.3, basta esibire gli automi che decidono X_1 e X_2. L'automa che decide X_1 è quello dell'esempio 2.6 (il grafo corrsipondente e' nella figura 1). Definiamo ora l'automa che decide X_2. Definiamo $Q_2 = \{q_0, q_1\}$ dove q_0 e/' sia stato iniziale che stato accettante, mentre q_1 e/' uno stato non accettante. E definiamo la funzione di transizione T_2 come segue:

$$\begin{array}{rccc} T_2: & Q_2 \times A & \to & Q_2 \\ & (q_0, 0) & \mapsto & q_0 \\ & (q_0, 1) & \mapsto & q_1 \\ & (q_1, 0) & \mapsto & q_1 \\ & (q_1, 1) & \mapsto & q_0 \end{array}$$

In questo modo, l'automa che decide X_2 accetta le parole che contengono un numero pari di 1 (si guardi il grafico dell'automa nella figura 2). A questo punto, utilizzando la costruzione effettuata nella dimostrazione del teorema 4.3 otteniamo che l'automache decide $X_1 + X_2$ e/' l'automa

$$\begin{cases} Q := Q_1 \times Q_2, \\ q_0 := (q_0', q_0''), \\ F := (F_1 \times Q_2) \cup (Q_1 \times F_2), \\ T((q_1, q_2), a) = (T_1(q_1, a), T_2(q_2, a)) \end{cases}$$

dove con un apice indichiamo gli stati di dell'automa che decide X_1 e con due apici gli stati dell'automa che decide X_2. Per completezza, presentiamo il grafo dell'automa che decide $X_1 + X_2$ (che puo/' esssere disegnato a partire dall'automa appena definito oppure usando i grafi degli automi che decidono X_1 e X_2).



Esercizio 13.1.3. Dato l'automa a stati finiti deterministico $\mathcal{A}$ in Figura 35:

1. calcolare l'espressione regolare associata all'automa $\mathcal{A}$;
2. a partire dall'espressione regolare trovata al punto precedente, fornire un automa non-deterministico equivalente ad $\mathcal{A}$.

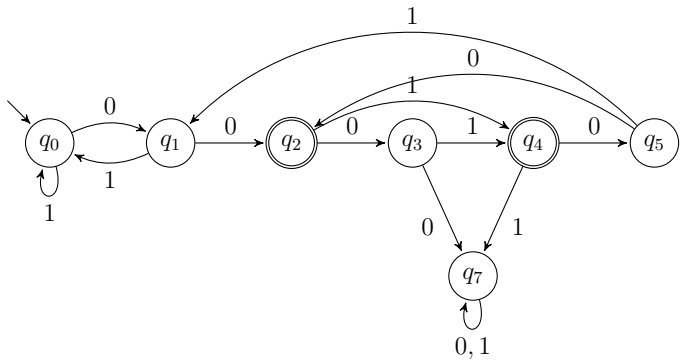


Figura 35: L'automa $\mathcal{A}$ dell'Esercizio 23.

Soluzione esercizio 13.1.3. Usiamo l'algoritmo di eliminazione degli stati. Gli stati da eliminare sono, nell'ordine, $q_0, q_1, q_7, q_3, q_5, q_4$ (e infine q_2).

1. (Setup e Merge) Aggiungiamo un nuovo stato iniziale q_0^* con una $\mathbb{1}$-mossa verso q_0 e un nuovo stato finale q_F^* con una $\mathbb{1}$-mossa da ciascuno degli stati finali q_2 e q_4; gli stati q_2, q_4 non sono più finali. Le due etichette del cappio su q_7 vengono unite in $0 + 1$.

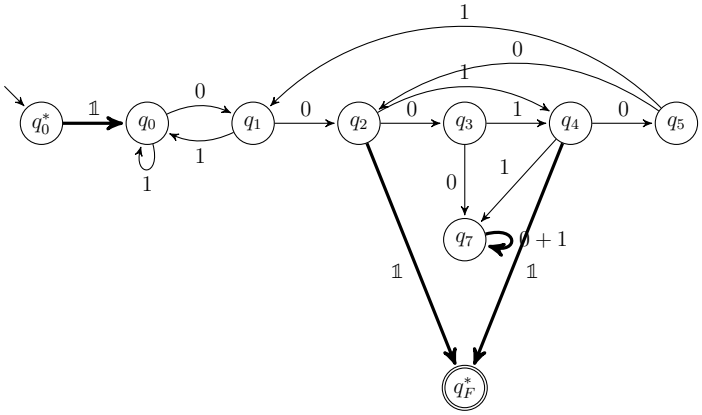


Figura 36: Automa generalizzato associato ad $\mathcal{A}$.

2. (Erase q_0) I cammini $q_0^* \to q_0 \to q_1$ e $q_1 \to q_0 \to q_1$ diventano rispettivamente $\mathbb{1}\,1^*\,0 = 1^*0$ e $1\,1^*\,0$ (cappio su q_1).

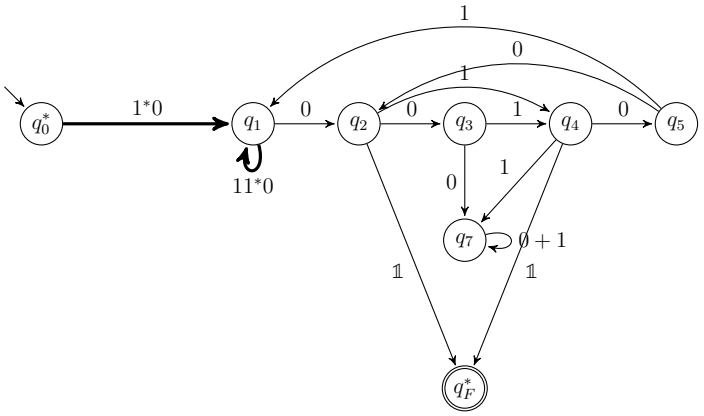


Figura 37: Dopo l'eliminazione di q_0.

3. (Erase q_1) I cammini $q_0^* \to q_1 \to q_2$ e $q_5 \to q_1 \to q_2$ (con cappio su q_1) diventano $1^*0(11^*0)^*0$ e $1(11^*0)^*0$. Quest'ultima va unita all'arco $q_5 \xrightarrow{0} q_2$ già presente: $1(11^*0)^*0 + 0$.

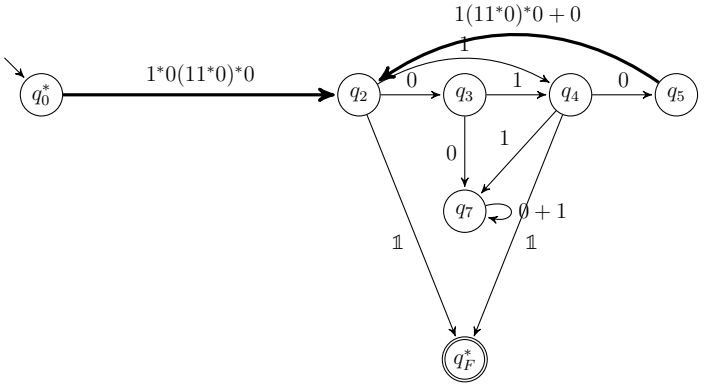


Figura 38: Dopo l'eliminazione di q_1.

4. (Erase q_7 e q_3) Lo stato q_7 è un pozzo (non raggiunge q_F^*), quindi si elimina senza modificare altri archi. Eliminando q_3 si ottiene un secondo arco $q_2 \xrightarrow{01} q_4$, che va unito a quello già presente: $1 + 01$.

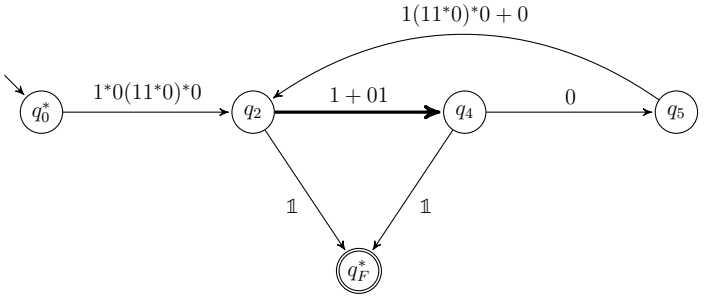


Figura 39: Dopo l'eliminazione di q_7 e q_3.

5. (Erase q_5) Il cammino $q_4 \xrightarrow{0} q_5 \to q_2$ diventa $q_4 \xrightarrow{0(1(11^*0)^*0+0)} q_2$.

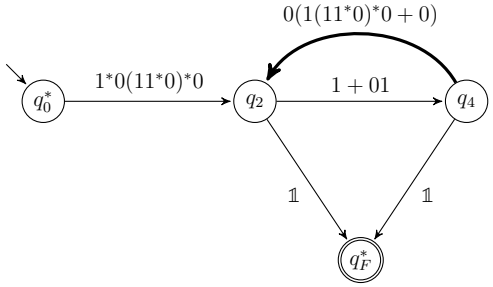


Figura 40: Dopo l'eliminazione di q_5.

6. (Erase q_4) Il cammino $q_2 \to q_4 \to q_2$ diventa un cappio su q_2, mentre $q_2 \to q_F^*$ raccoglie $\mathbb{1} + (1+01)\mathbb{1}$.

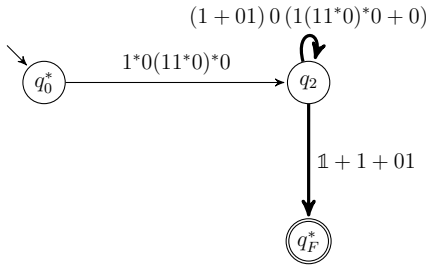


Figura 41: Dopo l'eliminazione di q_4.

7. (Erase q_2) Eliminando q_2 otteniamo un unico arco $q_0^* \to q_F^*$, la cui etichetta è l'espressione cercata:

$$r = 1^*0(11^*0)^*0\Big[(1+01)\,0\,\big(1(11^*0)^*0+0\big)\Big]^*(\mathbb{1}+1+01).$$

Affrontiamo il punto 2. Costruiamo un NFA per $r = S\,L^*\,T$, con $S = 1^*0(11^*0)^*0$, $L = (1+01)\,0\,(1(11^*0)^*0+0)$ e $T = \mathbb{1}+1+01$. Lo stato t_0 (fine di S) è il nodo del ciclo L^* ed è finale perché T contiene $\mathbb{1}$.

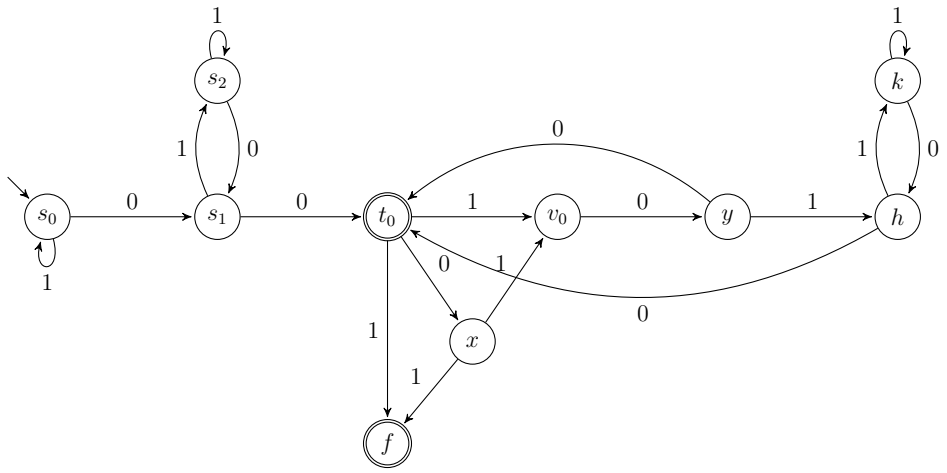
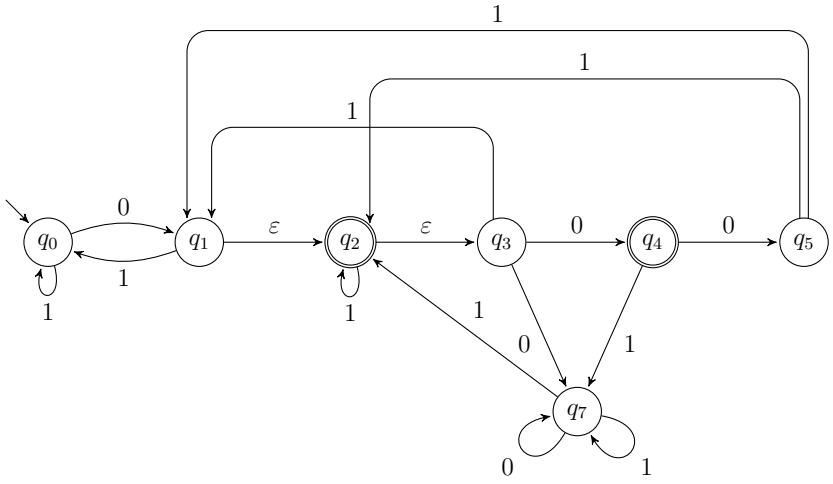


Figura 42: NFA equivalente ad $\mathcal{A}$, costruito a partire da r.

Esercizio 13.1.4. Dato il seguente automa non-deterministico:



fornire un automa deterministico equivalente (ovvero, che riconosce lo stesso insieme).

Soluzione esercizio 13.1.4. Costruiamo l'automa deterministico con la costruzione per sottoinsiemi

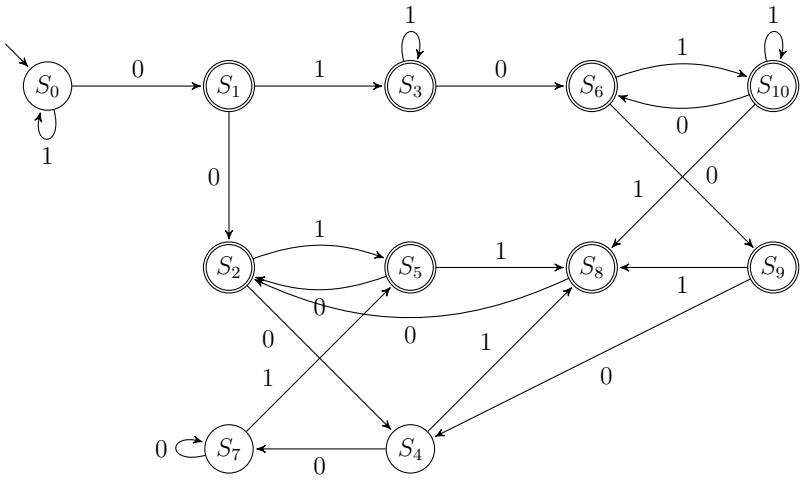
$$T\colon Q' \times A \to Q', \qquad Q' \subseteq 2^Q,$$

Lo stato iniziale è $S_0 = \{q_0\}$. Uno stato è finale se contiene q_2 oppure q_4. Calcolando la tabella si ottengono 11 stati:

	S	0	1
S_0	$\{q_0\}$	S_1	S_0
S_1	$\{q_1, q_2, q_3\}$	S_2	S_3
S_2	$\{q_4, q_7\}$	S_4	S_5
S_3	$\{q_0, q_1, q_2, q_3\}$	S_6	S_3
S_4	$\{q_5, q_7\}$	S_7	S_8
S_5	$\{q_2, q_3, q_7\}$	S_2	S_8
S_6	$\{q_1, q_2, q_3, q_4, q_7\}$	S_9	S_{10}
S_7	$\{q_7\}$	S_7	S_5
S_8	$\{q_1, q_2, q_3, q_7\}$	S_2	S_{10}
S_9	$\{q_4, q_5, q_7\}$	S_4	S_8
S_{10}	$\{q_0, q_1, q_2, q_3, q_7\}$	S_6	S_{10}

Gli stati finali sono $S_1, S_2, S_3, S_5, S_6, S_8, S_9, S_{10}$; gli unici non finali sono S_0, S_4, S_7. L'automa

deterministico è il seguente:



Esercizio 13.1.5. Sia $n \in \mathbb{N}_{\geq 1}$, $A = \{0, 1\}$ e definiamo

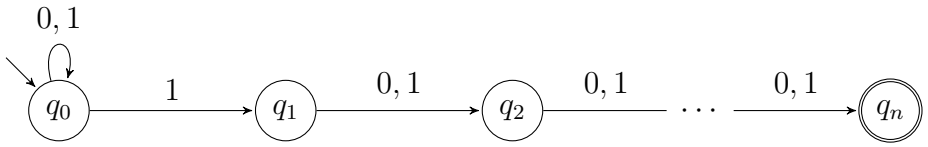
$$X_n := \{w \in A^* \mid w = w'1w'',\ |w''| = n - 1\}$$

(parole che hanno 1 in posizione $|w| - n - 1$).

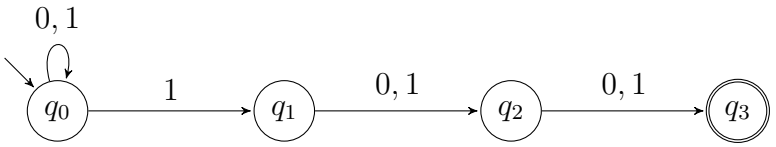
1. Mostrare che per ogni n, X_n è decidibile da un NDFA con $n+1$ stati (fornire l'automa).
2. Fissato $n = 3$, determinare un automa deterministico che decide X_3.

Soluzione esercizio 13.1.5.

1. L'automa che decide X_n è il seguente



2. Per $n = 3$ l'NDFA è il seguente.



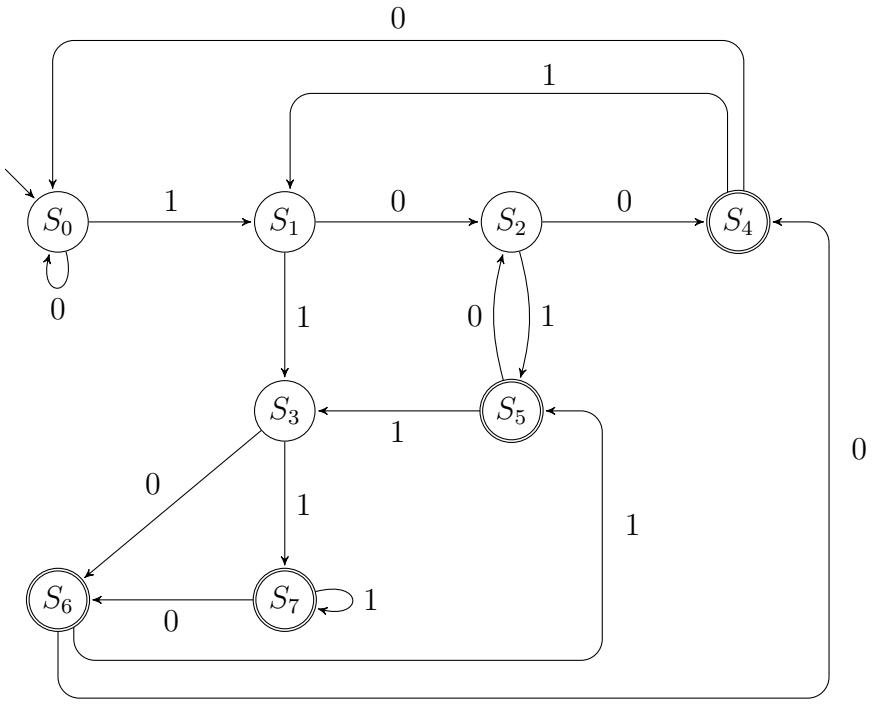
Applichiamo la costruzione per sottoinsiemi. Poniamo

$$S_0 = \{q_0\}, \quad S_1 = \{q_0, q_1\}, \quad S_2 = \{q_0, q_2\}, \quad S_3 = \{q_0, q_1, q_2\},$$
$$S_4 = \{q_0, q_3\}, \quad S_5 = \{q_0, q_1, q_3\}, \quad S_6 = \{q_0, q_2, q_3\}, \quad S_7 = \{q_0, q_1, q_2, q_3\}.$$

La funzione di transizione è

$Q' \backslash A$	0	1
$S_0 = \{q_0\}$	S_0	S_1
$S_1 = \{q_0, q_1\}$	S_2	S_3
$S_2 = \{q_0, q_2\}$	S_4	S_5
$S_3 = \{q_0, q_1, q_2\}$	S_6	S_7
$S_4 = \{q_0, q_3\}$	S_0	S_1
$S_5 = \{q_0, q_1, q_3\}$	S_2	S_3
$S_6 = \{q_0, q_2, q_3\}$	S_4	S_5
$S_7 = \{q_0, q_1, q_2, q_3\}$	S_6	S_7

Lo stato iniziale è S_0 e gli stati finali sono quelli che contengono q_3, cioè S_4, S_5, S_6, S_7. Il grafo asscoiato è il seguente:



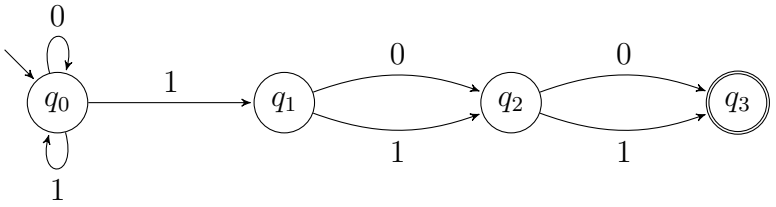
Esercizio 13.1.6. Sia X_3 come nell'esercizio 13.1.5,

1. dare l'espressione regolare e_3 tale che $[\![\mathsf{e}_3]\!] = X_3$,
2. generalizzare ad una famiglia di espressioni regolari e_n tale che $[\![\mathsf{e}_n]\!] = X_n$,

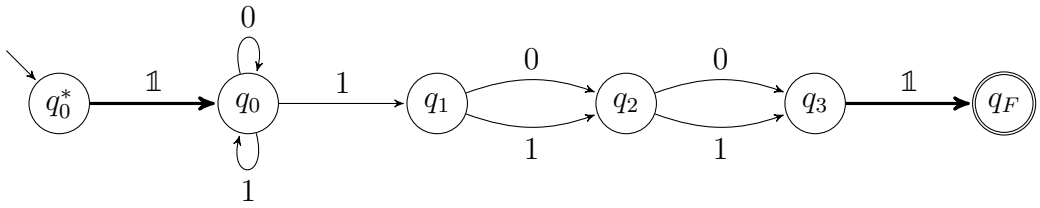
(la doppia parentesi quadra indica l'interpretazione insiemistica delle espressioni regolari).

Soluzione esercizio 13.1.6.

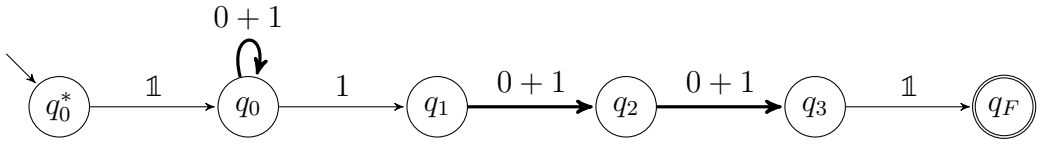
1. Per calcolare e_3 usiamo l'automa che decide X_3 e lo trasformiamo in un'espressione regolare.



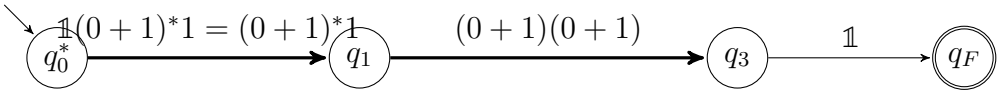
Passo 1, aggiunta di q_0^* e q_F:



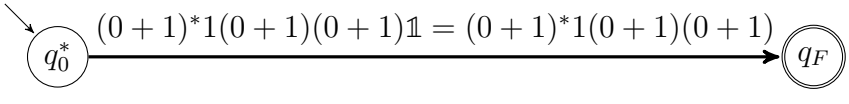
Passo 2, unione degli archi paralleli:



Passo 3, eliminazione di q_0 e q_2:



Passo 4, eliminazione di q_1 e q_3:



Quindi

$$\mathsf{e}_3 = (0+1)^*\,1\,(0+1)(0+1).$$

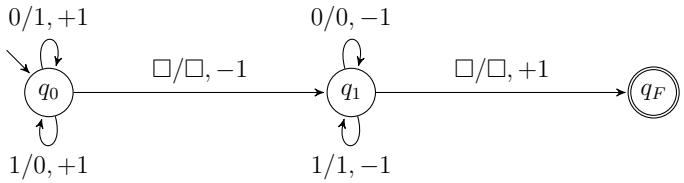
2. X_n è l'insieme delle parole il cui n-esimo simbolo da destra è 1, e si ha

$$\mathsf{e}_n = (0+1)^* \, 1 \, (0+1)^{n-1}.$$

13.2 Macchine di Turing

Esercizio 13.2.1. Dare la macchina di Turing che complementa una sequenza binaria e si riposiziona all'inizio della parola.

Soluzione esercizio 13.2.1. La macchina di turing è la seguente

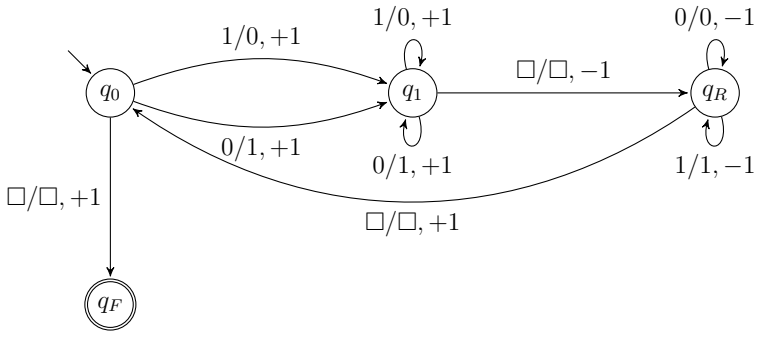


forniamo anche il codice in Wolfram Matematica

```
turing = {{
 {q0, 0} -> {q0, 1, +1},
 {q0, 1} -> {q0, 0, +1},
 {q0, B} -> {q1, B, -1},
 {q1, 0} -> {q1, 0, -1},
 {q1, 1} -> {q1, 1, -1},
 {q1, B} -> {qF, B, +1}
 }, q0, {qF}}
```

Esercizio 13.2.2. Come l'esercizio precedente, con l'unica differenza che la computazione termina solo con la parola vuota e in caso contrario, dopo essere tornata alla posizione iniziale, ripete il ciclo.

Soluzione esercizio 13.2.2. La macchina di turing è la seguente



forniamo anche il codice in Wolfram Matematica

```
mu = {{
 {q0,  0} -> {q1, 1, +1},
 {q0,  1} -> {q1, 0, +1},
 {q0, -1} -> {qF, -1, +1},
 {q1,  0} -> {q1, 1, +1},
 {q1,  1} -> {q1, 0, +1},
 {q1, -1} -> {qR, -1, -1},
 {qR,  0} -> {qR, 0, -1},
 {qR,  1} -> {qR, 1, -1},
 {qR, -1} -> {q0, -1, +1}
 }, q0, {qF}}
```

Esercizio 13.2.3. Dimostrare:

1. Se X Turing decidibile allora $A^* \setminus X$ è Turing decidibile;
2. Se $X \in \mathrm{DTIME}(T(n))$ allora $A^* \setminus X \in \mathrm{DTIME}(T(n))$.

Soluzione esercizio 13.2.3.

1. Se μ è una macchina di Turing che decide X, allora possiamo modificarla e creare una macchina di Turing μ_1 che è uguale a quella di partenza invertendo solamente gli stati q_F^0 con q_F^1. Definiamo quindi

$$\begin{array}{rccl} \mu_1: & Q \times \tilde{A} & \to & Q' \times \tilde{A} \times \{-1, 1\} \\ & (q, a) & \mapsto & \begin{cases} (p, b, m), & \text{se } \mu(q,a) = (p,b,m) \text{ e } p \neq q_F^0, q_F^1 \\ (q_F^1, b, m), & \text{se } \mu(q,a) = (p,b,m) \text{ e } p = q_F^0 \\ (q_F^0, b, m), & \text{se } \mu(q,a) = (p,b,m) \text{ e } p = q_F^1 \end{cases} \end{array}$$

   Questa chiaramente decide $A^* \setminus X$.

2. La macchina di turing μ_1 costruita nel punto precedente fa esattamente lo stesso numero di passi che compie una macchina μ. Di conseguenza, se una macchina μ decide X in tempo T, allora μ' decide $A^* \setminus X$ in tempo T.

Esercizio 13.2.4. Dimostrare:

1. Se X e Y sono Turing decidibili allora $X \cap Y$ è Turing decidibile;
2. Se $X \in \mathrm{DTIME}_2(T_1(n))$ e $Y \in \mathrm{DTIME}(T_2(n))$ allora $X \cap Y \in \mathrm{DTIME}(\max(T_1(n), T_2(n)))$.

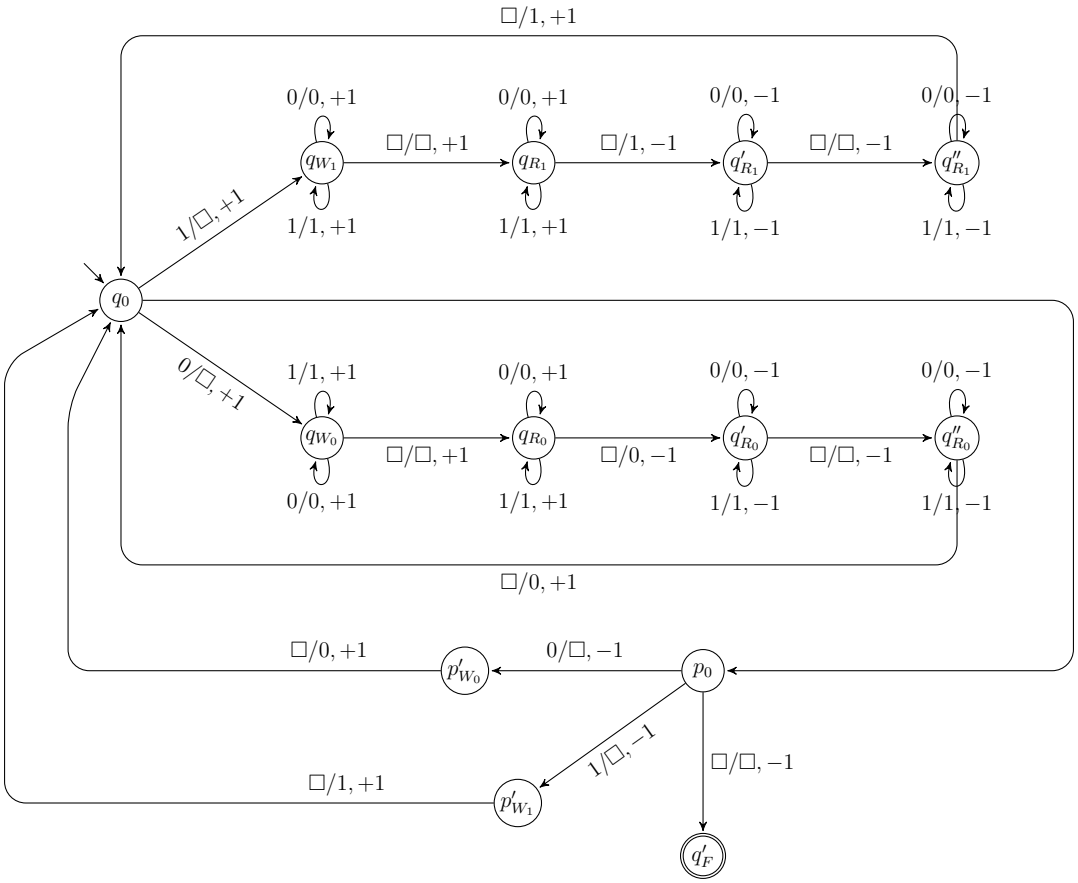
Soluzione esercizio 13.2.4.

1. Siano μ_X e μ_Y due macchina di Turing che decidono rispettivamente X e Y. Creiamo la macchina di Turing seguente:

$$\begin{array}{rccl} \overline{\mu}: & Q \times \tilde{A} & \to & Q' \times \tilde{A} \times \{-1, 1\} \\ & (q, a) & \mapsto & \begin{cases} \alpha(q, a), & \text{se } q = q_0 \\ (p, b, m), & \text{se } \mu_X(q, a) = (p, b, m) \text{ e } p \neq q_F^1 \\ (q_{F_X}^1, b, m), & \text{se } \mu_X(q, a) = (p, b, m) \text{ e } p = q_F^1 \\ (q_{F_Y}^0, b, m) & \end{cases} \end{array}$$

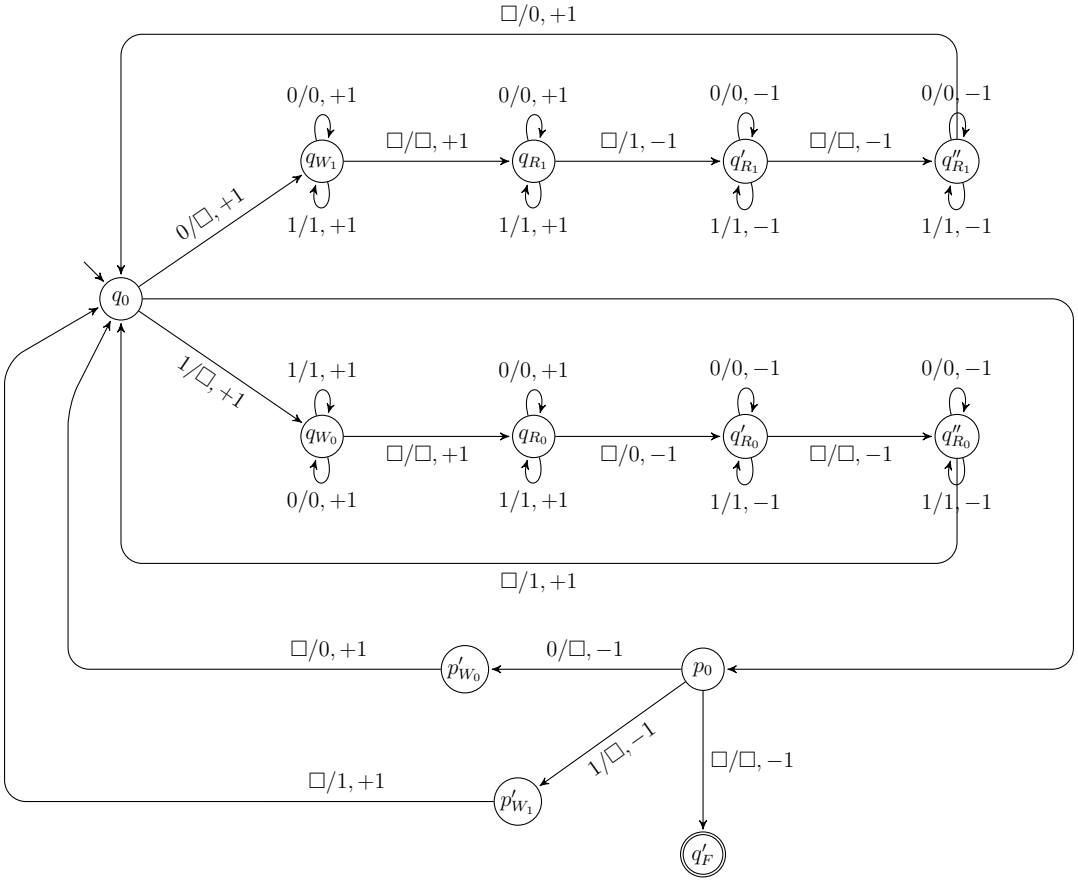
Esercizio 13.2.5. Data una parola w_1, fornire la macchina di Turing che scrive $w_1 w_1$.

Soluzione esercizio 13.2.5.



Esercizio 13.2.6. Data una parola w_1 fornire la macchina di Turing che scrive $w_1 w_2$ con w_2 il complementare di w_1.

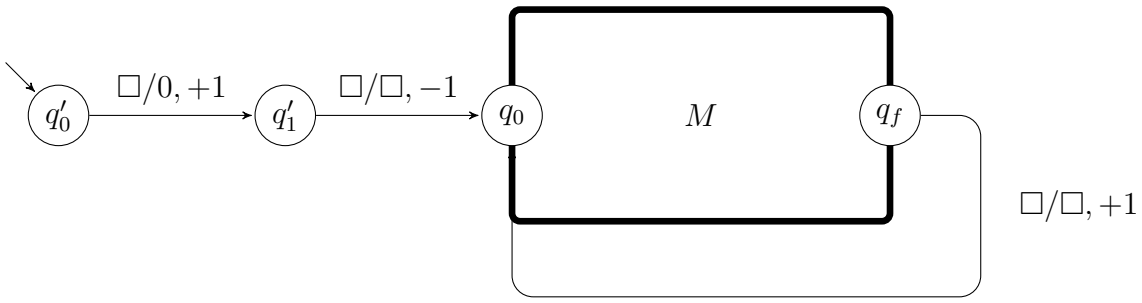
Soluzione esercizio 13.2.6.



Esercizio 13.2.7. Costruire la macchina di Turing che costruisce la sequenza costruita nel modo induttivo nel seguente modo: $w = *_{i=1}^{\infty} w_i$ dove $w_1 = 0$, e $w_i = (w_1 \dots w_{i-1})^c$, dove a^c è la sequenza complementare di a. Ovvero la sequenza:

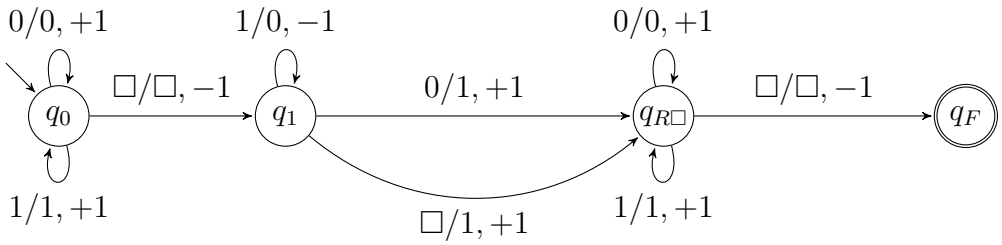
$$0110100110010110100101100110100 1 \cdots$$

Soluzione esercizio 13.2.7. Sia M la macchina dell'esercizio 13.2.6:



Esercizio 13.2.8. Macchina che scrive il successore in binario e si posiziona alla fine della parola

Soluzione esercizio 13.2.8.



14 Esoneri ed esami

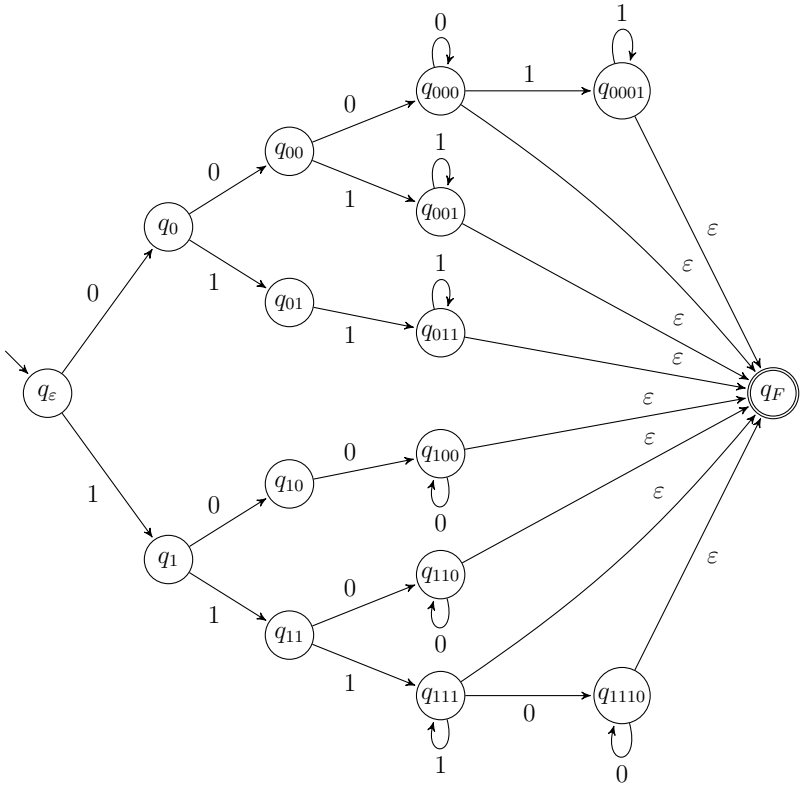
14.1 Esonero 1 - 13/11/2024

Esercizio 14.1.1. Sia $A := \{0, 1\}$ l'alfabeto binario. Stabilire se, per ogni dato n intero positivo, l'insieme $X_n \subset A^*$

$$X_n := \{a^i b^j c^k \mid i + j \geq 3,\ k \equiv 0 \bmod n \text{ e } a, b, c \in A\}$$

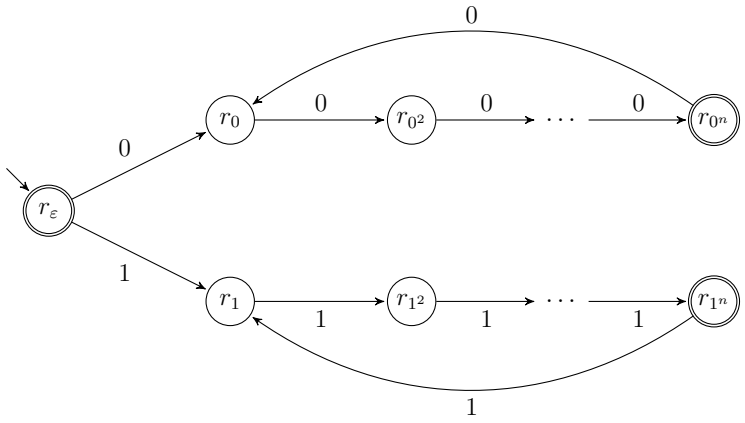
è DFA-decidibile. In caso affermativo fornire un automa che lo decide.

Soluzione esercizio 14.1.1. L'insieme X_n è DFA-decidibile per ogni n. Per decidere il primo pezzo, cioè $a^i b^j$ con $i + j \geq 3$, basta l'automa $\mathcal{A}$ qui sotto (le transizioni non disegnate vanno in uno stato pozzo).

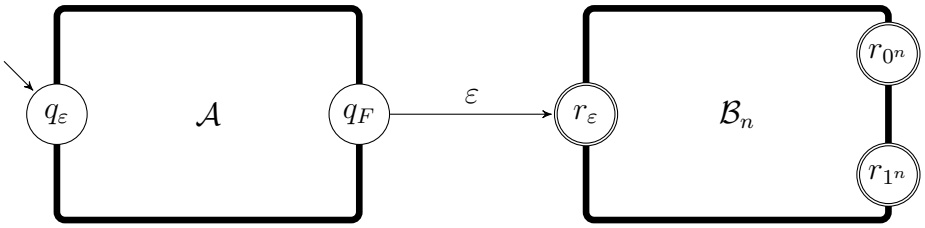


Lo concateniamo con l'automa $\mathcal{B}_n$, che accetta $\{c^k \mid k \equiv 0 \bmod n\}$. Per $n = 1$ si ha $r_{0^1} = r_{0^n}$

e $r_{1^1} = r_{1^n}$.



L'automa che decide X_n è il seguente: gli stati finali sono quelli di $\mathcal{B}_n$, ossia r_ε, r_{0^n} e r_{1^n}.



Abbiamo così mostrato che l'iniseme X_n è ϵ-NDFA decidibile e quindi è anche DFA decidibile.

Esercizio 14.1.2. Con X_n come nell'esercizio precedente,

1. dare l'espressione regolare e_3 tale che $[\![\mathsf{e}_3]\!] = X_3$,
2. generalizzare ad una famiglia di espressioni regolari e_n tale che $[\![\mathsf{e}_n]\!] = X_n$,

(la doppia parentesi quadra indica l'interpretazione insiemistica delle espressioni regolari).

Soluzione esercizio 14.1.2. Poiché X_n è la concatenazione di $\{a^i b^j \mid i+j \geq 3\}$ con $\{c^k \mid k \equiv 0 \bmod n\}$, cerchiamo un'espressione regolare per ciascuno dei due pezzi e le concateniamo.

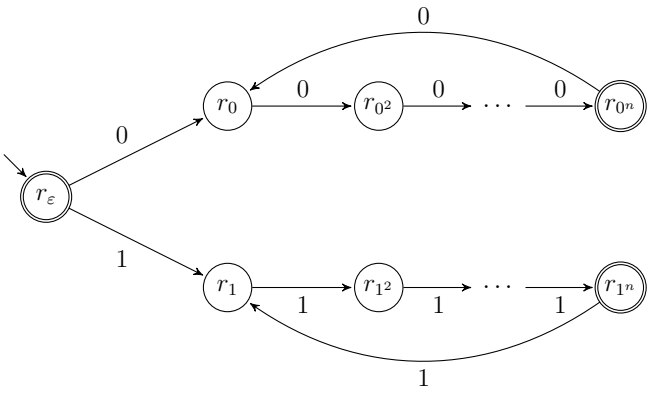
Leggendo l'automa $\mathcal{A}$ dell'esercizio precedente si ottiene

$$\mathsf{e}_{\mathcal{A}} := 0\Big(0\big(00^*1^* + 11^*\big) + 111^*\Big) + 1\Big(000^* + 1\big(00^* + 11^*0^*\big)\Big)$$

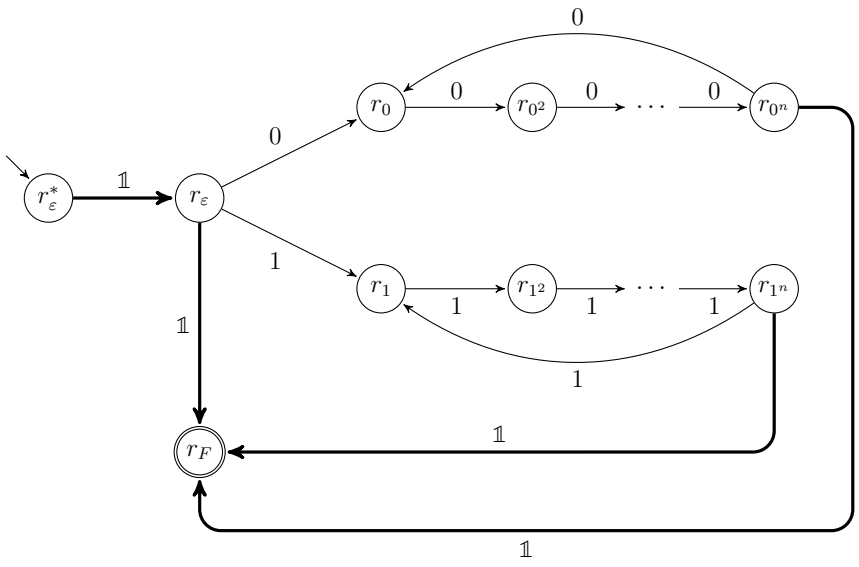
che è l'espressione per $\mathcal{A}$.

Per trovare l'espressione di $\mathcal{B}_n$ facciamo le riduzioni.

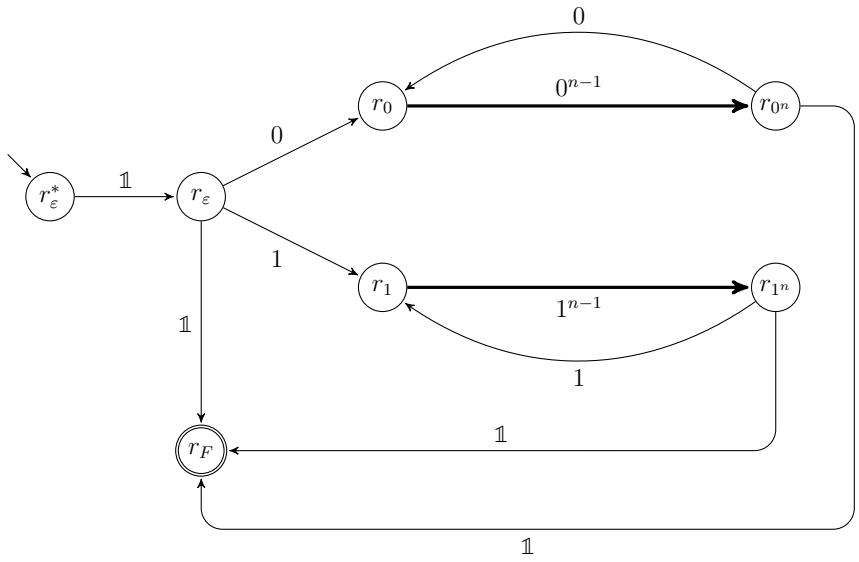
Passo 0. Partiamo da $\mathcal{B}_n$.



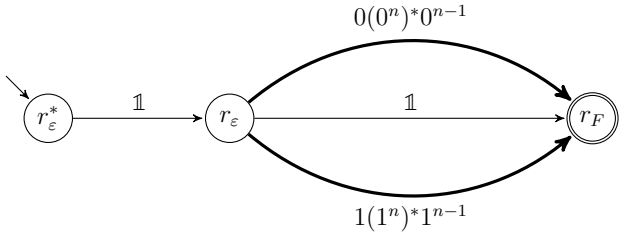
Passo 1. Aggiungiamo un nuovo stato iniziale r^*_ε e un nuovo stato finale r_F. Gli stati che erano finali, cioè r_ε, r_{0^n} e r_{1^n}, vanno in r_F con una $\mathbb{1}$-mossa.



Passo 2. Comprimiamo le due catene: da r_0 a r_{0^n} si leggono $n-1$ zeri, da r_1 a r_{1^n} si leggono $n-1$ uni.



Passo 3. Eliminiamo r_0, r_{0^n}, r_1, r_{1^n}. Da r_0 si legge 0^{n-1} e poi si esce, oppure si torna in r_0 leggendo 0 e si ripete, quindi da r_ε a r_F passando per questi stati si ha $0(0^{n-1}0)^*0^{n-1} = 0(0^n)^*0^{n-1}$. Lo stesso vale per gli uni.



Quindi $\mathsf{e}_{\mathcal{B}_n} := \varepsilon + 0(0^n)^*0^{n-1} + 1(1^n)^*1^{n-1}$, che equivale a $(0^n)^* + (1^n)^*$ (per $n = 1$ la formula resta valida con $0^0 = 1^0 = \varepsilon$).

1. Per $n = 3$ si ha $\mathsf{e}_{\mathcal{B}_3} = (000)^* + (111)^*$, quindi
$$\mathsf{e}_3 := \mathsf{e}_{\mathcal{A}}\big((000)^* + (111)^*\big).$$

2. Per ogni n la concatenazione di $\mathsf{e}_{\mathcal{A}}$ con $\mathsf{e}_{\mathcal{B}_n}$ dà
$$\mathsf{e}_n := \mathsf{e}_{\mathcal{A}}\big((0^n)^* + (1^n)^*\big),$$
e $[\![\mathsf{e}_n]\!] = [\![\mathsf{e}_{\mathcal{A}}]\!] \cdot [\![\mathsf{e}_{\mathcal{B}_n}]\!] = X_n$.

Esercizio 14.1.3. Sia $X \in \mathrm{DTIME}_1^A(c \cdot n^2)$ e sia $X' \subset X$ DFA-decidibile. Mostrare che l'insieme

$$X'' = \{w \mid ww \in X \text{ e } w \notin X'\} \in \mathrm{DTIME}_1^A(c' \cdot n^2).$$

(si precisa che l'esistenza di macchine di Turing eventualmente utilizzate, va argomentata nel dettaglio - ad esempio fornendone una rappresentazione come grafo etichettato).

Soluzione esercizio 14.1.3. Sia α una macchina du Turing mononastro che decide X in tempo cn^2 ed A un DFA che decide X'. Possiamo associare ad A una macchina di Turing mononastro β che decide X' in tempo lineare, quindi esiste $b \in \mathbb{N}$ tale che β decide X' in tempo bn^2. Indichiamo con γ la macchina che data una parola w, scrive sul nastro la parola ww, ovvero la macchina nella soluzione dell'esercizio 13.2.5 (la macchina citata è progettata per un alfabeto binario, per ottenere la macchina corretta basta aggiungere gli stati opportuni per ogni lettera di A). Indicando con δ la machcina di Turing che effettua passi indietro fino a quando non incontra due blank, possiamo definire la macchina μ come la concatenazione seguente

$$\mu := \quad \beta \to \delta \to \gamma \to \delta \to \alpha$$

Abbiamo chiaramente che la macchina μ decide l'insieme X'', dobbiamo ora capire in quanto tempo. Consideriamo quindi una parola w in input di lungezza n:

1. la macchina β compie al più bn^2 ;
2. la macchina β, non ha modificato la parola e si arresta nel primo blank dopo la parola, quindi la macchina δ compie $n+2$ passi, di cui $n+1$ per incontrare il primo blank prima della parola e poi un passo per posizionarsi all'inizio della parola;
3. a questo punto γ compie $T(n)$ passi;
4. di nuovo δ compie $(2n)+2$ passi poiché la parola è ww ed è lunga $2n$;
5. infine α compie $(cn)^2$ passi.

Quindi, abbiamo che μ decide X'' in tempo

$$bn^2 + (n+2) + T(n) + (2n+2) + c^2n^2$$

Mostriamo ora che la macchina γ termina in dn^2 passi. Per copiare la prima lettera in fondo alla parola dopo il primo blank si compiono $n+1$ passi (si scrive blank nella prima cella e ci si posiziona nella cella $n+2$), poi se ne compiono 2 per posizionarsi sull'ultima lettera della parola originale, ulteriori $n-1$ per posizionarsi sul primo blank che si incontra (che corrisponde alla posizione della prima lettera della parola w) e infine 1 per riposizionare la lettera cancellata all'inizio e tornare in q_0. Per un totale di

$$(n+1) + 2 + (n-1) + 1 = 2n + 3$$

passi. Per copiare la seconda lettera, si compiono $n-1$ passi per arrivare al primo blank e poi altri 2 passi per posizionarsi nella prima casella vuota dopo la parola già copiata. A questo punto ci vorranno 3 passi per scrivere la lettera nuova e posizionarsi sull'ultima lettera

della parola originale, ulteriori $n-2$ per posizionarsi sul primo blank che si incontra (che corrisponde alla posizione della seconda lettera della parola w) e infine 1 per riposizionare la lettera cancellata all'inizio e tornare in q_0. Per un totale di

$$(n-1)+2+3+(n-2)+1 = 2n+3$$

passi. Facendo questo ragionamento per ogni lettera della parola si trova che il costo non dipende dalla lettera, quindi la macchina scrive sul nastro la parola $w\square w$ dopo $n(2n+3)$ passi e si trova sul blank tra le due parole. A questo punto con 3 passi la macchina scala una lettera indietro, e quindi dopo $3n$ passi siamo nella situazione ww, e alla macchina serviranno solamente altri 2 passi per arrestarsi. Quindi per traslare la seconda parola si compieranno

$$3n+2$$

passi. In totale, la macchina compierà

$$n(2n+3)+3n+2$$

passi e si posizionerà sul primo blank dopo la parola. In conclusione abbiamo che μ decide X'' in tempo

$$T'(n) := bn^2 + (n+2) + n(2n+3) + 3n + 2 + (2n+2) + c^2n^2$$

per un'opportuna costante c' abbiamo chiaramente che

$$T'(n) \leq c'n^2$$

Abbiamo mostrato quindi che $X'' \in \mathrm{DTIME}_1^A(c'n^2)$.

14.2 Esonero 1 - 10/11/2023

Esercizio 14.2.1. Sia $A := \{0, 1\}$ l'alfabeto binario. Stabilire se, per ogni dato n intero positivo, l'insieme $X_n \subset A^*$

$$X_n := \{a^i b^j c^k \mid i + j \geq 3,\ k \equiv 0 \bmod n \text{ e } a, b, c \in A\}$$

è DFA-decidibile. In caso affermativo fornire un automa che lo decide.

Soluzione esercizio 14.2.1. Guarda in 14.1

Esercizio 14.2.2. Con X_n come nell'esercizio precedente,

1. dare l'espressione regolare e_3 tale che $[\![\mathsf{e}_3]\!] = X_3$,
2. generalizzare ad una famiglia di espressioni regolari e_n tale che $[\![\mathsf{e}_n]\!] = X_n$,

(la doppia parentesi quadra indica l'interpretazione insiemistica delle espressioni regolari).

Soluzione esercizio 14.2.2. Guarda in 14.1

Esercizio 14.2.3. Dati $X := \{0^k 1^{2k} \mid k \geq 0\}$ e $A := \{0, 1\}$.

1. Fornire esplicitamente una macchina di Turing che mostra
$$X \in \mathrm{DTIME}_1^A(c \cdot n^2), \quad \text{per qualche } c > 0.$$
2. Mostrare inoltre che $X \notin \mathrm{DTIME}_1^A(c \cdot n)$, per nessuna $c > 0$.

Soluzione esercizio 14.2.3.

1.

2. Sia $p > 0$ e assumiamo per assurdo $X \in \mathrm{DTIME}_1^A(c \cdot n)$ per qualche $c > 0$. Supponiamo inoltre $p = 3k$ per qualche k e consideriamo la parola $0^k 1^{2k} \in X$, di lunghezza p. Scriviamola come $0^k 1^{2k} = uxv$ con $x \neq \varepsilon$. Distinguiamo i casi su u.

   (a) Se $u = 0^t$ con $0 \leq t \leq k - 1$, distinguiamo due sottocasi.

      i. Se $x = 0^{k-t}$, allora $v = 1^{2k}$ e
      $$ux^2v = 0^t\, 0^{2(k-t)}\, 1^{2k} = 0^{t+2k-2t} 1^{2k} = 0^{2k-t} 1^{2k}.$$
      Poiché $t \leq k - 1$ si ha $2k - t > k$, quindi $ux^2v \notin X$.

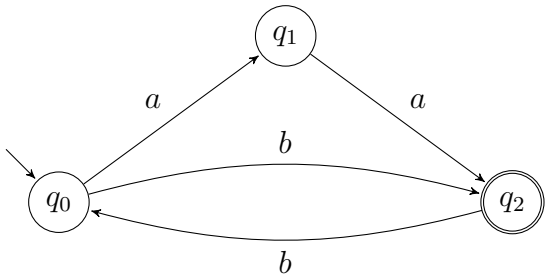
      ii. Se $x = 0^{k-t} 1^m$ con $m \geq 1$, allora $v = 1^{2k-m}$ e
      $$ux^2v = 0^t\, 0^{k-t} 1^m\, 0^{k-t} 1^m\, 1^{2k-m} = 0^k 1^m 0^{k-t} 1^{2k}.$$
      Quindi $ux^2v \notin X$, poiché c'è uno 0 in mezzo (infatti $k - t \geq 1$).

   (b) Se $u = 0^k 1^t$ con $t \geq 0$, allora $x = 1^m$ con $1 \leq m \leq 2k - t$ e $v = 1^{2k-t-m}$, quindi
   $$ux^2v = 0^k 1^t\, 1^{2m}\, 1^{2k-t-m} = 0^k 1^{m+2k} \notin X,$$
   poiché $m \geq 1$.

   In ogni caso $ux^2v \notin X$, il che è assurdo poichè contraddice il Pumping lemma. Dunque $X \notin \mathrm{DTIME}_1^A(c \cdot n)$ per nessuna $c > 0$.

14.3 Esonero 2 - 08/11/2022

Esercizio 14.3.1. Si consideri il seguente automa di alfabeto $A = \{a, b\}$ e insieme degli stati $Q = \{q_0, q_1, q_2\}$:



1. Scrivere la descrizione dell'automa in termini della tabella $T : Q \to A \to Q$, dello stato iniziale e dell'insieme F degli stati finali accettanti;

2. Dire se quello rappresentato sia un automa deterministico oppure un automa non deterministico, nel secondo caso applicare la procedura per calcolare l'automa deterministico equivalente;

3. Costruire l'espressione regolare corrispondente all'automa (deterministico) seguendo la procedura di conversione;

4. Calcolare la matrice M corrispondente all'automa deterministico calcolato, separare la matrice M in due matrici di adiacenza (con elementi 0/1) relative al grafo delle transizioni etichettate con a e quelle etichettate con b, in modo tale che: $M = aM_a + bM_b$;

5. Calcolare M_a^2, M_a^3, M_b^2, M_b^3;

6. Calcolare, con l'aiuto del punto precedente, l'insieme X deciso dall'automa tramite la formula dell'esecuzione.

Soluzione esercizio 14.3.1.

1. Lo stato iniziale è q_0 e $F = \{q_2\}$. La tabella delle transizioni è

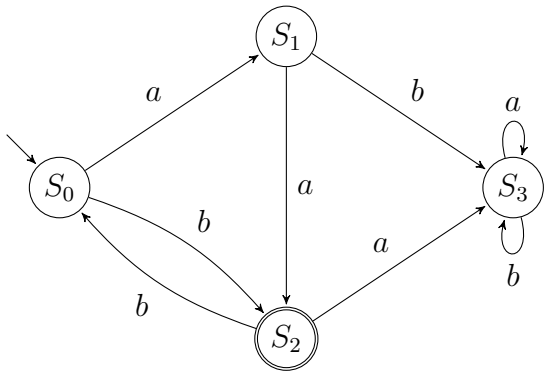
T	a	b
q_0	q_1	q_2
q_1	q_2	$\perp$
q_2	$\perp$	q_0

2. La funzione T non è totale, poiché $T(q_1, b) = \perp$ e $T(q_2, a) = \perp$. Applichiamo la procedura di determinizzazione (costruzione per sottoinsiemi), ponendo $S_0 = \{q_0\}$, $S_1 = \{q_1\}$, $S_2 = \{q_2\}$, $S_3 = \emptyset$:

T'	a	b
S_0	S_1	S_2
S_1	S_2	S_3
S_2	S_3	S_0
S_3	S_3	S_3

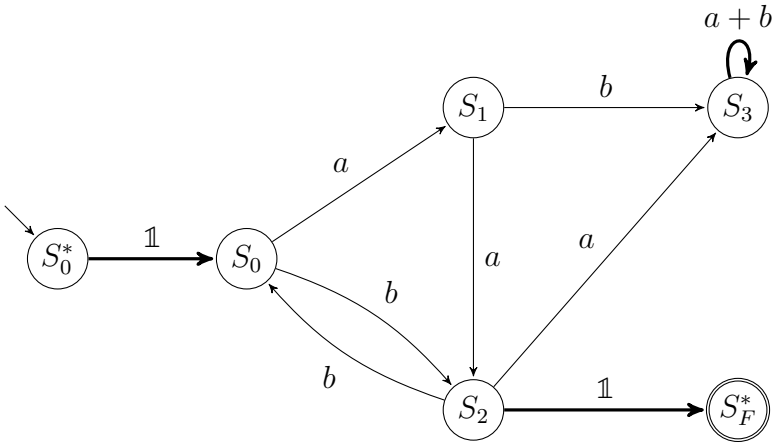
$F' = \{S_2\}$

L'automa deterministico equivalente è

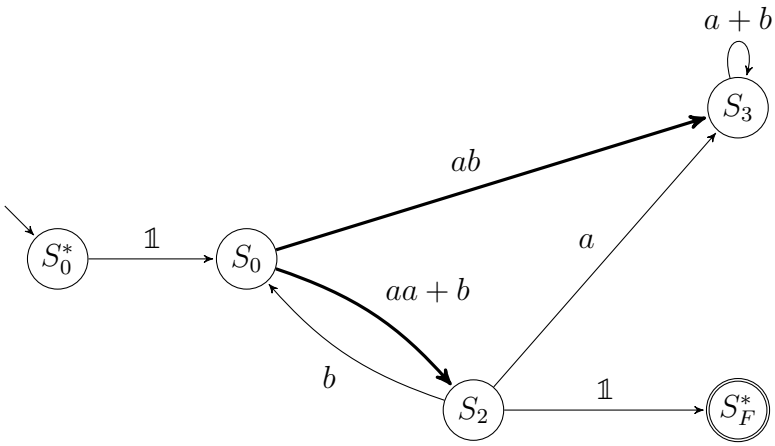


3. Applichiamo la procedura di conversione in espressione regolare.

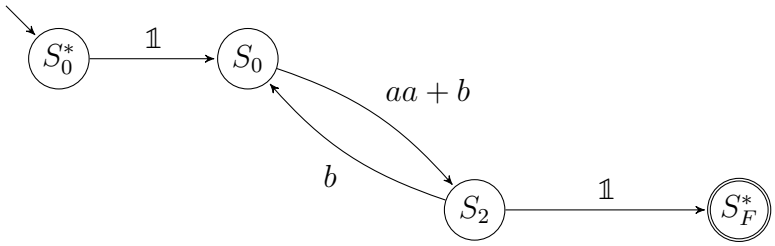
   (a) Setup: aggiungiamo il nuovo stato iniziale S_0^*, il nuovo stato finale S_F^* e riuniamo i due cappi di S_3.



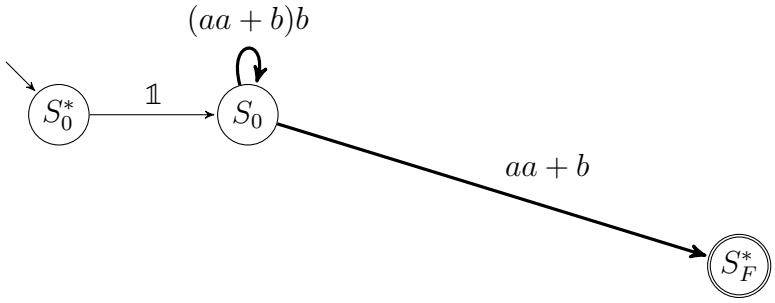
   (b) Eliminiamo S_1: i cammini $S_0 \xrightarrow{a} S_1 \xrightarrow{a} S_2$ e $S_0 \xrightarrow{a} S_1 \xrightarrow{b} S_3$ diventano rispettivamente aa (da riunire con b) e ab.



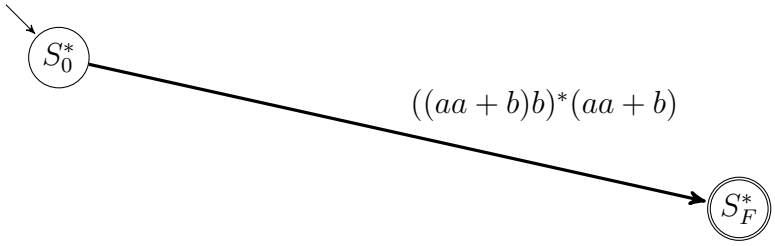
(c) Eliminiamo S_3: da S_3 non si raggiunge alcuno stato finale, quindi lo stato e le transizioni che lo coinvolgono non influenzano il linguaggio.



(d) Eliminiamo S_2: il cammino $S_0 \to S_2 \to S_0$ diventa un cappio $(aa+b)b$ e il cammino $S_0 \to S_2 \to S_F^*$ diventa $aa+b$.



(e) Eliminiamo S_0: il cappio diventa una stella di Kleene.



L'espressione regolare cercata è quindi $((aa+b)b)^*(aa+b)$.

4. Ordinando gli stati come S_0, S_1, S_2, S_3, la matrice dell'automa deterministico è

$$M = \begin{pmatrix} \bot & a & b & \bot \\ \bot & \bot & a & b \\ b & \bot & \bot & a \\ \bot & \bot & \bot & a+b \end{pmatrix}.$$

Separando le transizioni etichettate con a da quelle etichettate con b si ottiene $M = aM_a + bM_b$ con

$$M_a = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

5. Calcolando i prodotti si ottiene

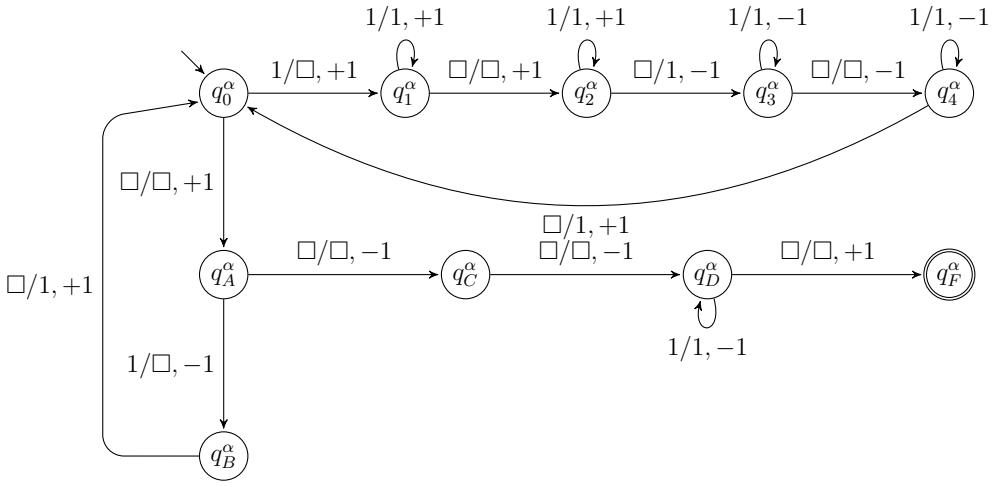
$$M_a^2 = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_a^3 = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$M_b^2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad M_b^3 = M_b = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Esercizio 14.3.2. Fornire una macchina di Turing μ di alfabeto $A = \{1\}$ (unario) che decide l'insieme

$$X = \{1^n \mid n = 2^k \text{ per qualche } k\}.$$

Soluzione esercizio 14.3.2. Sia α la macchina che, partendo da 1^m, scrive 1^{2m} (raddoppia il contenuto del nastro) e riporta la testina sulla prima cella:



La macchina α funziona in due fasi. Nella prima, a partire da 1^m copia il contenuto a destra, separandolo con una cella vuota: q_0^α cancella il primo 1 non ancora copiato, q_1^α e q_2^α raggiungono la fine della copia e vi scrivono un 1, poi q_3^α e q_4^α tornano indietro e riscrivono l'1 cancellato, passando al successivo. Quando l'originale è esaurito si ottiene $1^m \square 1^m$. Nella seconda fase q_A^α e q_B^α spostano la copia di una cella a sinistra, un 1 alla volta, eliminando la cella vuota, così sul nastro rimane 1^{2m}. Infine q_C^α e q_D^α riportano la testina sulla prima cella e la macchina si ferma in q_F^α.
Definiamo la macchina a due nastri μ come segue, dove Q^α è l'insieme degli stati di α e $\alpha(q, b) = (\alpha(q, b)_{\pi_1}, \alpha(q, b)_{\pi_2}, \alpha(q, b)_{\pi_3})$ indica la terna (nuovo stato, simbolo scritto, movi-

mento) data da α:

$$\mu: \quad Q \times A'^2 \longrightarrow Q \times A'^2 \times \{+1,0,-1\}^2$$

$$(q,(a,b)) \mapsto \begin{cases} (q_F^0,(a,b),(0,0)) & \text{se } q = q_0,\ a = \square \\ (q_1,(a,1),(+1,0)) & \text{se } q = q_0,\ a = 1 \\ (q_F^1,(a,b),(0,0)) & \text{se } q = q_1,\ a = \square \\ (q_2,(a,b),(-1,0)) & \text{se } q = q_1,\ a \neq \square \\ (q_0^\alpha,(a,b),(0,0)) & \text{se } q = q_2 \\ \big(\alpha(q,b)_{\pi_1},(a,\alpha(q,b)_{\pi_2}),(0,\alpha(q,b)_{\pi_3})\big) & \text{se } q \in Q^\alpha \setminus \{q_F^\alpha\} \\ (q_R,(a,b),(0,0)) & \text{se } q = q_F^\alpha \\ (q_R,(a,b),(-1,0)) & \text{se } q = q_R,\ a \neq \square \\ (q_A,(a,b),(+1,0)) & \text{se } q = q_R,\ a = \square \\ (q_A,(a,b),(+1,+1)) & \text{se } q = q_A,\ a,b \neq \square \\ (q_F^0,(a,b),(0,0)) & \text{se } q = q_A,\ a = \square,\ b = 1 \\ (q_{R'},(a,b),(-1,-1)) & \text{se } q = q_A,\ a = 1,\ b = \square \\ (q_{R'},(a,b),(-1,-1)) & \text{se } q = q_{R'},\ a,b \neq \square \\ (q_0^\alpha,(a,b),(+1,+1)) & \text{se } q = q_{R'},\ a,b = \square \\ (q_F^1,(a,b),(0,0)) & \text{se } q = q_A,\ a,b = \square \end{cases}$$

La macchina μ usa due nastri: sul primo c'è l'input 1^n e sul secondo si costruisce la successione delle potenze di 2. Se $n = 0$ rifiuta. Altrimenti scrive un 1 sul secondo nastro e, se $n = 1$, accetta ($2^0 = 1$). In caso contrario esegue α sul secondo nastro, ottenendo 1^2, e confronta i due nastri avanzando in parallelo. Se i due nastri finiscono insieme accetta (n è la potenza di 2 appena raggiunta). Se finisce prima il primo, n è compreso strettamente tra due potenze di 2 consecutive e rifiuta. Se finisce prima il secondo, riavvolge le testine, raddoppia di nuovo il secondo nastro con α e ripete il confronto. Il procedimento termina perché la lunghezza del secondo nastro raddoppia a ogni giro, quindi prima o poi supera n.

Esercizio 14.3.3. Sia $A = \{0, 1\}$ l'alfabeto binario; per $w \in A^*$ sia $|w|_0$ (rispettivamente $|w|_1$) il numero di occorrenze dello 0 (rispettivamente dell'1) in w. Si considerino i seguenti insiemi:

$$X = \{0^n 1^n \mid n \geq 0\}, \qquad X' = \{0^n 1^m \mid n \geq 0,\ m \geq 0\}, \qquad Y = \{w \in A^* \mid |w|_0 = |w|_1\}.$$

Mostrare che:

1. X non è regolare poiché non soddisfa la condizione necessaria del Pumping Lemma;
2. X' è regolare poiché è interpretazione di un'espressione regolare (quale?);
3. $X = X' \cap Y$;
4. Y non è regolare.